

University of Manchester
School of Computer Science
Project Report 2025

**Predicting Conversion from Subjective Cognitive Decline to
Mild Cognitive Impairment or Alzheimer's Disease using
Ensemble Machine Learning**

Author: Vishal Kandasamy Sekar

Supervisor: Dr. Fumie Costen

Abstract

Predicting Conversion from Subjective Cognitive Decline to Mild Cognitive Impairment or Alzheimer's Disease using Ensemble Machine Learning

Author: Vishal Kandasamy Sekar

The neurodegenerative process underlying Alzheimer's Disease (AD) begins at least 15–20 years before the onset of noticeable cognitive symptoms. Early identification of individuals who progress from Subjective Cognitive Decline (SCD) to Mild Cognitive Impairment (MCI) or AD (Dementia) is therefore critical for timely intervention and for selecting participants in studies on AD progression and treatment development.

This project develops ensemble machine learning models to predict the conversion from SCD to MCI/AD within 10-year and 7-year time windows. The input predictors include demographic information, cognitive and neuropsychological test scores, genetic markers, and imaging and biomarker data from PET, MRI, CSF, and plasma. Several imputation techniques, such as Mean, Median, Mode, MICE, and Random Forest, were evaluated, with the best-performing method selected to address missing data. The class imbalance was addressed using various resampling techniques, including Random Oversampling, ADASYN, SMOTE, Borderline SMOTE, Random Undersampling, and Clustered Centroids Undersampling. Feature selection involved both manually defined groups based on domain knowledge and criteria such as the absence of multicollinearity, as well as automated selection using Recursive Feature Elimination (RFE) with Logistic Regression and Random Forest. This approach facilitated the evaluation of the trade-offs between data acquisition cost and model performance.

An extensive set of ensemble configurations was evaluated for model development, comprising 494 stacking ensembles, 247 voting ensembles, and 8 bagging ensembles. These ensembles were constructed using combinations of eight base models: Logistic Regression, Support Vector Classifier (SVC), k-Nearest Neighbours (kNN), Multi-layer Perceptron (MLP), Decision Tree, Gaussian Naive Bayes, Random Forest, and Extreme Gradient Boosting (XGBoost).

Random Forest imputation yielded the best performance among the evaluated techniques, while SMOTE was the most effective method for addressing class imbalance. The final ensemble model for predicting 10-year conversion achieved an average accuracy of 0.804 ($\pm$ 0.053) and a ROC-AUC of 0.848 ($\pm$ 0.033) based on 5-fold cross-validation. The 7-year model achieved an average accuracy of 0.771 ($\pm$ 0.028) and a ROC-AUC of 0.818 ($\pm$ 0.043). Manual feature group analysis for the 10-year dataset confirmed that all 50 features extracted from the ADNI database provided non-redundant and relevant information for predicting progression from SCD to MCI/AD. Automatic feature selection using RFE identified the most informative features as FDG, AV45, CSF ABETA, Plasma ABETA42/40 ratio, ADAS13, DIGITSCOR, MoCA, EcogPtPlan, EcogPtTotal, mPACCtrailsB, and mPACCdigit.

Supervisor: Dr. Fumie Costen

Acknowledgements

I would like to express my deepest gratitude to Dr. Fumie Costen, whose invaluable support and guidance made this project possible. I am especially thankful for her expertise, encouragement, and unwavering moral support throughout the process. It was truly a pleasure working with her.

I would also like to sincerely thank my family and friends for their endless patience, understanding, and support throughout my four years at university. Their encouragement and willingness to listen made this journey all the more manageable and meaningful.

In addition, I would like to acknowledge the use of generative AI tools, which were employed to assist in reviewing and refining the grammatical structure of this report.

ADNI Acknowledgement

Data used in preparation of this article were obtained from the Alzheimer's Disease Neuroimaging Initiative (ADNI) database (adni.loni.usc.edu). As such, the investigators within the ADNI contributed to the design and implementation of ADNI and/or provided data but did not participate in analysis or writing of this report. A complete listing of ADNI investigators can be found at: http://adni.loni.usc.edu/wp-content/uploads/how_to_apply/ADNI_Acknowledgement_List.pdf.

Declaration

No portion of the work referred to in this thesis has been submitted in support of an application for another degree or qualification of this or any other university or other institute of learning.

Copyright

- i. The author of this thesis (including any appendices and/or schedules to this thesis) owns certain copyright or related rights in it (the “Copyright”) and s/he has given The University of Manchester certain rights to use such Copyright, including for administrative purposes.
- ii. Copies of this thesis, either in full or in extracts and whether in hard or electronic copy, may be made **only** in accordance with the Copyright, Designs and Patents Act 1988 (as amended) and regulations issued under it or, where appropriate, in accordance with licensing agreements which the University has from time to time. This page must form part of any such copies made.
- iii. The ownership of certain Copyright, patents, designs, trade marks and other intellectual property (the “Intellectual Property”) and any reproductions of copyright works in the thesis, for example graphs and tables (“Reproductions”), which may be described in this thesis, may not be owned by the author and may be owned by third parties. Such Intellectual Property and Reproductions cannot and must not be made available for use without the prior written permission of the owner(s) of the relevant Intellectual Property and/or Reproductions.
- iv. Further information on the conditions under which disclosure, publication and commercialisation of this thesis, the Copyright and any Intellectual Property and/or Reproductions described in it may take place is available in the University IP Policy (see <http://documents.manchester.ac.uk/DocuInfo.aspx?DocID=24420>), in any relevant Thesis restriction declarations deposited in the University Library, The University Library’s regulations (see <http://www.library.manchester.ac.uk/about/regulations/>) and in The University’s policy on presentation of Theses

Contents

1	Introduction	1
1.1	Motivation	1
1.2	Aims and Objectives	1
1.2.1	Aims	1
1.2.2	Objectives	2
1.3	Evaluation Strategy	3
1.4	Report Structure	3
1.4.1	Chapter 2 - Background	3
1.4.2	Chapter 3 - Literature Review	3
1.4.3	Chapter 4 - Preprocessing	3
1.4.4	Chapter 5 - Model Construction	4
1.4.5	Chapter 6 - Results	4
1.4.6	Chapter 7 - Discussion	4
1.4.7	Chapter 8 - Conclusion and Future Work	4
2	Background	5
2.1	Alzheimer's Disease (AD)	5
2.1.1	AD Continuum	6
2.1.2	Diagnosis	7
2.2	Machine Learning (ML)	9
2.2.1	Introduction to ML	9
2.2.2	ML Process	10
2.2.3	Base Models	11
2.2.4	Ensembling Techniques	16
2.2.5	Base Ensemble Models	18
2.2.6	Imputation Techniques	20
2.2.7	Resampling Techniques	21
2.2.8	Evaluation	24
3	Literature Review	25
3.1	Preprocessing	25
3.1.1	Model Input	25
3.1.2	Handling Missing Data	25
3.1.3	Handling Class Imbalance	26
3.2	Classification Methodology	26
3.2.1	Models	26
3.2.2	Evaluation	26

3.3	Feature Selection	26
3.3.1	Multicollinearity	26
3.3.2	Recursive Feature Elimination (RFE)	27
3.4	Summary	28
4	Preprocessing	29
4.1	Development Environment	29
4.2	Data Collection	29
4.3	Data Exploration	30
4.3.1	Selecting Features	30
4.3.2	Selecting Patients	42
4.4	Training and Test Datasets	43
4.5	Missing Data	43
4.5.1	Mean + Mode Imputation	44
4.5.2	Median + Mode Imputation	44
4.5.3	Random Forest Imputation	44
4.5.4	MICE Imputation	45
4.6	Class Imbalance	45
4.6.1	Implementation of Class-balancing Techniques	45
4.6.2	Data Leakage	46
4.7	Feature Selection	46
4.7.1	Manually Selected Feature Groups	46
4.7.2	Automatic Feature Selection	48
5	Model Construction	49
5.1	Hyperparameter Tuning	49
5.2	Logistic Regression (LR)	50
5.3	Support Vector Classifier (SVC)	50
5.4	k-Nearest Neighbours (kNN)	50
5.5	Multi-Layer Perceptron (MLP)	51
5.6	Decision Tree (DT)	51
5.7	Gaussian Naive Bayes (Gaussian NB)	51
5.8	Random Forest (RF)	52
5.9	Extreme Gradient Boosting (XGBoost)	52
5.10	Ensemble Model	52
5.10.1	Combinations of Base Models	52
5.10.2	Stacking	53
5.10.3	Voting (Soft)	54
5.10.4	Bagging	56
5.10.5	Selecting Best Ensemble Model	57
6	Results	58
6.1	Imputation Datasets	59
6.1.1	Best Imputation Method	59
6.1.2	Detailed Performance of Best Imputation Method: Random Forest Imputation	59
6.2	Class-Balanced Datasets	62
6.2.1	Best Class-Balancing Method	63
6.2.2	Detailed Performance of Best Class-balancing Method: SMOTE	63

6.3	Feature Selection	67
6.3.1	Manually Selected Feature Groups	67
6.3.2	Automatic Feature Selection	68
6.4	Importance of Individual Features	70
6.5	Conversion within 7 years	72
7	Discussion	74
7.1	Discussion of Results	74
7.2	Discussion of Methods	75
7.3	Evaluation of Objectives	75
7.4	Personal Reflection	76
8	Conclusion and Future Work	77
8.1	Conclusions	77
8.2	Future Work	78
	References	79
A	ADNI Criteria	88
B	Combining Plasma data with ADNIMERGE dataset	90
C	Constructing the 10-year conversion dataset	93
D	Code for Finding Multicollinear Features	96
E	Selecting the Best Ensemble Model	97
F	Ensemble Model Configurations	100
F.1	Stacking Model Configurations	100
F.2	Voting Model Configurations	115
F.3	Bagging Model Configurations	123
G	Best Ensemble Models	124
G.1	Best Ensemble Models of Imputed Datasets	124
G.1.1	Multiple Imputations by Chained Equations (MICE) with Bayesian Ridge Estimator	124
G.1.2	Mean + Mode Imputation	125
G.1.3	Median + Mode Imputation	126
G.2	Best Ensemble Models of Class-balanced Datasets	127
G.2.1	Random Oversampling (ROS)	127
G.2.2	Adaptive Synthetic Sampling (ADASYN)	128
G.2.3	Borderline SMOTE (BL-SMOTE)	129
G.2.4	Random Undersampling (RUS)	130
G.2.5	Clustered Centroids Undersampling (CCUS)	131
G.3	Best Ensemble Models of Manually Selected Feature Groups	132
G.3.1	Group A - All Features	132
G.3.2	Group B - Base Features	132
G.3.3	Group C - Base + CSF Features	133
G.3.4	Group D - Base + PET Features	134

G.3.5	Group E - Base + MRI Features	135
G.3.6	Group F - Base + Plasma Features	136
G.3.7	Group G - Base + APOE4 Features	137
G.3.8	Group H - All Features Except MRI and PET Data	138
G.3.9	Group I - Non-collinear Features selected with Multicollinearity check	139

Word Count: 14982

List of Figures

2.1	The physiological structure of the brain and neurons in (a) healthy brain and (b) AD brain (Breijyeh and Karaman 2020).	5
2.2	The continuum of AD (Sperling et al. 2011).	6
2.3	Graphic representation of the proposed staging framework for preclinical AD (Sperling et al. 2011).	6
2.4	Hypothetical timeline of functional and biomarker changes across the AD continuum (Sperling et al. 2011).	7
2.5	Supervised Learning (Koteluk et al. 2021).	9
2.6	ML Process (Koteluk et al. 2021).	10
2.7	Support Vector Classifier (Cervantes et al. 2020).	12
2.8	SVC with soft margins (Cervantes et al. 2020).	12
2.9	kNN with two classes (Taunk et al. 2019).	13
2.10	Multi-Layer Perceptron (Bisong 2019).	14
2.11	Decision Tree (Mienye and Jere 2024).	15
2.12	Stacking Ensembling (Kalirane 2025).	16
2.13	Hard vs Soft Voting Ensembling (Parashar 2022).	17
2.14	Bagging Ensembling (Kalirane 2025).	17
2.15	Random Forest Model (Omarzai 2024a).	18
2.16	Random Forest Model (Omarzai 2024b).	19
2.17	ADASYN Algorithm (Fan et al. 2020).	21
2.18	SMOTE Algorithm (Mishra et al. 2022).	22
2.19	Borderline Samples (Kummer et al. 2022).	23
2.20	CCUS Algorithm (Charfaoui 2023).	23
3.1	RFE algorithm with RF Model (Shi et al. 2021).	27
4.1	Density plots of numerical features in 10-year conversion dataset	34
4.1	Density plots of numerical features in 10-year conversion dataset (continued)	35
4.1	Density plots of numerical features in 10-year conversion dataset (continued)	36
4.1	Density plots of numerical features in 10-year conversion dataset (continued)	37
4.1	Density plots of numerical features in 10-year conversion dataset (continued)	38
4.1	Density plots of numerical features in 10-year conversion dataset (continued)	39
4.1	Density plots of numerical features in 10-year conversion dataset (continued)	40
4.1	Density plots of numerical features in 10-year conversion dataset (continued)	41
4.2	10-year Conversion Dataset: Only resampling the training set to prevent data leakage.	46
5.1	Selecting optimal hyperparameter configuration of base models.	49
5.2	Structure of Stacking Ensemble Model	54

5.3	Structure of Soft Voting Ensemble Model.	55
5.4	Structure of Bagging Ensemble Model	56
5.5	Selecting the best ensemble model from all the ensemble model configurations (listed in Appendix F).	57
6.1	Final Evaluation Method for Each Machine Learning Model Developed. M_i denotes evaluation metric being calculated in the i^{th} iteration.	58
6.2	Best Ensemble Model for dataset imputed using Random Forest: Soft Voting of LR, kNN and MLP base models.	60
6.3	Average Confusion Matrix of Best Ensemble Model of dataset imputed using RF from 5-fold CV	62
6.4	Average ROC curve of Best Ensemble Model for dataset imputed using RF from 5-fold CV. (Note: std. dev. refers to standard deviation)	62
6.5	Best Ensemble Model for dataset imputed using Random Forest and class- balanced using SMOTE: Stacking of MLP and LR with RF as the final estimator.	64
6.6	Average Confusion Matrix of Best Ensemble Model for dataset class-balanced using SMOTE from 5-fold CV	66
6.7	Average ROC curve of Best Ensemble Model for dataset class-balanced using SMOTE from 5-fold CV. (Note: std. dev. refers to standard deviation)	67
6.8	Best Ensemble Model for dataset for predicting conversion within 7 years: Stacking of MLP and RF with LR as the final estimator.	73
G.1	Best Ensemble Model for MICE Imputation: Voting	124
G.2	Best Ensemble Model for Mean+Mode Imputation: Voting	125
G.3	Best Ensemble Model for Median+Mode Imputation: Voting	126
G.4	Best Ensemble Model for dataset class-balanced using ROS Method: Voting	127
G.5	Best Ensemble Model for dataset class-balanced using ADASYN Method: Voting	129
G.6	Best Ensemble Model for dataset class-balanced using Borderline SMOTE Method: Voting	130
G.7	Best Ensemble Model for dataset class-balanced using RUS Method: Stacking	131
G.8	Best Ensemble Model for dataset class-balanced using CCUS Method: Stacking	132
G.9	Best Ensemble Model for Base Features: Stacking	133
G.10	Best Ensemble Model for Base + CSF Features: Voting	134
G.11	Best Ensemble Model for Base + PET Features: Stacking	135
G.12	Best Ensemble Model for Base + MRI Features: Voting	136
G.13	Best Ensemble Model for Base + PET Features: Stacking	137
G.14	Best Ensemble Model for Base + APOE4 Features: Voting	138
G.15	Best Ensemble Model for All Features except MRI and PET Data: Stacking	139
G.16	Best Ensemble Model for Non-collinear Features: Stacking	140

List of Tables

2.1	Evaluation metrics used for model assessment	24
3.1	Summary of Previous Studies on Predicting AD Conversion	28
4.1	Python libraries used in the development	29
4.2	Descriptive statistics of Continuous Features in 10-year Conversion Data (Note: Std is Standard Deviation)	32
4.3	Descriptive statistics of Categorical Features in 10-year Conversion Data	33
4.4	Percentage of Missing Data at Baseline Visit Vs All visits in 10-year Conversion Data	42
4.5	Resampling techniques and their respective library implementations	45
4.6	Feature Groups and Corresponding Features	47
5.1	Hyperparameters and their values tested in tuning of LR Model	50
5.2	Hyperparameters and their values tested in tuning of SVC Model	50
5.3	Hyperparameters and their values tested in tuning of kNN Model	51
5.4	Hyperparameters and their values tested in tuning of MLP Model	51
5.5	Hyperparameters and their values tested in tuning of DT Model	51
5.6	Hyperparameters and their values tested in tuning of Gaussian NB Model	52
5.7	Hyperparameters and their values tested in tuning of RF Model	52
5.8	Hyperparameters and their values tested in tuning of XGBoost Model	52
6.1	Performance Comparison of Ensemble Models of different Imputation Methods	59
6.2	Optimal hyperparameters of Base Models used within Best Ensemble Model for dataset imputed using Random Forest	60
6.3	Performance Comparison of Hyperparameter-tuned Base Models for dataset imputed using Random Forest	60
6.4	Performance Metrics of Best Ensemble Model of dataset imputed using Random Forest	61
6.5	Performance Comparison of Ensemble Models of Different Class-Balancing Methods	63
6.6	Optimal hyperparameters of Base Models used in Best Ensemble Model for dataset class-balanced using SMOTE	64
6.7	Performance Comparison of Hyperparameter-tuned Base Models for dataset class-balanced using SMOTE	65
6.8	Average Performance Metrics of Best Ensemble Model for dataset class-balanced using SMOTE	65
6.9	Performance Comparison of Ensemble Models of Different Feature Groups	68

6.10	Top 20 Features selected using RFE with RF and LR over the 5 SMOTE Training Folds	69
6.11	Average Performance of SMOTE Ensemble Model with Individual Features . .	71
6.12	Optimal hyperparameters of Base Models used in SMOTE Ensemble Model . .	73
A.1	Summarised ADNI criteria for CN, SCD, MCI and AD (Petersen and al. 2005; Petersen and al. 2015).	88
F.1	All combinations of stacking base models with final estimators	100
F.2	All combinations of voting base models	115
F.3	All base models in bagging configurations	123
G.1	Optimal hyperparameters of Individual Models Models within Ensemble Model selected for dataset imputed using MICE	125
G.2	Optimal hyperparameters of Individual Models Models within Ensemble Model selected for dataset imputed using Mean+Mode Method	125
G.3	Optimal hyperparameters of Individual Models Models within Ensemble Model selected for dataset imputed using Median+Mode Method	127
G.4	Optimal hyperparameters of Individual Models Models within Ensemble Model selected for dataset class-balanced using ROS Method	128
G.5	Optimal hyperparameters of Individual Models Models within Ensemble Model selected for dataset class-balanced using ADASYN Method	128
G.6	Optimal hyperparameters of Individual Models Models used within Ensemble Model selected for dataset class-balanced using Borderline SMOTE Method . .	129
G.7	Optimal hyperparameters of Individual Models Models within Ensemble Model selected for dataset class-balanced using RUS Method	130
G.8	Optimal hyperparameters of Individual Models Models within Ensemble Model selected for dataset class-balanced using CCUS Method	131
G.9	Optimal hyperparameters of Individual Models Models used in Ensemble Model of Base Features	132
G.10	Optimal hyperparameters of Individual Models Models used in Ensemble Model of Base + CSF Features	133
G.11	Optimal hyperparameters of Individual Models Models used in Ensemble Model of Base + PET Features	135
G.12	Optimal hyperparameters of Individual Models Models used in Ensemble Model of Base + MRI Features	136
G.13	Optimal hyperparameters of Individual Models Models used in Ensemble Model of Base + Plasma Features	136
G.14	Optimal hyperparameters of Individual Models Models used in Ensemble Model of Base + APOE4 Features	137
G.15	Optimal hyperparameters of Individual Models Models used in Ensemble Model of All Features except MRI and PET Data	139
G.16	Optimal hyperparameters of Individual Models Models used in Ensemble Model of Non-collinear Features	140

Abbreviations and Acronyms

ABETA Cerebrospinal Fluid Amyloid-Beta 42. 32

AD Alzheimer’s Disease. viii, x, 1–3, 5–9, 14, 15, 25, 28, 30, 32, 33, 42, 45, 46, 53, 58, 59, 61, 63, 65, 67–72, 74–78, 88

ADASYN Adaptive Synthetic Sampling. viii, 21, 77

ADNI Alzheimer’s Disease Neuroimaging Initiative. 1, 3, 9, 29, 31, 42, 46

AI Artificial Intelligence. 9

APOE4 ϵ 4 Allele count of the Apolipoprotein E gene. 33

AV45 Flortetapir. 32

BL-SMOTE Borderline Synthetic Minority Oversampling Technique. 22

CCUS Clustered Centroids Undersampling. viii, 23

CDRSB Clinical Dementia Rating Sum of Boxes. 32

CN Cognitively Normal. 42

CSF Cerebrospinal Fluid. 30

CV Cross-Validation. ix, 24, 26, 28, 44, 53, 58, 59, 62–64, 66, 67

DIGITSCOR Digit Symbol Substitution Score. 32

DT Decision Tree. x, 15, 18, 51, 100–123

EcogPtDivatt Self-reported Divided Attention score. 32

EcogPtLang Self-reported Language score. 32

EcogPtMem Self-reported Memory score. 32

EcogPtOrgan Self-reported Organisation score. 32

EcogPtPlan Self-reported Planning score. 32

EcogPtTotal Self-reported Total score. 32

EcogPtVispat Self-reported Visuospatial score. 32

EcogSPDivatt Study-partner reported Divided Attention score. 32

EcogSPLang Study-partner reported Language score. 32

EcogSPMem Study-partner reported Memory score. 32

EcogSPOrgan Study-partner reported Organisation score. 32

EcogSPPlan Study-partner reported Planning score. 32

EcogSPTotal Study-partner reported Total score. 32

EcogSPVisspat Study-partner reported Visuospatial score. 32

FDG Fluorodeoxyglucose. 32

FN False Negative. 24

FP False Positive. 24

FPR False Positive Rate. 24

Gaussian NB Gaussian Naive Bayes. x, 15, 16, 51, 52, 100–123

GTB Gradient Tree Boosting of Decision Trees. 26, 28

IDE Integrated Development Environment. 29

kNN k-Nearest Neighbours. viii–x, 13, 22, 26, 28, 50, 51, 60, 100–123

LDELTOTAL Logical Memory Delayed Recall score. 32

LIME Local Interpretable Model-agnostic Explanations. 78

LOOCV Leave-one-out Cross-Validation. 26, 28

LR Logistic Regression. ix, x, 11, 26–28, 50, 60, 63, 64, 73, 100–123

MCI Mild Cognitive Impairment. 1–3, 6, 7, 9, 14, 15, 25, 28, 32, 33, 42, 45, 46, 53, 58, 59, 61, 63, 65, 67–72, 74–78, 88

MICE Multiple Imputation by Chained Equations. 77

ML Machine Learning. viii, 9–11, 24, 26, 29

MLP Multi-Layer Perceptron. ix, x, 14, 26, 28, 51, 60, 63, 64, 73, 100–123

MMSE Mini-Mental State Examination. 32

MoCA Montreal Cognitive Assessment. 32

mPACCdigit ADNI modified PACC with Digit Symbol Substitution. 33

mPACCtrailsB ADNI modified PACC with Trails B. 33

MRI Magnetic Resonance Imaging. 25, 30

OPLS Orthogonal Partial Least Squares. 26, 28

PET Positron Emission Tomography. 30

PLASMA_ABETA42/40 Plasma Amyloid-Beta 42/40 Ratio. 32

PLASMA_NFL Plasma Neurofilament Light Chain. 32

PLASMA_PTAU Plasma Phosphorylated Tau. 32

PTAU CSF Phosphorylated Tau. 32

PTEDUCAT Years of Education. 32

PTGENDER Gender. 33

PTMARRY Marital Status. 33

RAVLT_forgetting RAVLT Forgetting Score. 32

RAVLT_immediate RAVLT Immediate Score. 32

RAVLT_learning RAVLT Learning Score. 32

RAVLT_perc_forgetting RAVLT Percentage Forgetting Score. 32

RF Random Forest. viii–x, 18, 26–28, 52, 59, 62–64, 67, 72, 73, 77, 100–123

RFE Recursive Feature Elimination. viii, 27, 48, 68

ROC Receiver Operating Characteristic curve. ix, 24, 62, 67

ROC-AUC Area Under the Receiver Operating Characteristic curve. 2, 24, 26, 59, 64

ROS Random Oversampling. 21

SCD Subjective Cognitive Decline. 1–3, 6, 8, 9, 14, 15, 25, 28, 32, 33, 42, 45, 46, 58, 59, 61, 63, 65, 67–72, 74–78, 88

SHAP SHapley Additive exPlanations. 78

SMOTE Synthetic Minority Oversampling Technique. viii–xi, 22, 26, 63–69, 71–73, 77

SVC Support Vector Classifier. viii, 12, 26, 28, 50, 100–123

SVM Support Vector Machine. 12, 26, 28

TAU CSF Total Tau. 32

TN True Negative. 24

TNR True Negative Rate. 24

TP True Positive. 24

TPR True Positive Rate. 24

TRABSCOR Trail Making Test B Score. 32

XGBoost Extreme Gradient Boosting. x, 19, 26, 28, 52, 100–123

Chapter 1

Introduction

1.1 Motivation

Alzheimer’s Disease (AD) is a progressive neurodegenerative disorder and the most common cause of dementia worldwide (Dolcet-Negre et al. 2023). The neurodegenerative process underlying AD is believed to begin at least 15–20 years before the onset of noticeable cognitive symptoms (Silverberg et al. 2018). Detecting individuals at the Mild Cognitive Impairment (MCI) stage may be too late for effective intervention, as substantial neuronal loss and irreversible cognitive decline may have already occurred (Yue et al. 2020).

Therefore, predicting conversion to AD (Dementia) at an even earlier stage is critical for improving patient outcomes and supporting families. Subjective Cognitive Decline (SCD), which occurs during the preclinical stage of the AD continuum, is increasingly recognised as the earliest clinical indicator of future AD, often preceding MCI by several years (Jessen, Amariglio, et al. 2014). Accurately predicting the conversion of SCD patients to MCI or AD using machine learning enables timely interventions, informs clinical decision-making, and supports selecting candidates for research into AD progression and treatment development.

1.2 Aims and Objectives

1.2.1 Aims

This project aims to develop an ensemble machine learning model to predict the conversion of SCD patients to MCI or AD within 10 years, using a set of features extracted from the Alzheimer’s Disease Neuroimaging Initiative (ADNI) database. The process involves applying preprocessing techniques to clean, filter, and prepare the input data, resulting in a complete and suitable dataset for machine learning. Additionally, a comprehensive suite of models is developed and systematically evaluated to identify the most effective predictive model. Additionally, an experimental extension exploring prediction within a 7-year window is planned, if time permits.

1.2.2 Objectives

The objectives are closely aligned with the pipeline of model development and the primary challenges encountered in this classification task.

(a) Handling Missing Data

- **Description:** Explore and apply various imputation techniques to handle missing data, generating multiple imputed datasets. The imputation method with the highest predictive performance should be selected to create the final model training and evaluation dataset.
- **Success Criteria:** The final dataset should be fully imputed, containing no missing values. The selected imputation technique should consistently demonstrate superior predictive performance based on the average of accuracy and Area Under the Receiver Operating Characteristic curve (ROC-AUC) across evaluation experiments.

(b) Handling Class Imbalance

- **Description:** Investigate and apply various resampling techniques to address class imbalance, generating multiple resampled datasets. The technique that yields the highest predictive performance should be selected to construct the final model training dataset.
- **Success Criteria:** The final training dataset should be class-balanced, containing an equal number of samples from both classes. The chosen resampling method should consistently demonstrate superior predictive performance based on the average of accuracy and ROC-AUC across evaluation experiments.

(c) Developing Ensemble Model

- **Description:** A diverse set of individual models should be developed and hyperparameter-tuned. Subsequently, various ensemble techniques should be explored to combine these models. The ensemble configuration that achieves the best predictive performance in predicting conversion from SCD to MCI/AD within 10 years, based on the average of accuracy and ROC-AUC across evaluation experiments, should be selected as the final model.
- **Success Criteria:** The final model should accurately predict the conversion of SCD patients to MCI/AD within 10 years and demonstrate improved performance compared to the models presented in Lerch et al. 2024, Dolcet-Negre et al. 2023, and Yue et al. 2020.

(d) Feature Selection

- **Description:** Both manual and automated feature selection methods should be applied to assess the importance of individual features and feature groups in predicting the conversion of SCD patients to MCI/AD. Special attention should be given to excluding costly features, such as MRI and PET data, to evaluate the trade-off between improvements in model performance and the costs associated with data collection.
- **Success Criteria:** All features are thoroughly evaluated for relevance, with the final feature set including only those that significantly impact predictive performance.

Feature Selection focused on minimising data collection costs while maintaining or improving the model’s average accuracy and ROC-AUC across evaluation experiments.

Extensions

(e) Extend to Conversion Within 7 Years

- **Description:** Apply the complete preprocessing and model development pipeline to predict the conversion of SCD patients to MCI/AD within a 7-year time-frame, replicating the steps used for the 10-year prediction.
- **Success Criteria:** The final model should outperform the predictive performance of the model presented in Yue et al. 2020, which also focuses on 7-year conversion prediction.

1.3 Evaluation Strategy

Due to the limited size of the dataset available from the ADNI database, 5-fold cross-validation was employed for all models developed during model construction (Chapter 5) using the datasets generated through preprocessing (Chapter 4). The evaluation metrics described in Section 2.2.8 were then calculated. The results of this evaluation are presented in Chapter 6.

1.4 Report Structure

The report has been structured to minimise repetition and introduce any concept before being referenced in the work.

1.4.1 Chapter 2 - Background

Background chapter provides a basic overview of concepts related to AD and Machine Learning (ML). The AD overview covers the stages of the AD progression and the diagnostic process for each stage. The ML overview introduces key concepts such as preprocessing techniques, model types, hyperparameters, ensembling, and evaluation methods.

1.4.2 Chapter 3 - Literature Review

Literature Review chapter provides a specific literature review of previous work focused on predicting the conversion from one stage of AD to a later stage, including the transition from MCI to AD and from SCD to MCI/AD. The review also includes an overview of the preprocessing techniques and model construction pipelines employed in these studies.

1.4.3 Chapter 4 - Preprocessing

Preprocessing chapter outlines the detailed preprocessing approach employed in this project, including data filtering from the ADNI dataset, formatting the data for use with machine learning models, applying imputation techniques to address missing data, and implementing resampling methods for class balancing. Additionally, this section discusses the creation of multiple datasets using the different preprocessing techniques and feature selection methods.

1.4.4 Chapter 5 - Model Construction

Model Construction chapter outlines the process of constructing and hyperparameter-tuning individual models, developing various ensemble models based on these individual models, and selecting the best-performing ensemble model for each dataset produced in Chapter 4.

1.4.5 Chapter 6 - Results

Results chapter presents the evaluation results from 5-fold cross-validation of the best-performing models developed for each dataset described in Chapter 4. It includes comparing the top model performances across various imputation techniques, resampling strategies, and feature groups. Additionally, results from automatic feature selection using Recursive Feature Elimination (RFE) are reported.

1.4.6 Chapter 7 - Discussion

Discussion chapter provides an extended discussion of the results presented in Chapter 6, including an evaluation of the methodologies employed, an assessment of how well the project objectives were achieved, and a personal reflection.

1.4.7 Chapter 8 - Conclusion and Future Work

The conclusion section summarises the key findings of the project, while the future work section outlines potential directions for extending and enhancing the experiments conducted in this project.

Chapter 2

Background

2.1 Alzheimer's Disease (AD)

AD is a progressive and irreversible neurodegenerative disorder and the most common cause of dementia, accounting for 60–70% of the 57 million global dementia cases (WHO 2025). AD is characterised by the accumulation of amyloid-beta ($A\beta$) plaques and neurofibrillary tangles composed of tau protein, particularly in the medial temporal lobe and neocortical regions of the brain (Figure 2.1 ; Breijyeh and Karaman 2020). These changes lead to synaptic loss and, consequently, cognitive and functional decline.

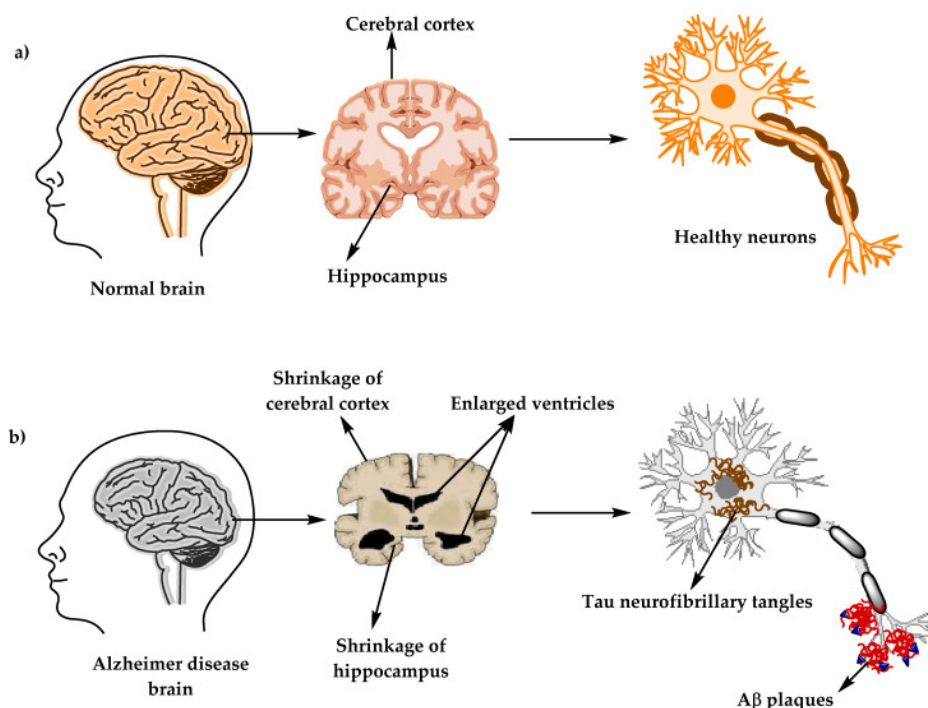


Figure 2.1: The physiological structure of the brain and neurons in (a) healthy brain and (b) AD brain (Breijyeh and Karaman 2020).

Increasing evidence suggests that these pathological changes begin years, or even decades, before the appearance of clinical symptoms (Silverberg et al. 2018; Jansen et al. 2015; Busche

and Konnerth 2016). This extended preclinical phase presents a critical window for early detection and intervention, which could be pivotal in slowing disease progression (Canevelli et al. 2013; Sperling et al. 2011).

2.1.1 AD Continuum

AD is recognised as a continuum of progressive stages marked by a gradual decline in cognitive function (Figure 2.2), as defined by the National Institute on Aging and the Alzheimer's Association (NIA-AA) Jack Jr. et al. 2018. This continuum is typically divided into three primary phases: preclinical phase, MCI, and AD (Dementia) (Tahami Monfared et al. 2022).

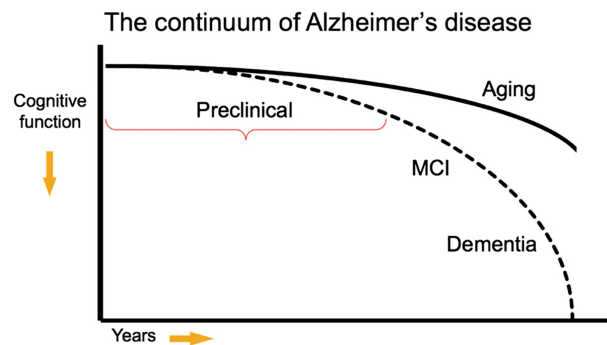


Figure 2.2: The continuum of AD (Sperling et al. 2011).

Preclinical

The preclinical stage is asymptomatic but biologically active, marked by pathophysiological changes such as the accumulation of A β plaques and phosphorylated tau (p-tau) protein (Tahami Monfared et al. 2022). It is subdivided into three stages (Figure 2.3), with Stage 3 associated with the highest risk of progression to MCI or AD (Parnetti et al. 2019). SCD, defined as self-perceived cognitive decline despite normal neuropsychological performance, is considered the earliest clinical manifestation of this phase (Jessen, Wolfsgrubner, et al. 2023).

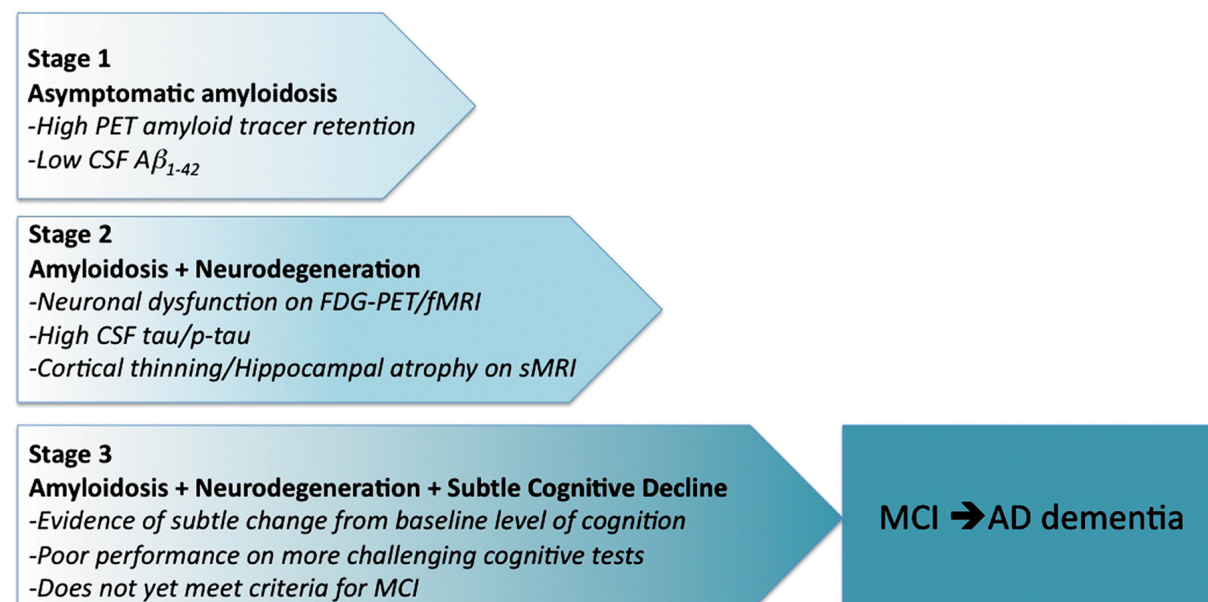


Figure 2.3: Graphic representation of the proposed staging framework for preclinical AD (Sperling et al. 2011).

Mild Cognitive Impairment (MCI)

MCI, also referred to as prodromal AD, is characterised by mild but measurable declines in cognitive function and neuropsychological test performance. According to the Petersen criteria, individuals with MCI typically perform at least 1.5 standard deviations below normal on memory assessments while maintaining independence in daily functional activities (Tahami Monfared et al. 2022). Individuals in this phase are at an increased risk of progressing to AD (Albert et al. 2011).

Dementia

AD Dementia (commonly referred to simply as AD) represents the final stage of AD progression, characterised by substantial and progressive cognitive decline across multiple domains, often accompanied by neurobehavioral symptoms and resulting in significant functional impairment in daily living activities (Jack Jr. et al. 2018). At this stage, individuals are no longer fully independent and require assistance with everyday tasks. This loss of functional autonomy is the primary distinction between AD (Dementia) and MCI (Jack Jr. et al. 2018). AD Dementia is further classified by severity into mild, moderate, and severe stages.

Note: Throughout this report, AD is used to refer both to Alzheimer’s Disease as a pathological process and to the dementia stage of the disease, depending on the context.

2.1.2 Diagnosis

Diagnosing stages in the AD continuum involves clinical evaluation, cognitive testing, and biomarker analysis. Figure 2.4 illustrates the hypothetical progression of functional and biomarker changes across the AD continuum.

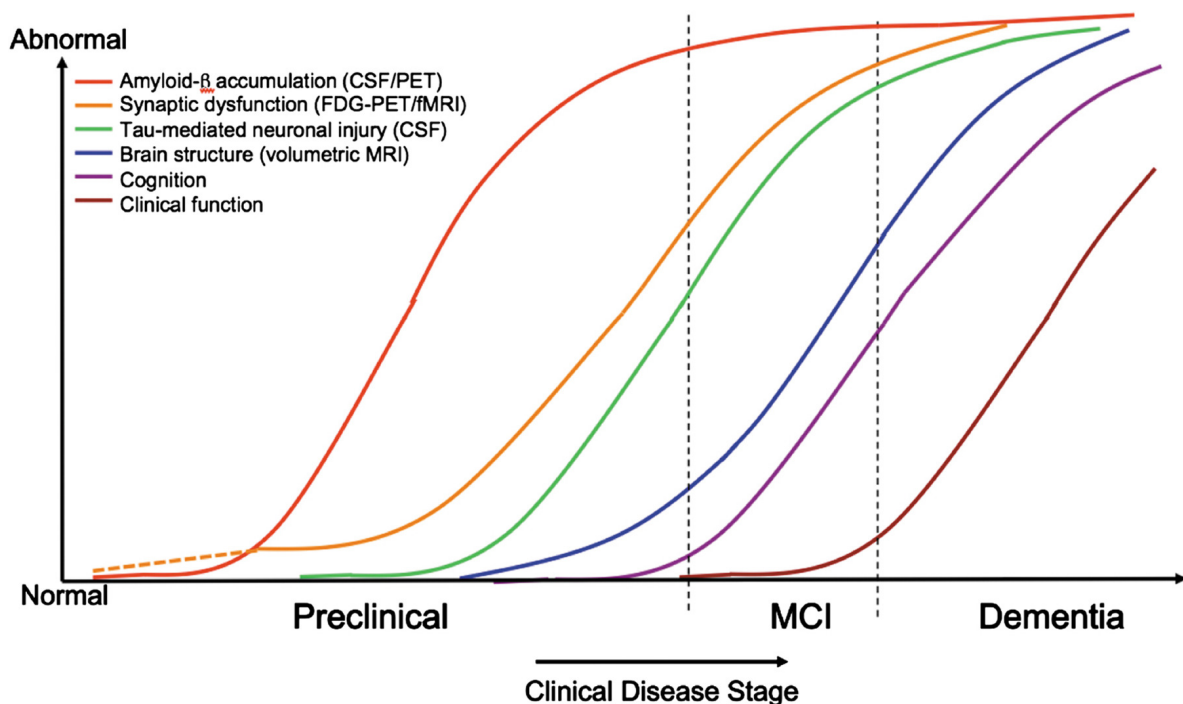


Figure 2.4: Hypothetical timeline of functional and biomarker changes across the AD continuum (Sperling et al. 2011).

Clinical and Cognitive Assessment

Clinical diagnosis begins with subjective symptom reporting, informant input, and structured clinical assessments. During diagnosis, particularly in individuals with SCD, self-reported cognitive concerns are evaluated using standardised tools such as the Everyday Cognition (Ecog) questionnaire and Cognitive Change Index score (Song et al. 2021; Rattanabannakit et al. 2016). These assessments are typically supplemented with a detailed medical history, physical and neurological examinations, and standardised cognitive screening instruments such as the Mini-Mental State Examination (MMSE) and Clinical Dementia Rating (CDR) scale (Bomasang-Layno and Bronsther 2021; Tombaugh and McIntyre 1992; Morris 1997).

Biomarker Analysis

Biomarkers provide objective measures of AD pathology, particularly in early stages where symptoms may not be apparent. The A/T/N classification system categorises biomarkers as follows (C. R. Jr. Jack et al. 2016):

- **A:** Amyloid-Beta ($A\beta$) biomarker (amyloid PET or CSF $A\beta_{42}$).
- **T:** Tau pathology biomarker (CSF p-tau or tau PET).
- **N:** Quantitative or topographic biomarker of neurodegeneration or neuronal injury (CSF t-tau, FDG-PET, or structural MRI).

Magnetic Resonance Imaging (MRI)-based volumetric measurements of brain regions such as the hippocampus, amygdala, and parahippocampal gyrus are widely used as structural biomarkers of AD-related neurodegeneration (Bottino et al. 2002). Functional imaging techniques further support AD diagnosis, including fluorodeoxyglucose positron emission tomography (FDG-PET) and $A\beta$ PET. Cerebrospinal fluid (CSF) biomarkers, particularly the $A\beta_{42}/40$ ratio and phosphorylated tau (p-tau), provide biochemical evidence of amyloid and tau pathology (Blennow et al. 2015; C. R. Jr. Jack et al. 2016).

More recently, blood-based biomarkers have gained attention as a non-invasive alternative for early detection. Plasma markers such as p-tau, neurofilament light chain (NFL), and the $A\beta_{42}/40$ ratio demonstrate strong concordance with CSF and PET findings, offering MORE scalable screening (Palmqvist et al. 2019; Nakamura et al. 2018; Mattsson et al. 2019).

2.2 Machine Learning (ML)

2.2.1 Introduction to ML

Machine Learning (ML) is a subfield of Artificial Intelligence (AI) that enables systems (models) to learn and improve automatically from experience (data) without explicit programming (Sarker 2021). ML models are typically categorised into four types: supervised learning, unsupervised learning, semi-supervised learning, and reinforcement learning.

Supervised Learning

Supervised learning involves training a model on a labelled dataset, where the input data is paired with known output values (Figure 2.5). In this project, data from the ADNI cohort is used alongside conversion labels that indicate whether an individual progresses from SCD to MCI/AD. The objective is to learn patterns that differentiate converters from non-converters. This task is framed as a supervised binary classification problem, where the model predicts one of two categories: "Stay in SCD" (non-conversion) or "Convert to MCI/AD" (conversion). The prediction is based on a predefined conversion window, with the main task focusing on a 10-year period and an additional experiment exploring a 7-year period.

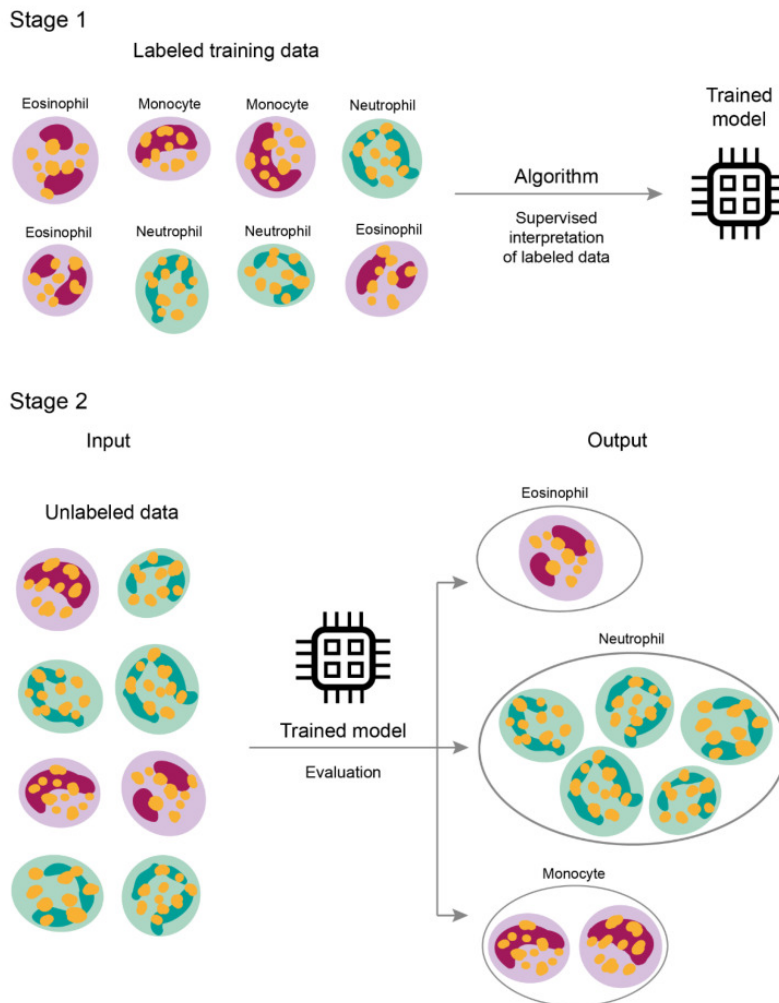


Figure 2.5: Supervised Learning (Koteluk et al. 2021).

2.2.2 ML Process

The ML process mainly consists of data preprocessing, model construction, model optimisation, and model evaluation (Figure 2.6; Koteluk et al. 2021).

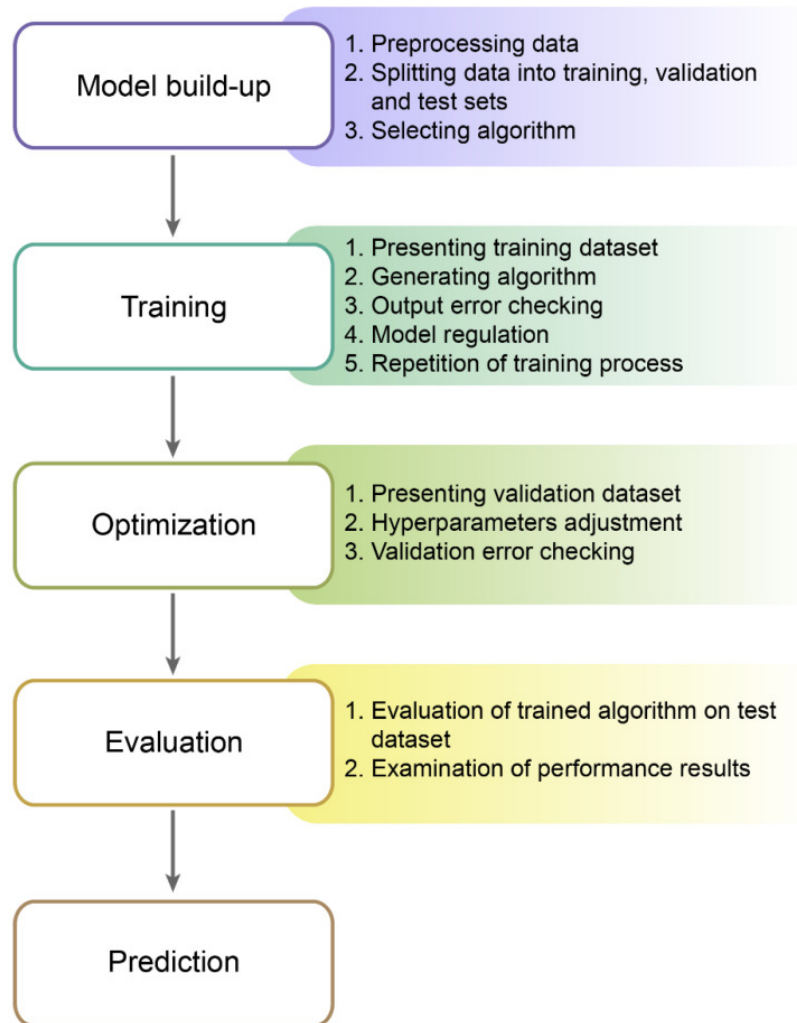


Figure 2.6: ML Process (Koteluk et al. 2021).

Data preprocessing

When working with multimodal datasets, the data is often incomplete (missing values), inconsistent, or corrupted. As a result, preprocessing techniques—such as imputation and standardisation—are applied to prepare the data for use in machine learning models. Preprocessing also involves splitting the dataset into training, validation, and test sets to support effective model construction, optimisation, and evaluation. Additionally, it includes feature selection to identify the most relevant variables for the predictive task.

Model Construction and Optimisation

Model construction involves selecting an appropriate ML model for the task and training it using the training data prepared during preprocessing. In this project, it specifically refers to selecting models suitable for binary classification. Model optimisation involves tuning the

model's hyperparameters and identifying the optimal configuration to maximise performance. The validation set, prepared during preprocessing, is used to evaluate different hyperparameter configurations and select the best one. This process may also include comparing multiple models and selecting the one that performs best for the given task.

Model Evaluation

Once the model is trained and optimised, it must be evaluated to assess its performance and ensure it meets the desired criteria for practical use. Evaluation also tests the model's generalisability to unseen data, helping identify overfitting, where the model performs well on training data but generalises poorly on unseen data (Ying 2019). This involves computing relevant performance metrics and applying suitable evaluation methods for the task.

2.2.3 Base Models

In this project, various ML models suitable for binary classification were developed and hyperparameter-tuned for integration into ensemble models, as detailed in Chapter 5. Section 2.2.3 presents an overview of the base models implemented using the **scikit-learn** library (Scikit-Learn Developers 2025), along with their corresponding hyperparameters.

Logistic Regression (LR)

LR estimates the relationship between a binary dependent variable (output label) and one or more independent variables (input features) (Zabor et al. 2022; Ahmed Arafa et al. 2022). It uses the odds ratio to model the relationship between the input data and the binary output. The hypothesis for LR is expressed as (Ahmed Arafa et al. 2022):

$$h(\theta^T X) = P(Y = 1 | X_1, X_2, \dots, X_n) = \frac{1}{1 + e^{-\theta^T X}} \quad (2.1)$$

Where Y is the dependent variable, $X_{1..n}$ are the independent input variables, and θ is the weight vector for the given input variables. If the probability of Y being 1 exceeds a predefined threshold (commonly 0.5), the predicted output label is 1; otherwise, it is 0.

The loss function used during training is the cross-entropy function, which is formulated as (Ahmed Arafa et al. 2022):

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m \left[y^{(i)} \log(h_{\theta}(x^{(i)})) + (1 - y^{(i)}) \log(1 - h_{\theta}(x^{(i)})) \right] \quad (2.2)$$

Where m is the number of training examples, $y^{(i)}$ is the actual label, and $h_{\theta}(x^{(i)})$ is the predicted probability of label 1 for the i th training example.

During training, the loss calculated for each sample is used to update the weights, as shown by the gradient descent update rule (Ahmed Arafa et al. 2022), where α is the learning rate:

$$\theta_j = \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta) \quad (2.3)$$

Hyperparameters in `sklearn.linear_model.LogisticRegression`

1. **Penalty Type (`penalty`):** Adds regularisation to the weight update equation 2.3, with L1 (Equation 2.4) promoting sparsity, and L2 (Equation 2.5) penalising large coefficients, preventing overfitting (Ahmed Arafa et al. 2022).

$$\lambda \sum_{j=1}^n |\theta_j| \quad (2.4)$$

$$\lambda \sum_{j=1}^n (\theta_j)^2 \quad (2.5)$$

2. **Regularisation Strength Parameter (`C`):** C is the inverse of the regularisation strength, λ , shown in equations 2.4 and 2.5, which controls the strength of the regularisation term. Smaller C applies stronger regularisation, helping to reduce overfitting by shrinking model coefficients.
3. **Optimisation Algorithm (`solver`):** Specifies the numerical algorithm used to estimate model parameters. "`liblinear`" is suitable for small datasets, while "`saga`" is faster and better suited for large datasets.

Support Vector Classifier (SVC)

SVC is a classification model based on Support Vector Machine (SVM), which maps inputs to a high-dimensional space and separates classes using a hyperplane that maximises the margin between them, improving generalisation (Figure 2.7; Cervantes et al. 2020). The decision boundary is defined by $w^T x + b = 0$, with parallel hyperplanes $w^T x + b = \pm 1$. The margin is given by $\frac{2}{\|w\|}$, and the optimisation problem is formulated as:

$$\min_{w,b} \frac{1}{2} \|w\|^2 \quad \text{s.t.} \quad y^{(i)} (w^T x^{(i)} + b) \geq 1 \quad (2.6)$$

where w is the weight vector, b is the bias term, $x^{(i)}$ is the input vector of the i^{th} sample, and $y^{(i)}$ is its corresponding output label.

Hyperparameters in `sklearn.svm.SVC`

1. **kernel:** Specifies the kernel function used to transform input data into a higher-dimensional space, enabling the model to separate non-linearly separable data.
2. **gamma:** Kernel coefficient for "`rbf`", "`poly`", and "`sigmoid`" kernels, which defines the

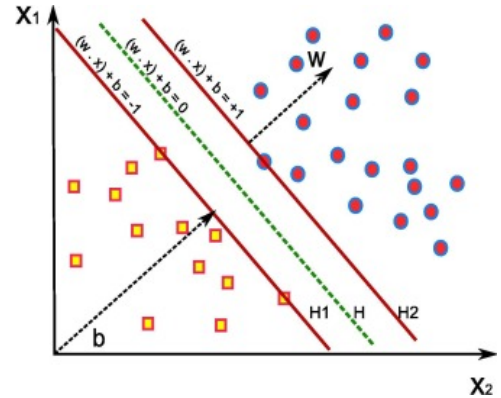


Figure 2.7: Support Vector Classifier (Cervantes et al. 2020).

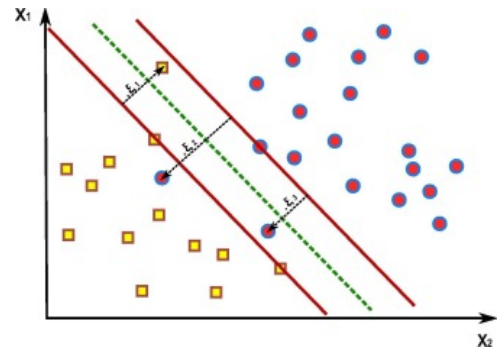


Figure 2.8: SVC with soft margins (Cervantes et al. 2020).

influence of a single training example, with low values implying far reach and high values implying close reach (GeeksforGeeks 2024).

3. **Regularisation Parameter (C):** For non-linearly separable data, soft margins are introduced using slack variables ξ_i to allow limited misclassifications (Figure 2.8). The optimisation objective becomes:

$$\min_{\mathbf{w}, b, \xi} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^m \xi_i \quad \text{s.t.} \quad y^{(i)}(\mathbf{w}^T \mathbf{x}^{(i)} + b) \geq 1 - \xi_i, \quad \xi_i \geq 0 \quad (2.7)$$

where C controls the trade-off between maximising the margin and allowing classification errors.

k-Nearest Neighbours (kNN) Classifier

kNN classifier identifies the k nearest samples to an input using a distance metric and assigns the majority label among them as the predicted output (Figure 2.9; Cunningham and Delany 2021).

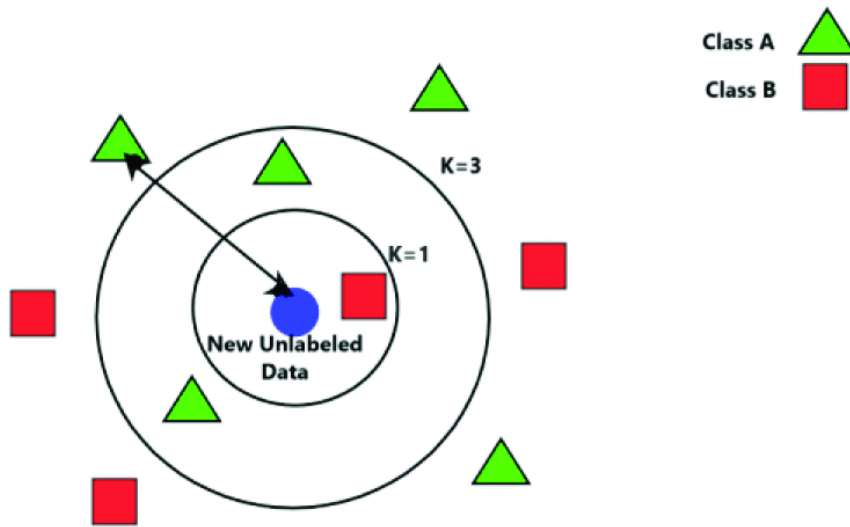


Figure 2.9: kNN with two classes (Taunk et al. 2019).

Hyperparameters in `sklearn.neighbors.KNeighborsClassifier`

1. **n_neighbors:** Specifies the number of nearest neighbors to consider for classification.
2. **weights:** Determines the influence of each neighbor. The "uniform" scheme treats all neighbors equally, while "distance" assigns higher weight to closer neighbors (inverse of distance).
3. **metric:** Defines the method used to compute distance between samples, such as "euclidean", "manhattan", or "minkowski".

Multi-Layer Perceptron (MLP)

MLP, as shown in Figure 2.10, is a fully connected feedforward neural network comprising an input layer, one or more hidden layers, and an output layer (Popescu et al. 2009; Bisong 2019). Each neuron in a layer is connected to every neuron in the subsequent layer. A non-linear activation function is applied at each layer to introduce non-linearity, which is essential for the model's expressive power. The network is trained using backpropagation, where weights are initialised and iteratively updated via gradient descent (Popescu et al. 2009). For this binary classification task of predicting conversion from SCD to MCI/AD, the output layer uses a softmax activation to produce a probability for the positive class. A threshold of 0.5 is typically applied to determine the predicted label.

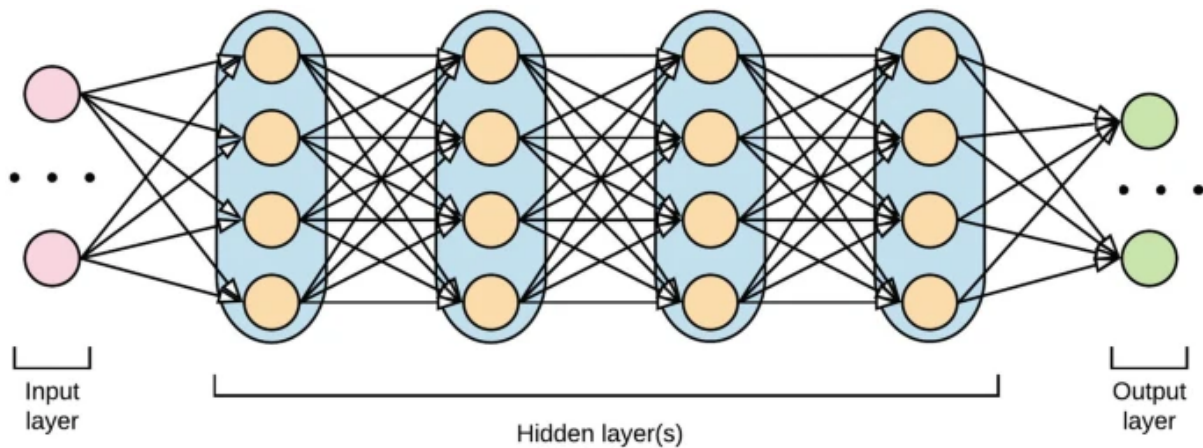


Figure 2.10: Multi-Layer Perceptron (Bisong 2019).

Hyperparameters in `sklearn.neural_network.MLPClassifier`

1. **Number and Size of Hidden Layers (`hidden_layer_sizes`):** Specifies the architecture of hidden layers. It is a tuple where each element represents the number of neurons in a corresponding hidden layer, e.g., `(100, 50)` defines two hidden layers with 100 and 50 neurons, respectively.
2. **`activation`:** Defines the non-linear function applied at each layer. Options include `"relu"`, `"logistic"`, and `"tanh"`.
3. **`solver`:** Determines the weight optimisation method. Common options are `"adam"` and `"sgd"`.
4. **`alpha`:** Controls the strength of L2 penalty applied during optimisation to prevent overfitting.
5. **`learning_rate`:** Specifies the schedule for weight updates: `"constant"`, `"adaptive"`, or `"invscaling"`. The initial rate is set by `learning_rate_init`, with a default of 0.001.
6. **`max_iter`:** Defines the maximum number of training iterations (epochs).

Decision Tree (DT)

DT is a non-parametric supervised learning algorithm structured as a hierarchical tree comprising a root node, internal nodes, branches, and leaf nodes (Figure 2.11; Mienye and Jere 2024). The tree begins with a root node, which represents the entire dataset. From this node, the algorithm iteratively splits the data at each internal node based on a specific criterion. This recursive process continues, with each subset of data being further split at child nodes until a stopping condition is met—typically when the nodes become pure (i.e., all data points in a node belong to the same class) or when a predefined maximum tree depth is reached (Mienye and Jere 2024). The terminal nodes, called leaf nodes, represent the final output class. In this binary classification task, each leaf node corresponds to one of the two possible outcomes:

"Stay in SCD" or "Convert to MCI/AD".

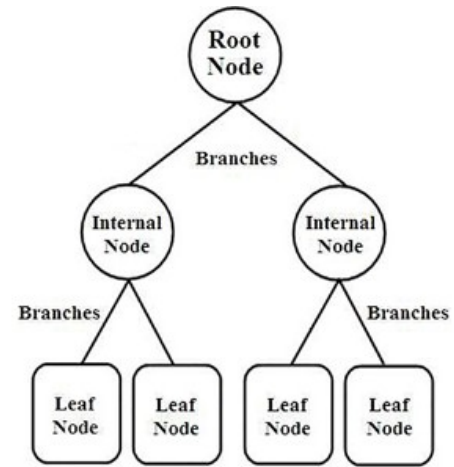


Figure 2.11: Decision Tree (Mienye and Jere 2024).

Hyperparameters in `sklearn.tree.DecisionTreeClassifier`

1. **criterion**: Defines the function used to measure the quality of a split at each node.
2. **splitter**: Specifies the strategy used to choose the split at each node. The options include choosing the "best" split based on the criterion or a "random" split among the features.
3. **max_depth**: Specifies the maximum depth of the tree. Limiting the depth prevents the tree from growing too large and helps to reduce overfitting.
4. **max_features**: Specifies the number of features to consider when looking for the best split. It can be set to a given number of features, square root of the total number of features ("sqrt"), the logarithm (base 2) of the total number of features ("log2"), or the total number of features ("None").

Gaussian Naive Bayes (Gaussian NB)

Gaussian NB is a variant of the Naive Bayes method that assumes the input data features follow a Gaussian (normal) distribution. The key assumption of the Naive Bayes method is that the features are conditionally independent given the class. Under this assumption, the probability of a particular class Y given the features $X_{1...n}$ can be computed as the product of the conditional probabilities of each feature X_i given the class Y . This is represented by the formula (Anand et al. 2022):

$$P(Y) = \prod_{i=1}^n P(X_i|Y) \quad (2.8)$$

Where $P(X_i|Y)$ represents the conditional probability of feature X_i given the class Y , and the product is taken over all features.

In Gaussian NB, the conditional probability $P(X_i|Y)$ is calculated using the Gaussian distribution formula (As'ad 2023):

$$P(X_i|C) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(X_i - \mu)^2}{2\sigma^2}\right)$$

Where μ and σ are the mean and standard deviation of the feature X_i for the class Y .

Hyperparameters in `sklearn.naive_bayes.GaussianNB`

1. **var_smoothing:** Adds a small value (a portion of the largest variance of all features) to each feature's variance to prevent division by zero and to smooth the Gaussian distribution, ensuring calculation stability (Jain 2021).

2.2.4 Ensembling Techniques

Stacking Ensembling Technique

Stacking, also known as Stacked Generalisation, is a machine learning ensembling technique that combines multiple individual (base) models to improve performance (Wolpert 1992). This method involves training a set of base models and using their predictions as the input to a final estimator, which learns to combine them optimally (Figure 2.12).

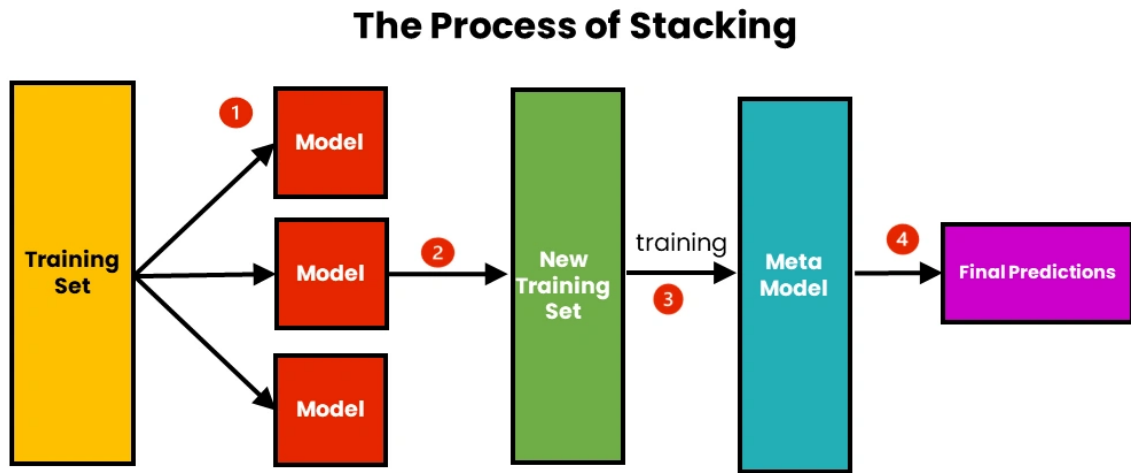


Figure 2.12: Stacking Ensembling (Kalirane 2025).

Voting Ensembling Technique

Similar to Stacking, Voting ensembling is an ensembling technique that combines multiple individual (base) models. It consists of two main approaches: hard voting and soft voting (Awan-Ur-Rahman 2023). In hard voting, each base model outputs a class label, and the final prediction is determined by majority vote—the class that receives the most votes is selected as the output. In contrast, soft voting uses the predicted probabilities from each base model. For each class, the probabilities are averaged across all models, and the class with the highest average probability is chosen as the final prediction. Figure 2.13 presents the visual representation of hard and soft voting ensembling technique.

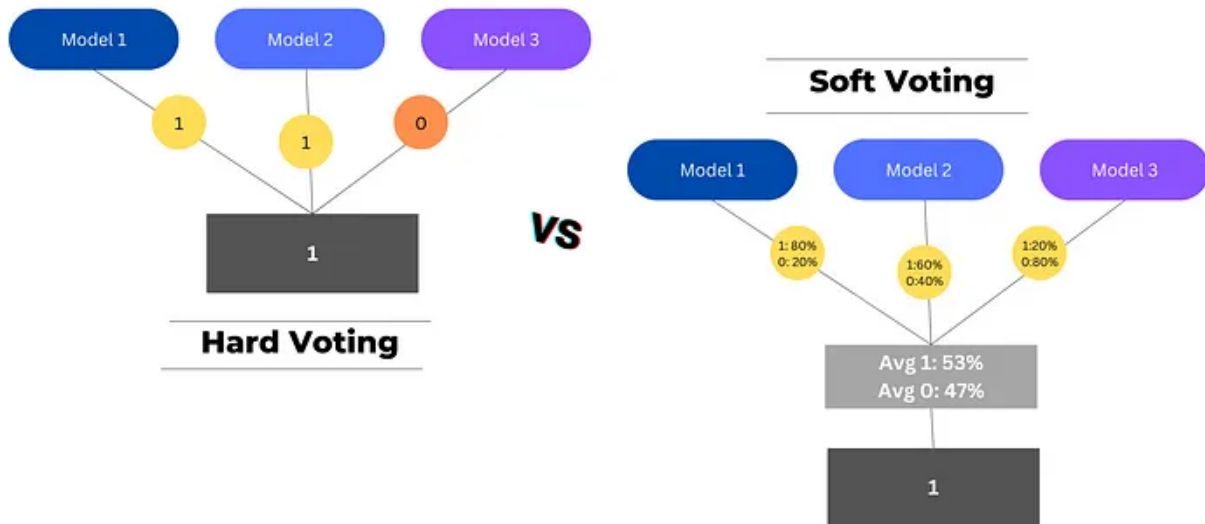


Figure 2.13: Hard vs Soft Voting Ensembling (Parashar 2022).

Bagging Ensembling Technique

Bagging, also known as bootstrap aggregating, is an ensembling technique that involves training multiple copies of a base estimator independently on different random subsets of the training set, sampled with replacement (Figure 2.14; Breiman 1996). The final prediction is obtained by aggregating the output of the multiple copies using hard voting or soft voting (Section 2.2.4). By training on diverse subsets of the data, bagging reduces the variance of the ensemble model when compared to the individual base estimator, which makes the model less prone to overfitting and more robust to noisy samples. This improved generalisation leads to better predictive performance.

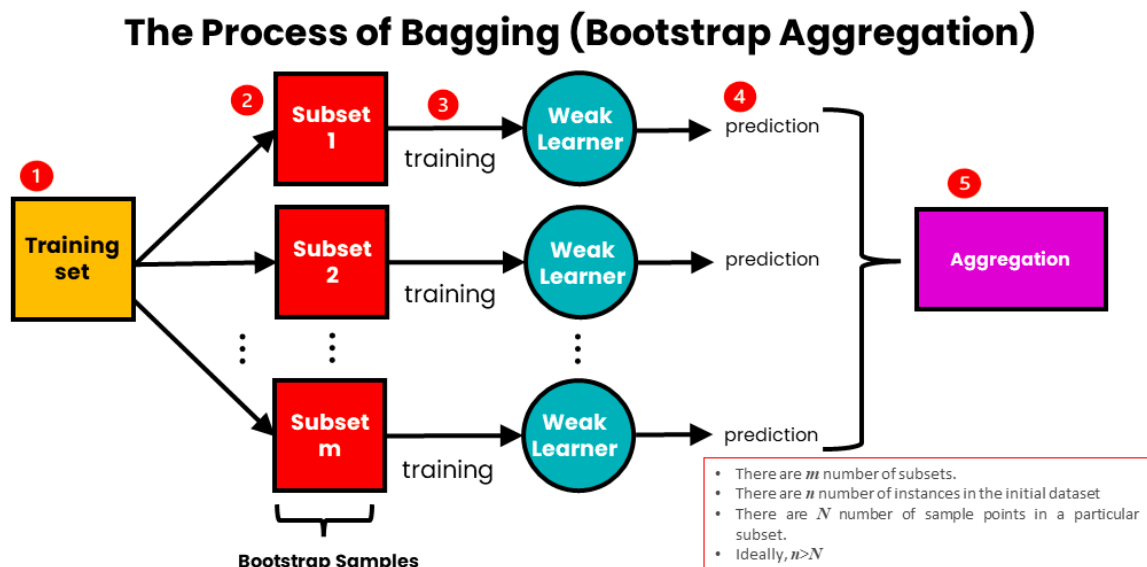


Figure 2.14: Bagging Ensembling (Kalirane 2025).

2.2.5 Base Ensemble Models

In Section 2.2.5, the ensemble models directly implemented using library interfaces are described, along with their corresponding hyperparameters used during instantiation. For the purposes of this project, these ensemble models are considered part of the base models discussed in Section 2.2.3 and are used as base estimators within the ensembling strategies detailed in Section 2.2.4.

Random Forest (RF)

RF is an ensemble model that employs the bagging technique in combination with feature randomness, which involves generating random subsets of features (Breiman 2001; Salman, Kalakech, and Steiti 2024). The RF model is composed of multiple DTs, each trained on a different subset of features and a different subset of training samples obtained through feature randomness and bootstrap sampling respectively. This approach enhances the decision-making process of individual Decision Tree (DTs) (described in Section 2.2.3) by ensuring that each tree only considers a limited subset of the available features when determining splits, thereby increasing model diversity and reducing overfitting. For classification tasks, the final prediction is made by majority voting (hard voting) among the individual trees, whereas in regression tasks, the final output is the average of the predictions made by all DTs (Figure 2.15).

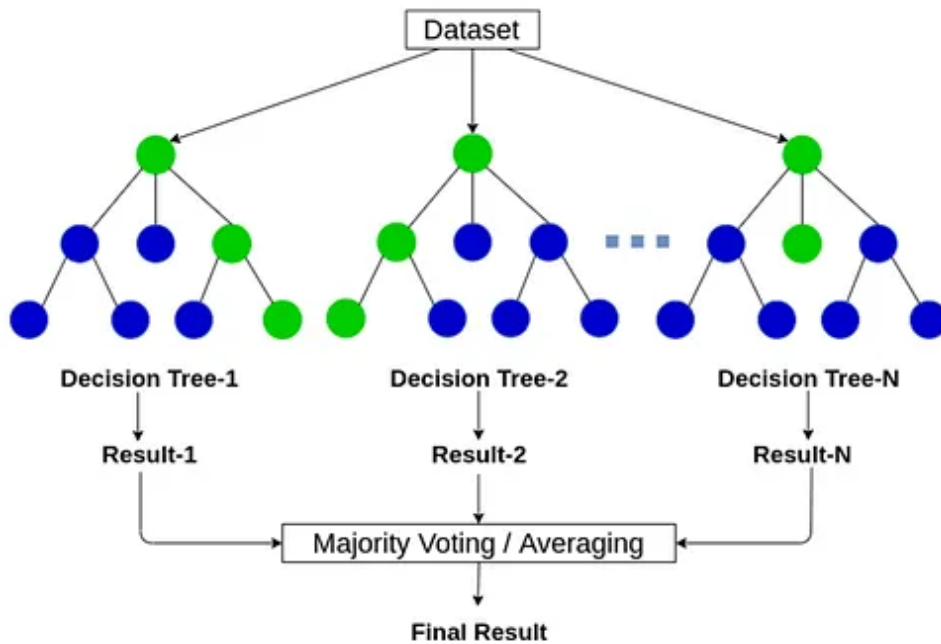


Figure 2.15: Random Forest Model (Omarzai 2024a).

Hyperparameters in `sklearn.ensemble.RandomForestClassifier`

1. **Number of Trees in the Forest (`n_estimators`):** Specifies the number of decision trees to be constructed in the random forest.
2. **`max_depth`:** Specifies the maximum depth allowed for each decision tree.
3. **`max_features`:** Specifies the number of features to consider when looking for the best split. It can be set to a given number of features, square root of the total number of

features ("sqrt"), the logarithm (base 2) of the total number of features ("log2"), or the total number of features ("None").

Extreme Gradient Boosting (XGBoost)

XGBoost is an ensemble learning algorithm that constructs a sequence of decision trees (DT), with each tree aiming to correct the errors made by the previous ones (Chen and Guestrin 2016). It utilises a gradient boosting framework, where each new tree is trained based on the residuals or gradients of the loss function of the previous model (Figure 2.16; Z. He et al. 2019). Specifically, after the first tree is trained on the data, the model computes the error (residual) between the predicted and actual values. This residual is then used to train the next tree, which focuses on correcting the errors from the previous iteration. The process is repeated iteratively for a predefined number of trees or until a stopping criterion is met. The final prediction is obtained by combining the outputs of all trees, with classification tasks typically using the soft voting method (Section 2.2.4).

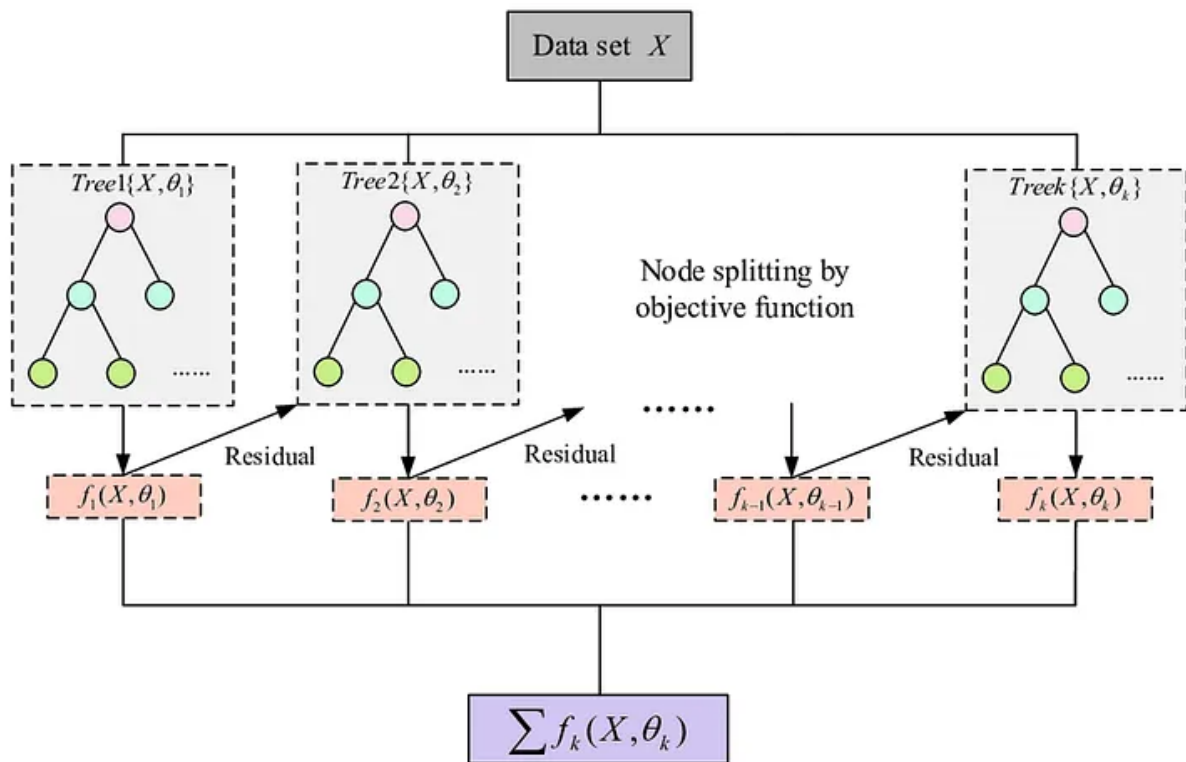


Figure 2.16: Random Forest Model (Omarzai 2024b).

Hyperparameters in `xgboost.XGBClassifier`

1. **Number of Trees (`n_estimators`):** Specifies the number of decision trees (boosting rounds) to be used as base learners.
2. **`max_depth`:** Specifies the maximum depth allowed for each decision tree.
3. **`max_features`:** Specifies the number of features to consider when looking for the best split. It can be set to a given number of features, square root of the total number of features ("sqrt"), the logarithm (base 2) of the total number of features ("log2"), or the total number of features ("None").

4. **learning_rate**: Controls the contribution of each tree to the final prediction. A lower learning rate makes the model more conservative, requiring more trees to achieve the same accuracy, while a higher rate leads to more aggressive updates but can increase overfitting.

2.2.6 Imputation Techniques

Single Imputation Techniques

Simple imputation methods assume data are missing at random. However, when data are missing not at random (MNAR), these methods can introduce bias, reduce variability, and impair model performance (Z. Zhang 2016).

Mean Imputation involves calculating the average of non-missing values for each feature and replacing missing data with this calculated average.

Median Imputation involves calculating the median of non-missing values for each feature and replacing missing data with this calculated median. This method is often preferred over mean imputation when the data contain outliers, as the median is less sensitive to extreme values (Vaj 2024).

Mode Imputation involves calculating the mode (most frequent value) of non-missing values for each feature and replacing missing data with this calculated mode. This technique is commonly used for categorical features, where the most frequent category is used to replace missing values (Vaj 2024).

Multiple Imputation by Chained Equations (MICE) Technique

Multiple Imputation by Chained Equations (MICE) follows an iterative process for imputing missing values. The steps are as follows (Azur et al. 2011):

1. All missing values are initially filled using simple imputation methods, typically the mean imputation.
2. For each feature with missing values:
 - 2.1. The feature is treated as the target variable.
 - 2.2. All other features (with current imputed values) are used as input predictors.
 - 2.3. A base regression model (such as Bayesian Ridge Regression) is trained on the subset of rows where the target feature is present.
 - 2.4. The trained model is then used to predict the missing values of the target feature.
3. Steps 2.1 to 2.4 are repeated for all features with missing values in sequence, forming one iteration or cycle of imputations.
4. Steps 2 to 3 are repeated for a fixed number of iterations, each time refining the imputed values using the results from the previous iteration.

2.2.7 Resampling Techniques

Oversampling

Oversampling is a data augmentation technique used to address class imbalance by increasing the number of samples in the minority class (Yang, Khorshidi, and Uwe 2024; Ashraf 2023). This helps create a more balanced distribution and allows the model to learn more effectively from both classes.

Types of Oversampling

1. **Random Oversampling (ROS)** increases the number of samples in the minority class by randomly duplicating existing samples until both classes are exactly balanced (Yang, Khorshidi, and Uwe 2024). This approach is simple and preserves the original feature distribution, ensuring that the model encounters both classes equally during training. However, since it reuses the same data samples, it can lead to overfitting, as the model may overly rely on repeated patterns from the duplicated samples (Ali et al. 2019).
2. **Adaptive Synthetic Sampling (ADASYN)** generates synthetic samples by focusing on minority class instances that are more challenging to classify, particularly those located in regions dominated by the majority class (H. He et al. 2008). It assigns higher weights to these instances, increasing the likelihood of their selection for generating synthetic data (Figure 2.17). This approach adapts the sampling distribution based on local data density, thereby improving the model's ability to learn complex decision boundaries and enhancing performance on difficult-to-classify instances (Ali et al. 2019).

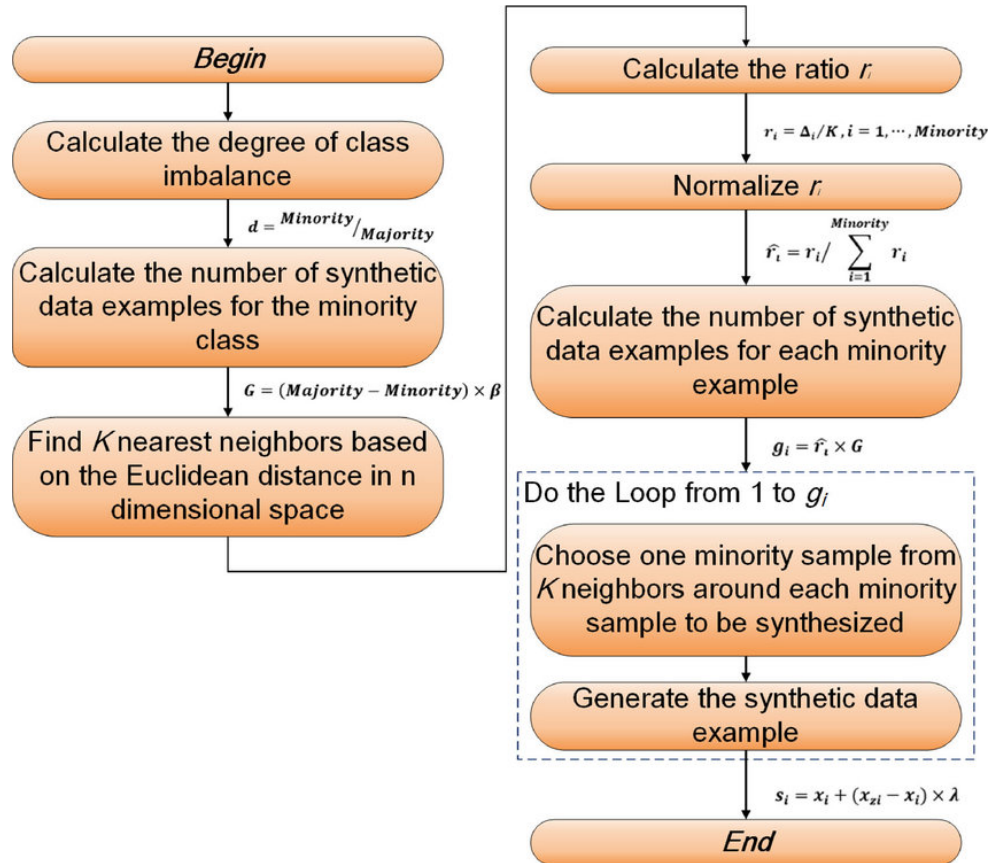


Figure 2.17: ADASYN Algorithm (Fan et al. 2020).

3. **Synthetic Minority Oversampling Technique (SMOTE)** generates new samples by linearly interpolating minority class samples with its randomly selected kNN samples, ensuring exact class balance (Figure 2.18; Chawla et al. 2002). However, as this method focuses only on the minority class, it may cause overlap between the two classes, potentially leading to over-generalisation (Ali et al. 2019).

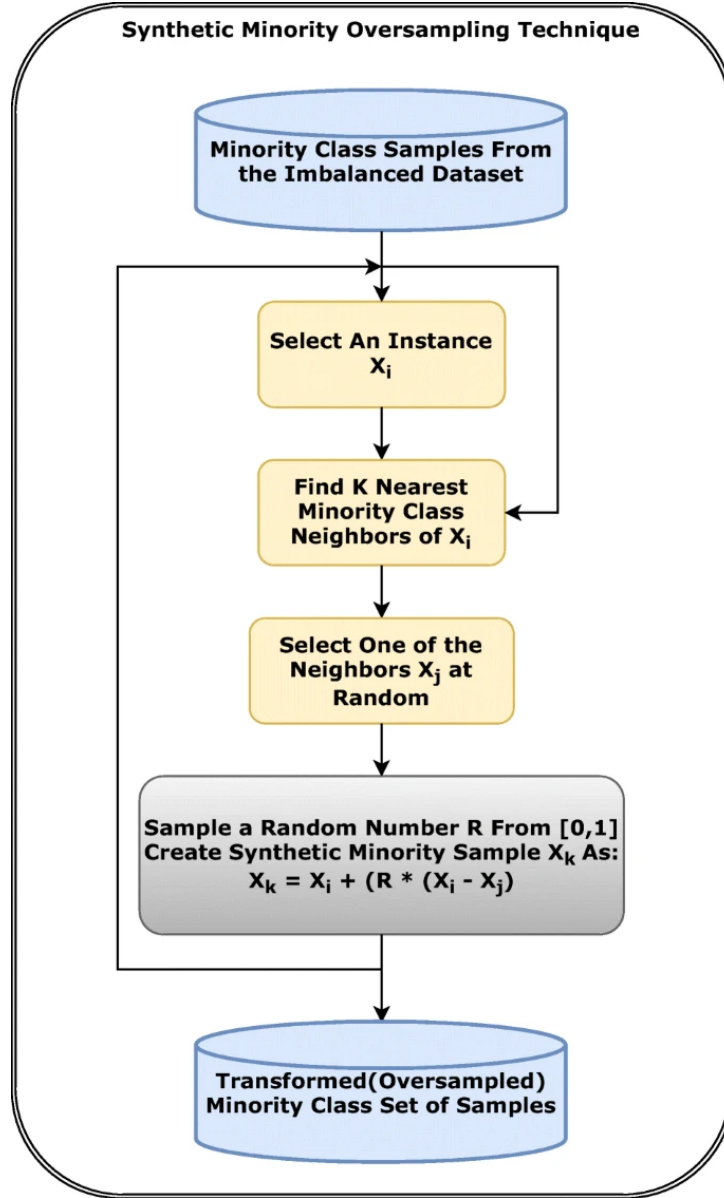


Figure 2.18: SMOTE Algorithm (Mishra et al. 2022).

4. **Borderline Synthetic Minority Oversampling Technique (BL-SMOTE)** extends the SMOTE algorithm by identifying the borderline samples of the minority class, which are those near the decision boundary between the two classes (Figure 2.19). It generates synthetic samples exclusively from these borderline samples, helping the model focus on the harder-to-classify instances (Han, W.-Y. Wang, and Mao 2005) (Ali et al. 2019).

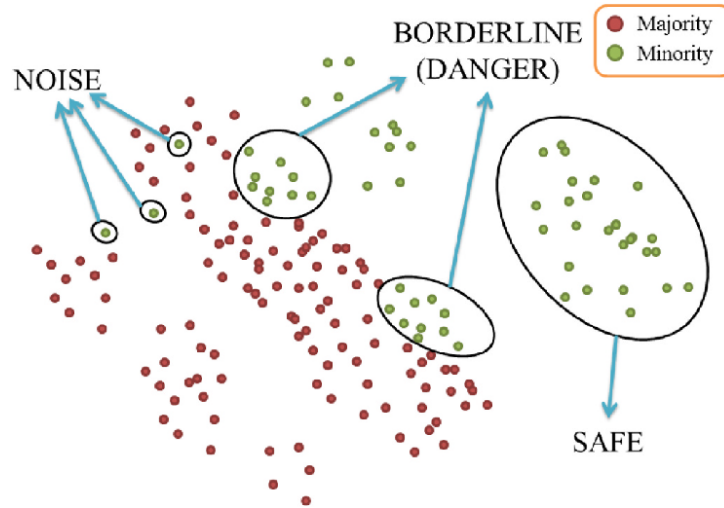


Figure 2.19: Borderline Samples (Kummer et al. 2022).

Undersampling

Undersampling involves reducing the number of samples in the majority class to achieve class balance. While this technique helps address class imbalance, it might lead to discarding valuable samples that could contribute to better learning of the decision boundary (Mohammed, Rawashdeh, and Abdullah 2020).

Types of Undersampling

1. **Random Undersampling (RUS)** removes samples from the majority class at random until the class distribution is balanced. While this technique is simple and computationally efficient, it may lead to the loss of informative samples, potentially reducing the ability to capture the underlying data distribution and generalise effectively (Ali et al. 2019).
2. **Clustered Centroids Undersampling (CCUS)** involves creating N clusters using the K-Means algorithm, where N is the number of minority class samples (Figure 2.20). All majority class samples are then replaced with these N clusters, resulting in a balanced dataset (Y.-P. Zhang, L.-N. Zhang, and Y.-C. Wang 2010). This technique aims to preserve the general distribution of the majority class while reducing its size.

Cluster Centroids

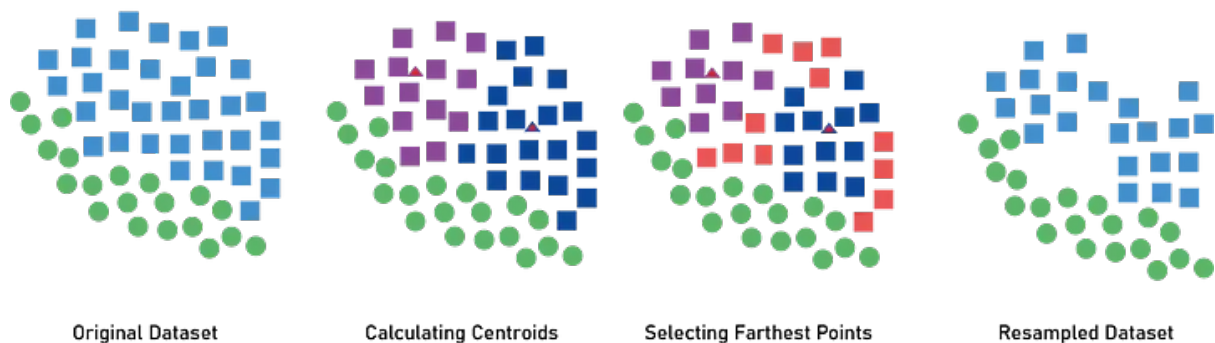


Figure 2.20: Clustered Centroids Undersampling (CCUS) Algorithm (Charfaoui 2023).

2.2.8 Evaluation

K-Fold Cross-Validation (CV)

K-Fold CV is a model evaluation technique in which the dataset is partitioned into K equally sized folds (Raschka 2020). The model is trained K times, each time using $K - 1$ folds for training and the remaining fold for validation. This process ensures that every sample is used for both training and validation exactly once. After all iterations, the evaluation metrics are averaged to provide an overall performance estimate.

Evaluation Metrics

The performance of the ML models was evaluated by comparing the predicted labels with the true labels. Table 2.1 summarises the evaluation metrics used in this study. Here, True Positive (TP) denotes the number of correctly predicted positive class samples, while False Positive (FP) indicates the number of negative samples incorrectly classified as positive. Similarly, True Negative (TN) and False Negative (FN) refer to the correctly and incorrectly predicted samples in the negative class, respectively.

Table 2.1: Evaluation metrics used for model assessment

Metric	Description
Accuracy	$\frac{TP+TN}{TP+TN+FP+FN}$
False Positive Rate (FPR)	$\frac{FP}{FP+TN}$
Specificity (True Negative Rate (TNR))	$\frac{TN}{TN+FP}$
Sensitivity (True Positive Rate (TPR)) / Recall	$\frac{TP}{TP+FN}$
Micro	Global recall over all instances
Macro	Unweighted average of class-wise recall
Weighted	Weighted average of class-wise recall
Precision	$\frac{TP}{TP+FP}$
Micro	Global precision over all instances
Macro	Unweighted average of class-wise precision
Weighted	Weighted average of class-wise precision
F1 Score	$\frac{2PR}{P+R}$
Micro	Global F1 score based on Micro Precision and Recall
Macro	Unweighted average of class-wise F1 scores
Weighted	Weighted average of class-wise F1 scores
ROC-AUC	Area under the Receiver Operating Characteristic curve (ROC) curve (TPR vs. FPR)

Chapter 3

Literature Review

Chapter 3 presents a focused yet comprehensive review of existing research on predicting conversion from SCD to MCI/AD. Owing to the limited number of studies directly addressing this progression, relevant methodologies from broader AD research have also been incorporated. Chapter 3 discusses approaches to data preprocessing, model development, and evaluation. Table 3.1 summarises the studies discussed.

3.1 Preprocessing

3.1.1 Model Input

In Lerch et al. 2024, input predictors were organised into three feature sets: (1a) socio-demographic and clinical information including MMSE and GDS scores, (1b) socio-demographic data combined with a battery of neuropsychological test scores, and (2) regional grey matter volumes derived from Magnetic Resonance Imaging (MRI) scans. Similarly, Yue et al. 2020 utilised 198 baseline neuroimaging features along with 25 clinical measurements to predict conversion from SCD to MCI. In Dolcet-Negre et al. 2023, structural MRI was employed exclusively, making use of its non-invasive and widely accessible nature. Overall, demographic factors, clinical and neuropsychological assessments, and neuroimaging data, particularly MRI-based biomarkers, are commonly used as input features for modelling SCD to MCI/AD conversion.

3.1.2 Handling Missing Data

Managing missing data was a critical consideration across studies. In Lerch et al. 2024, predictors with more than 30% missing values were excluded to maintain data quality. For the remaining variables, the mean was used to impute missing continuous values, and the mode was applied for categorical features. Similarly, Grassi et al. 2019, which focused on conversion from MCI to AD, imputed continuous features using the median. These methods ensure minimal loss of data while avoiding biases introduced by more complex imputations.

3.1.3 Handling Class Imbalance

Class imbalance was addressed through several strategies. In Lerch et al. 2024, oversampling, undersampling, and SMOTE were compared, with oversampling identified as the most effective based on model performance. Conversely, Grassi et al. 2019 did not apply resampling techniques but instead relied on the ROC-AUC metric, which remains robust in imbalanced settings. These approaches highlight the necessity of carefully managing class imbalance to achieve reliable and unbiased predictive performance.

3.2 Classification Methodology

3.2.1 Models

A variety of classification models have been explored in the reviewed studies. In Lerch et al. 2024, ensemble methods including RF, Treebag, and XGBoost were employed. In contrast, Yue et al. 2020 applied cost-sensitive SVMs to directly account for class imbalance. Dolcet-Negre et al. 2023 adopted Orthogonal Partial Least Squares (OPLS), a statistical modelling technique particularly used for multivariate data analysis. Additionally, Grassi et al. 2019 developed a range of ML models such as LR, SVC, kNN, MLP, RF, and Gradient Tree Boosting of Decision Trees (GTB), and integrated them using a rank-based weighted ensemble. Therefore, ensembling approaches serve to combine the strengths of multiple models, often resulting in improved predictive accuracy.

3.2.2 Evaluation

Evaluation strategies varied across studies to ensure robust and generalisable performance. In Lerch et al. 2024, 50 random train-test splits were used to assess the stability and generalisability of model predictions. In Yue et al. 2020, a nested Leave-one-out Cross-Validation (LOOCV) was employed, offering an unbiased estimate particularly suitable for small datasets. Dolcet-Negre et al. 2023 applied 10-fold CV, balancing bias and variance effectively. Similarly, Grassi et al. 2019 used 5-fold CV for model evaluation. The consistent emphasis on rigorous cross-validation underlines the importance of robust evaluation frameworks in clinical prediction studies, especially when working with limited and imbalanced datasets.

3.3 Feature Selection

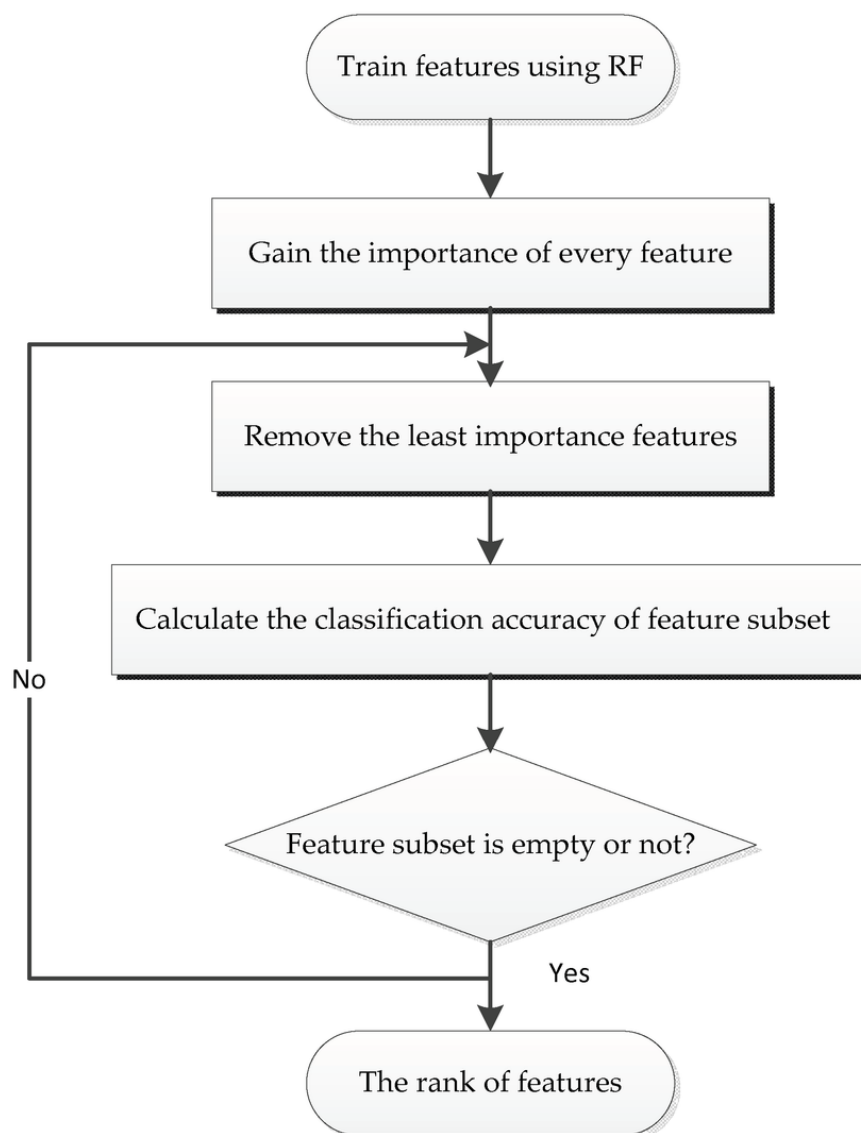
3.3.1 Multicollinearity

Multicollinearity refers to a high degree of linear intercorrelation among explanatory variables in a ML model, which can lead to unreliable or misleading results (Hae 2019). In Lerch et al. 2024, multicollinear features were identified by first filtering pairwise-correlated continuous variables using a threshold correlation coefficient of 0.75 or 0.5, depending on the feature group. Subsequently, features with a Variance Inflation Factor (VIF) greater than five were excluded. This systematic process helped ensure that the final set of input predictors was not adversely affected by multicollinearity.

3.3.2 Recursive Feature Elimination (RFE)

Recursive Feature Elimination (RFE) is a feature selection algorithm that recursively removes the least important features based on their contribution to model performance (Priyatno and Widiyaningtyas 2024). This iterative process ranks the features by importance, allowing the identification of the most influential features for the prediction task. Grassi et al. 2019 applied RFE for automatic feature selection using both LR and RF models. This process aims to identify the most influential features in the model performance. The workflow of RFE using a RF model is illustrated in Figure 3.1.

Figure 3.1: RFE algorithm with RF Model (Shi et al. 2021).



3.4 Summary

Table 3.1: Summary of Previous Studies on Predicting AD Conversion

Study Details	Model Input	Classification Model	Evaluation Strategy	Conversion Period
Lerch et al. 2024. (Predicting conversion from SCD to MCI/AD)	Three feature sets: (1a) socio-demographic and clinical information with MMSE and GDS scores, (1b) socio-demographic data combined with a battery of neuropsychological test scores, and (2) regional MRI grey matter volumes	RF, Treebag, and XGBoost	50 random train-test splits	16 years
Dolcet-Negre et al. 2023. (Predicting conversion from SCD to MCI/AD)	51 Structural-MRI measures	OPLS	10-fold CV	8 years
Yue et al. 2020. (Predicting conversion from SCD to MCI)	198 baseline neuroimaging features and 25 clinical measurements	Cost-sensitive SVMs	LOOCV	7 years
Grassi et al. 2019. (Predicting conversion from MCI to AD)	Socio-demographic information, clinical information and neuropsychological test scores	Base models include LR, SVC, kNN, MLP, RF, and GTB; Rank-based weighted ensembling of base models	5-fold CV	3 years

Chapter 4

Preprocessing

4.1 Development Environment

Regarding the software environment used for data preprocessing (Chapter 4), ML model development (Chapter 5), and model evaluation (Chapter 6), Visual Studio Code served as the Integrated Development Environment (IDE), and Python 3.10.11 was used as the programming language. The Python libraries utilised in the project and their respective versions are listed in Table 4.1.

Table 4.1: Python libraries used in the development

Library	URL	Version
scikit-learn	https://scikit-learn.org/1.5/index.html	1.5.2
XGBoost	https://xgboost.readthedocs.io/en/release_2.1.0/	2.1.3
Imbalanced-learn	https://imbalanced-learn.org/stable/	0.13.0
Matplotlib	https://matplotlib.org/stable/index.html	3.10.0
Pandas	https://pandas.pydata.org/docs/whatsnew/index.html	2.2.3
NumPy	https://numpy.org/doc/1.26/index.html	1.26.4
Seaborn	https://seaborn.pydata.org/index.html	0.13.2

4.2 Data Collection

The data used in this project was obtained from the Alzheimer’s Disease Neuroimaging Initiative (ADNI). All features, except for plasma biomarkers, were sourced from the ADNIMERGE dataset, which consolidates key variables from various case report forms and biomarker lab summaries across multiple ADNI protocols (ADNI1, ADNIGO, and ADNI2).

The plasma NFL data was obtained from the Blennow Lab dataset, which includes the analysis of NFL in plasma samples from ADNI1. The plasma PTAU (PTAU-181) data were sourced from the University of Gothenburg Longitudinal Plasma P-tau181 Dataset, which integrates p-tau measurements from different ADNI protocols. Plasma ABETA-40 and ABETA-42 data were collected from multiple sources, including the Bateman Lab, the University of Washington, the Foundation for the National Institutes of Health (FNIH), and the University of Pennsylvania (UPENN) datasets. To account for unit differences between datasets and to improve relevance to Alzheimer’s progression, the plasma ABETA-42/40 ratio was calculated.

The code for combining plasma data with the data from ADNIMERGE is presented in Appendix B.

4.3 Data Exploration

The features from ADNIMERGE and the separately obtained plasma data are merged to create a comprehensive dataset, which includes a wide range of variables designed to aid in understanding and assessing the progression of AD. In total, 50 features were selected for building the models, consisting of 3 categorical features and 47 continuous features. Table 4.2 and Table 4.3 present the descriptive statistics for continuous and categorical features in the 10-year conversion dataset, respectively. Additionally, the distribution of continuous features grouped by conversion status in the 10-year conversion dataset is visualised in Figure 4.1. Moreover, Section 4.3.1 provides a detailed description of these features.

4.3.1 Selecting Features

The features used in this project encompass a wide range of data types, including demographic information, genetic markers, Positron Emission Tomography (PET) imaging data, MRI volumetric data, Cerebrospinal Fluid (CSF) biomarkers, plasma biomarkers, and various cognitive and neuropsychological test scores. These diverse features are integral to building a comprehensive model for predicting Alzheimer’s disease progression.

Demographic Information

The demographic information include age, gender(PTGENDER), years of education (PTEDUCAT), and marital status (PTMARRY). These features provide basic participant background.

Genetic Markers

The $\epsilon 4$ allele count of the apolipoprotein E gene (APOE4) is an established genetic risk factor for Alzheimer’s Disease, with a number of studies demonstrating its strong association with increased susceptibility and accelerated disease progression (Corder et al. 1993; Liu et al. 2013). The number of APOE4 alleles carried by an individual, often referred to as the $\epsilon 4$ count, has been extensively examined for its impact on the onset and severity of AD.

PET Imaging Data

Positron Emission Tomography (PET) data includes fluorodeoxyglucose (FDG) and florbetapir (AV45) imaging biomarkers. The FDG value in ADNIMERGE represents the average FDG-PET signal across the angular gyrus, temporal cortex, and posterior cingulate regions, reflecting regional glucose metabolism. The AV45 value in ADNIMERGE corresponds to the average standardised uptake value ratio (SUVR) calculated using Pittsburgh Compound B (PIB) across the frontal cortex, anterior cingulate, precuneus cortex, and parietal cortex relative to the cerebellum, providing an estimate of amyloid deposition in the brain. These imaging biomarkers play a critical role in identifying abnormal brain activity and amyloid pathology (Jagust et al. 2010; Landau et al. 2011).

MRI Volumetric Measures

The MRI volumetric measures include the volumes of the Ventricles, Hippocampus, Whole Brain, Entorhinal Cortex, Fusiform Gyrus, Middle Temporal Gyrus (MidTemp), and Intracranial Volume (ICV), all measured in cubic millimeters (mm^3). These MRI measures capture the structural brain changes associated with Alzheimer's Disease (C. R. Jr Jack et al. 2010; Frisoni et al. 2010).

CSF Biomarkers

Cerebrospinal Fluid (CSF) biomarkers include amyloid-beta 42 (ABETA), total tau (TAU), and phosphorylated tau (PTAU), which serve as key indicators of Alzheimer's disease pathology, reflecting amyloid deposition and neurodegeneration (Blennow et al. 2015; C. R. Jr. Jack et al. 2016).

Plasma Biomarkers

Plasma biomarkers include the amyloid-beta 42/40 ratio (PLASMA_ABETA42/40), phosphorylated tau (PLASMA_PTAU), and neurofilament light chain (PLASMA_NFL), which serve as non-invasive indicators of Alzheimer's disease-related biological changes and neurodegeneration (Palmqvist et al. 2019; Nakamura et al. 2018; Mattsson et al. 2019).

Cognitive and Neuropsychological Assessments

Cognitive function is assessed through standardised cognitive and neuropsychological tests. The Clinical Dementia Rating Sum of Boxes (CDRSB) and Alzheimer's Disease Assessment Scales (ADAS11, ADAS13, ADASQ4) are widely used to evaluate the severity of cognitive impairment (Morris 1997; Williams et al. 2013; Rosen, Mohs, and Davis 1984; Kueper, Speechley, and Montero-Odasso 2018). General cognitive screening is performed using the Mini-Mental State Examination (MMSE) and Montreal Cognitive Assessment (MoCA) (Tombaugh and McIntyre 1992; Nasreddine et al. 2005).

Verbal memory performance is assessed through the Rey Auditory Verbal Learning Test (RAVLT) and its variations, along with the Logical Memory Delayed Recall score (LDEL-TOTAL) belonging to the Wechsler Memory Scale – Revised (WMS-R) (Rey 1964; Estévez-González et al. 2003; Wechsler 1987). Executive function and processing speed are measured using the Trail Making Test B (TRABSCOR) and Digit Symbol Substitution (DIGITSCOR) (Ashendorf et al. 2008; Williamson et al. 2022).

Additionally, parts of a ADNI modified version of the Preclinical Alzheimer Cognitive Composite (PACC), a more sensitive measure of early cognitive change, are included: mPACCtrailsB (ADNI modified PACC with Trails B) and mPACCdigit (ADNI modified PACC with Digit Symbol Substitution) (Donohue et al. 2014; Shen et al. 2021).

Subjective Cognitive Assessments

In addition to objective cognitive assessments, subjective cognitive decline is evaluated using the Everyday Cognition (Ecog) Questionnaire (Farias et al. 2008). This questionnaire includes self-reported (EcogPT) and study-partner reported (EcogSP) ratings of various cognitive domains, including memory (Mem), language (Lang), visuospatial abilities (Visspat), planning

(Plan), organisation (Organ), and divided attention (Divatt). These scores are particularly useful for detecting early-stage cognitive decline, making them valuable in predicting the conversion from SCD to MCI/AD (Thabtah, Spencer, and Ye 2020; Marshall et al. 2014).

Table 4.2: Descriptive statistics of Continuous Features in 10-year Conversion Data (Note: Std is Standard Deviation)

Continuous Features	Stay in SCD		Convert to MCI/AD	
	Mean	Std	Mean	Std
AGE	73.78	5.95	75.08	5.73
PTEDUCAT	17.23	2.47	16.18	2.55
FDG	1.33	0.10	1.24	0.14
AV45	1.06	0.10	1.19	0.22
ABETA	1359.23	372.93	1050.71	468.77
TAU	222.02	83.77	259.61	92.44
PTAU	19.84	7.76	24.43	9.90
PLASMA_NFL	33.65	22.48	35.21	19.51
PLASMA_PTAU	18.03	14.77	18.47	9.50
PLASMA_ABETA42/40	1.00	1.23	0.29	0.41
CDRSB	0.01	0.05	0.07	0.18
ADAS11	4.97	2.51	6.89	3.20
ADAS13	7.66	3.33	10.97	4.74
ADASQ4	2.43	1.47	3.54	2.01
MMSE	29.29	1.07	28.95	1.20
RAVLT_immediate	47.32	9.15	40.86	9.02
RAVLT_learning	6.29	2.17	5.28	2.54
RAVLT_forgetting	3.51	2.92	4.11	2.78
RAVLT_perc_forgetting	30.55	29.43	42.63	29.18
LDELTOTAL	13.89	3.11	12.05	3.42
DIGITSCOR	49.38	9.16	44.52	8.74
TRABSCOR	70.33	28.41	92.42	41.16
FAQ	0.05	0.26	0.38	1.35
MoCA	26.49	1.97	24.25	2.42
EcogPtMem	1.50	0.40	1.78	0.53
EcogPtLang	1.29	0.36	1.58	0.48
EcogPtVisspat	1.11	0.21	1.24	0.37
EcogPtPlan	1.07	0.19	1.30	0.41
EcogPtOrgan	1.22	0.42	1.43	0.47
EcogPtDivatt	1.31	0.40	1.73	0.68
EcogPtTotal	1.26	0.27	1.52	0.40
EcogSPMem	1.20	0.25	1.54	0.60
EcogSPLang	1.08	0.15	1.26	0.37
EcogSPVisspat	1.03	0.08	1.12	0.21
EcogSPPlan	1.06	0.14	1.23	0.49
EcogSPOrgan	1.14	0.30	1.33	0.61
EcogSPDivatt	1.23	0.49	1.48	0.63
EcogSPTotal	1.11	0.15	1.32	0.40
Ventricles	31229.21	18627.27	39636.91	21414.56
Hippocampus	7517.85	948.24	7146.60	823.97

Continuous Features	Stay in SCD		Convert to MCI/AD	
	Mean	Std	Mean	Std
WholeBrain	1041233.75	95055.48	1032031.39	100913.22
Entorhinal	3966.67	615.18	3760.03	710.32
Fusiform	18222.29	2400.10	17759.42	2335.74
MidTemp	20746.61	2478.80	20111.00	2725.78
ICV	1518244.71	147919.75	1524138.47	163670.33
mPACCdigit	0.85	2.16	-1.20	2.93
mPACCtrailsB	0.86	2.15	-1.22	2.70

Table 4.3: Descriptive statistics of Categorical Features in 10-year Conversion Data

Categorical Feature	Stay in SCD		Convert to MCI/AD	
	Count	Percentage	Count	Percentage
PTGENDER				
Male	43	49.43%	77	55.80%
Female	44	50.57%	61	44.20%
PTMARRY				
Married	62	71.26%	100	72.46%
Widowed	11	12.64%	18	13.04%
Divorced	10	11.49%	14	10.14%
Never married	4	4.60%	6	4.35%
APOE4				
0	67	77.01%	85	61.59%
1	20	22.99%	47	34.06%
2	0	0	6	4.35%

Figure 4.1: Density plots of numerical features in 10-year conversion dataset

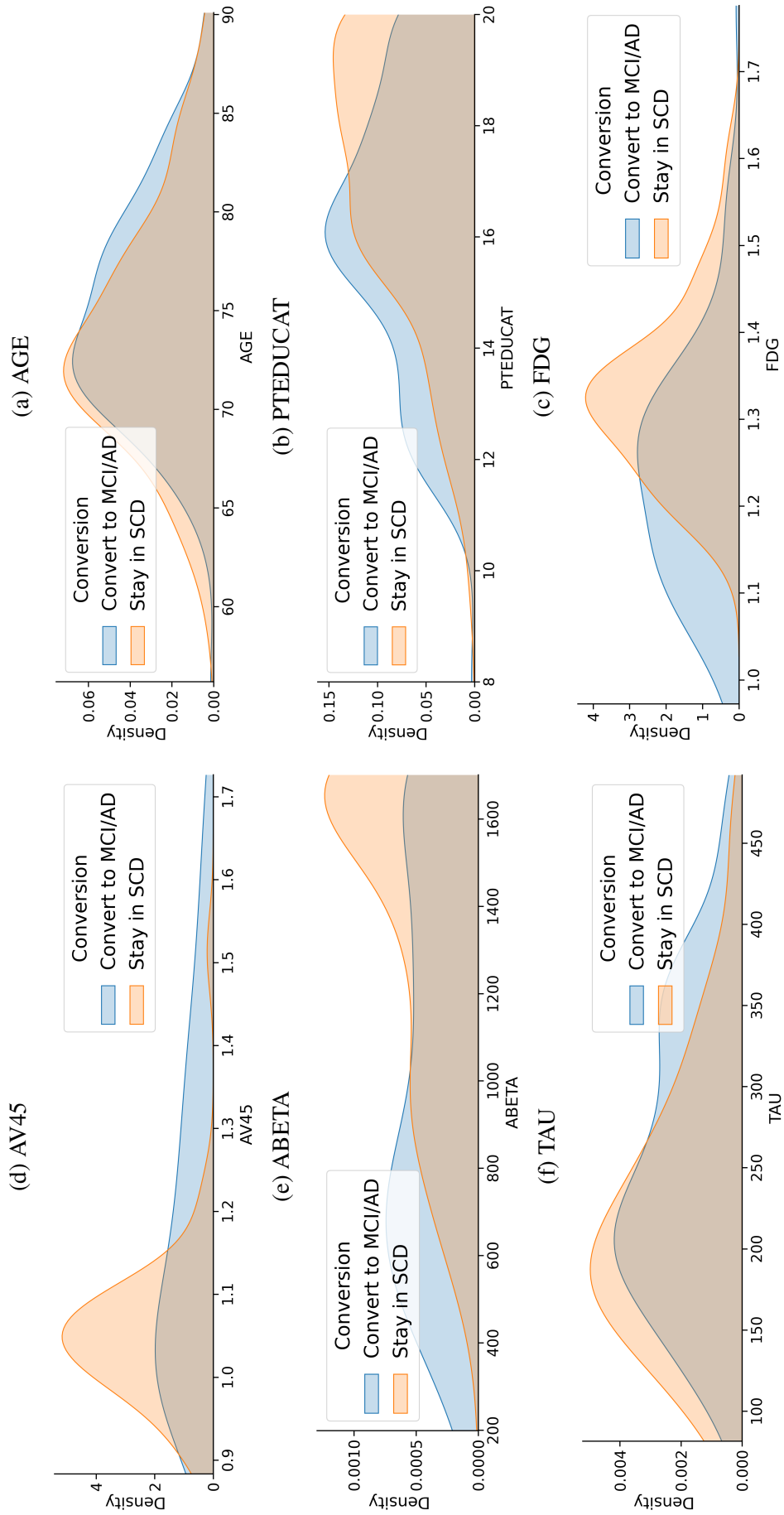


Figure 4.1: Density plots of numerical features in 10-year conversion dataset (continued)

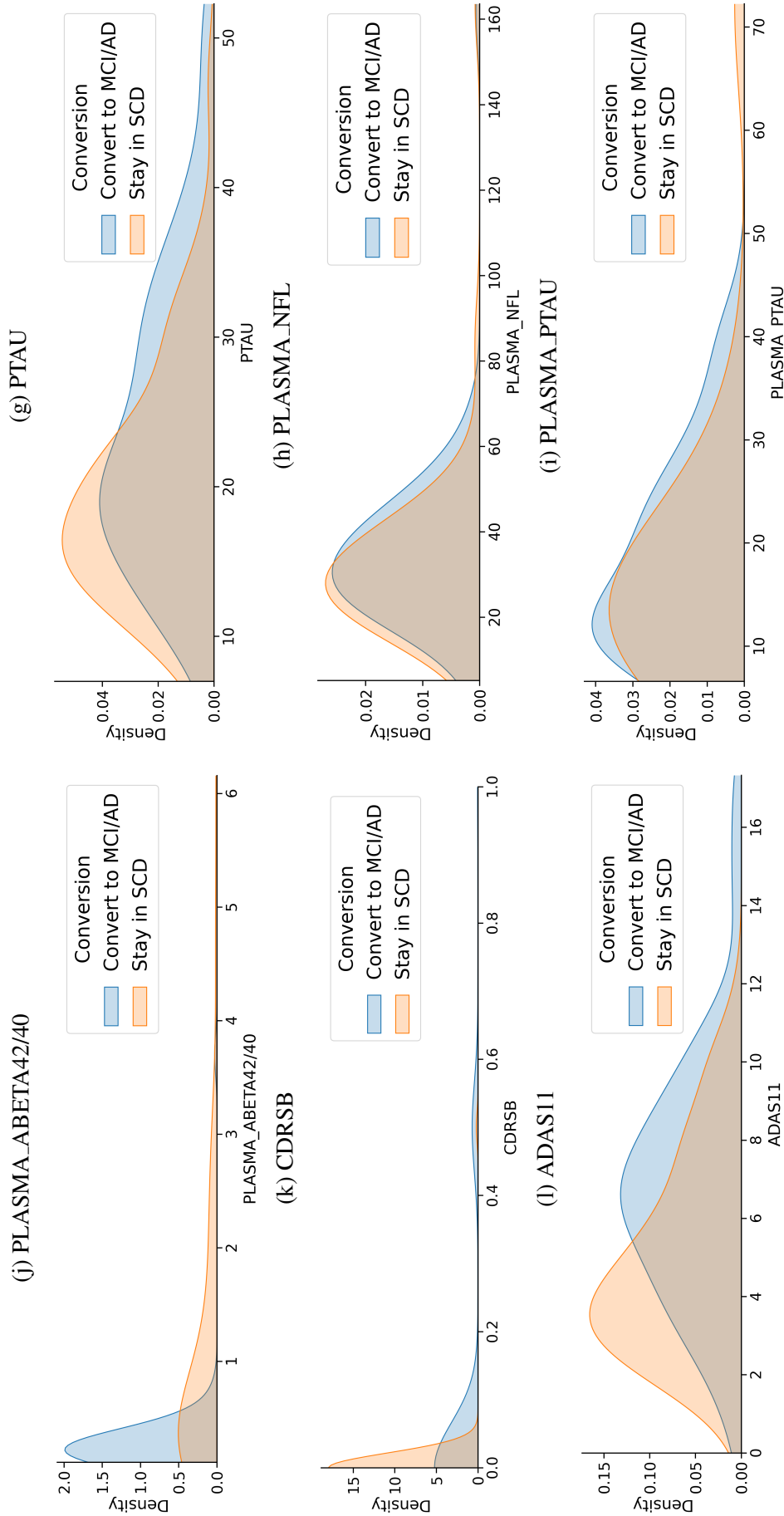


Figure 4.1: Density plots of numerical features in 10-year conversion dataset (continued)

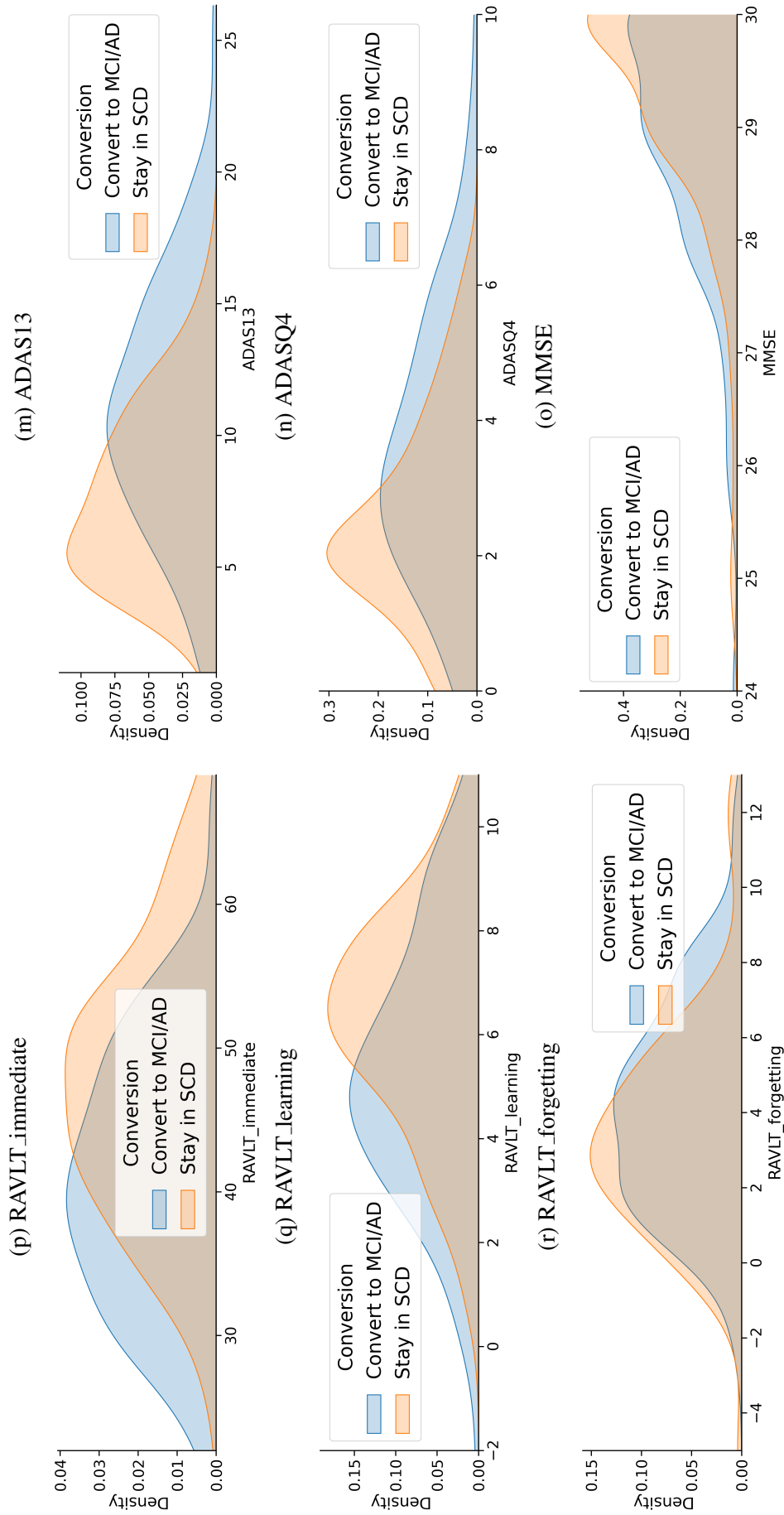


Figure 4.1: Density plots of numerical features in 10-year conversion dataset (continued)

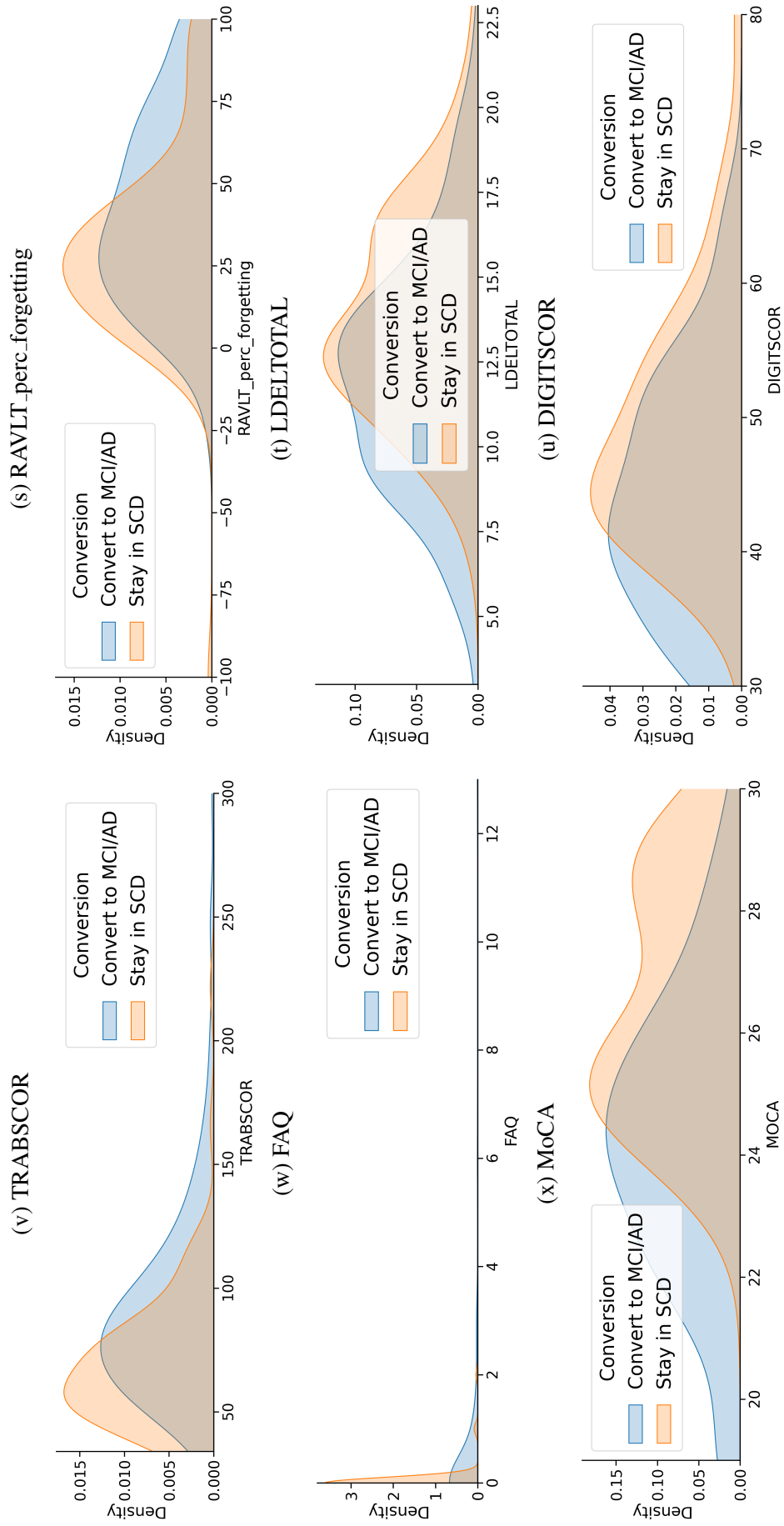


Figure 4.1: Density plots of numerical features in 10-year conversion dataset (continued)

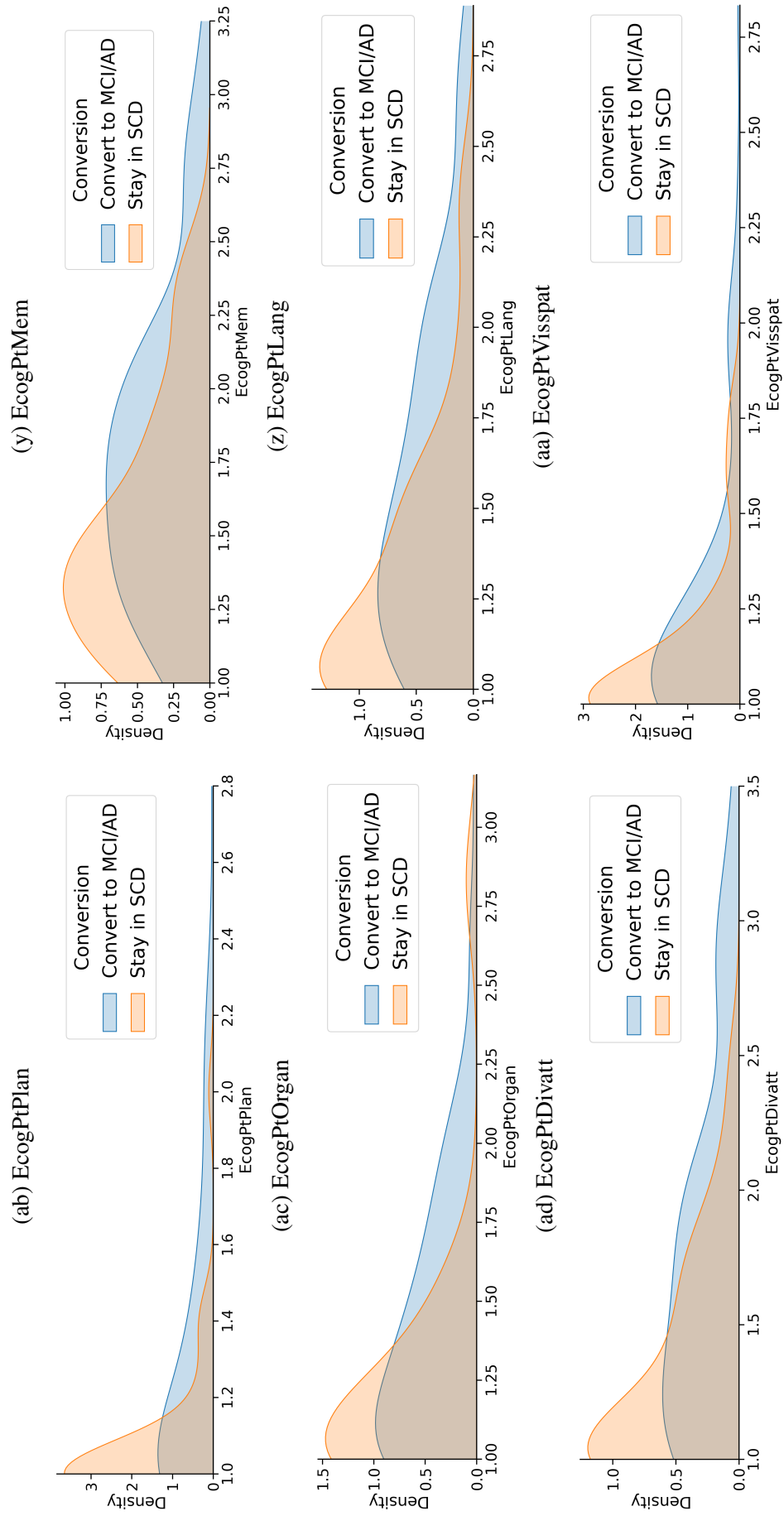


Figure 4.1: Density plots of numerical features in 10-year conversion dataset (continued)

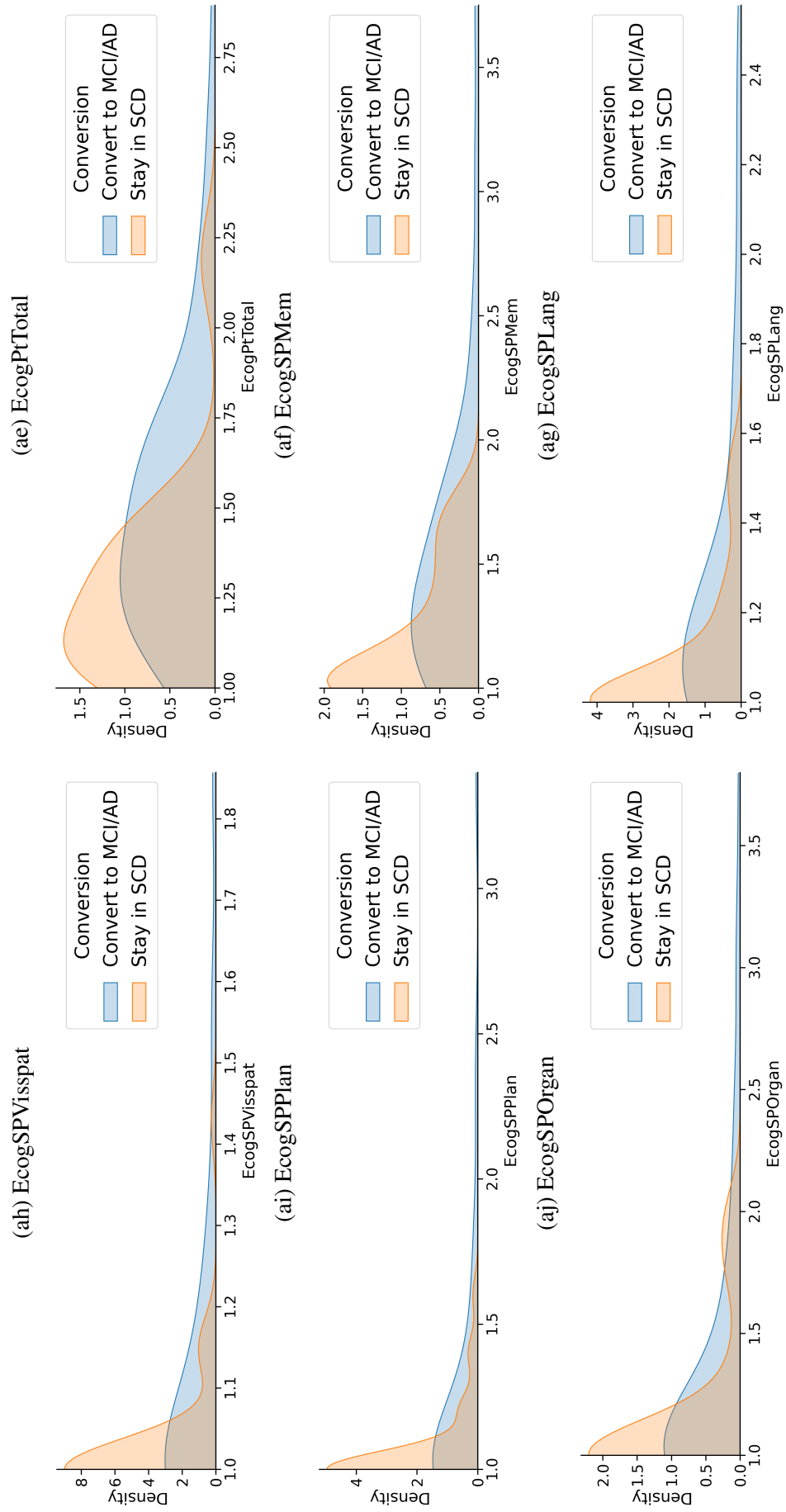


Figure 4.1: Density plots of numerical features in 10-year conversion dataset (continued)

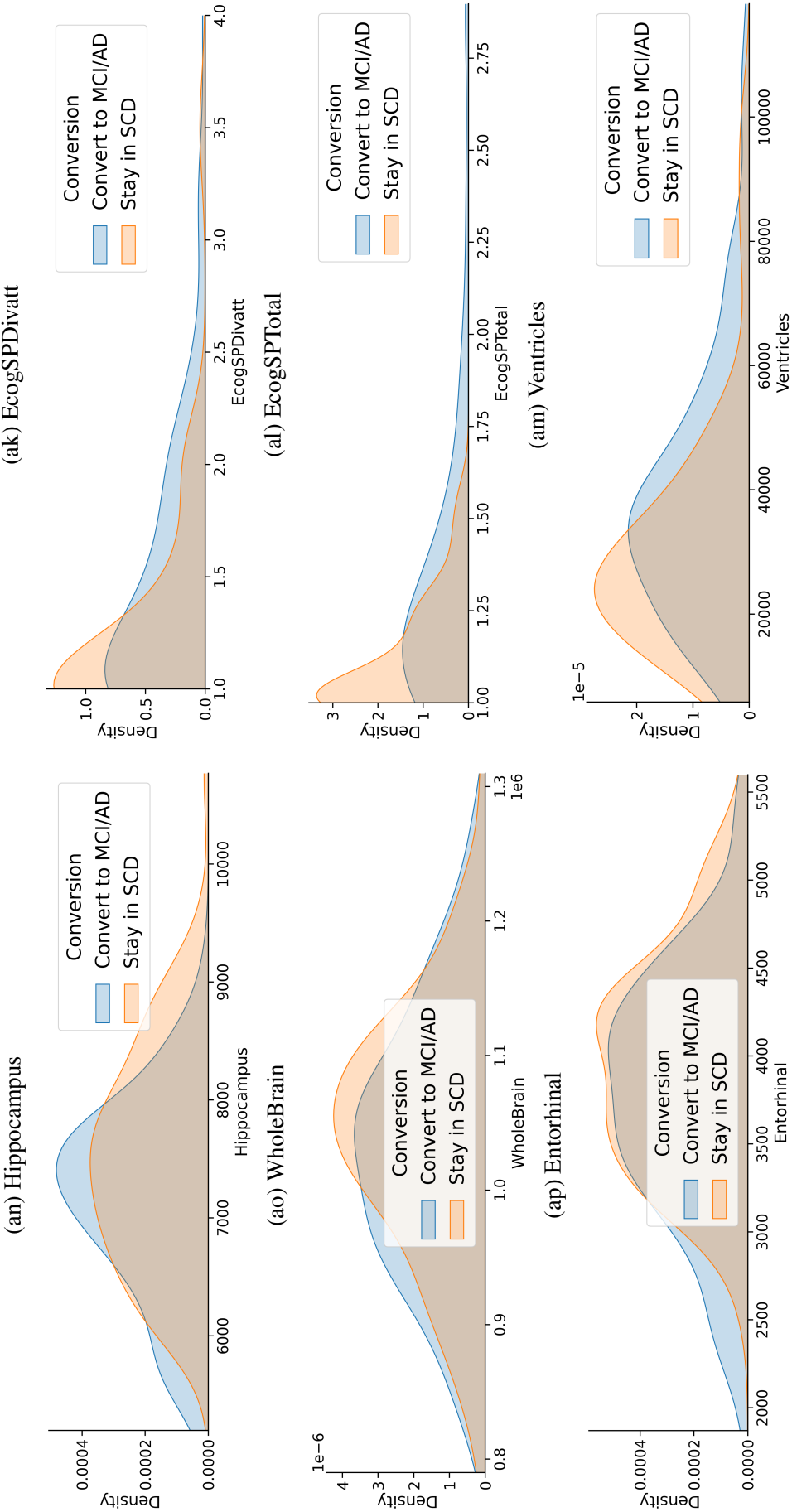
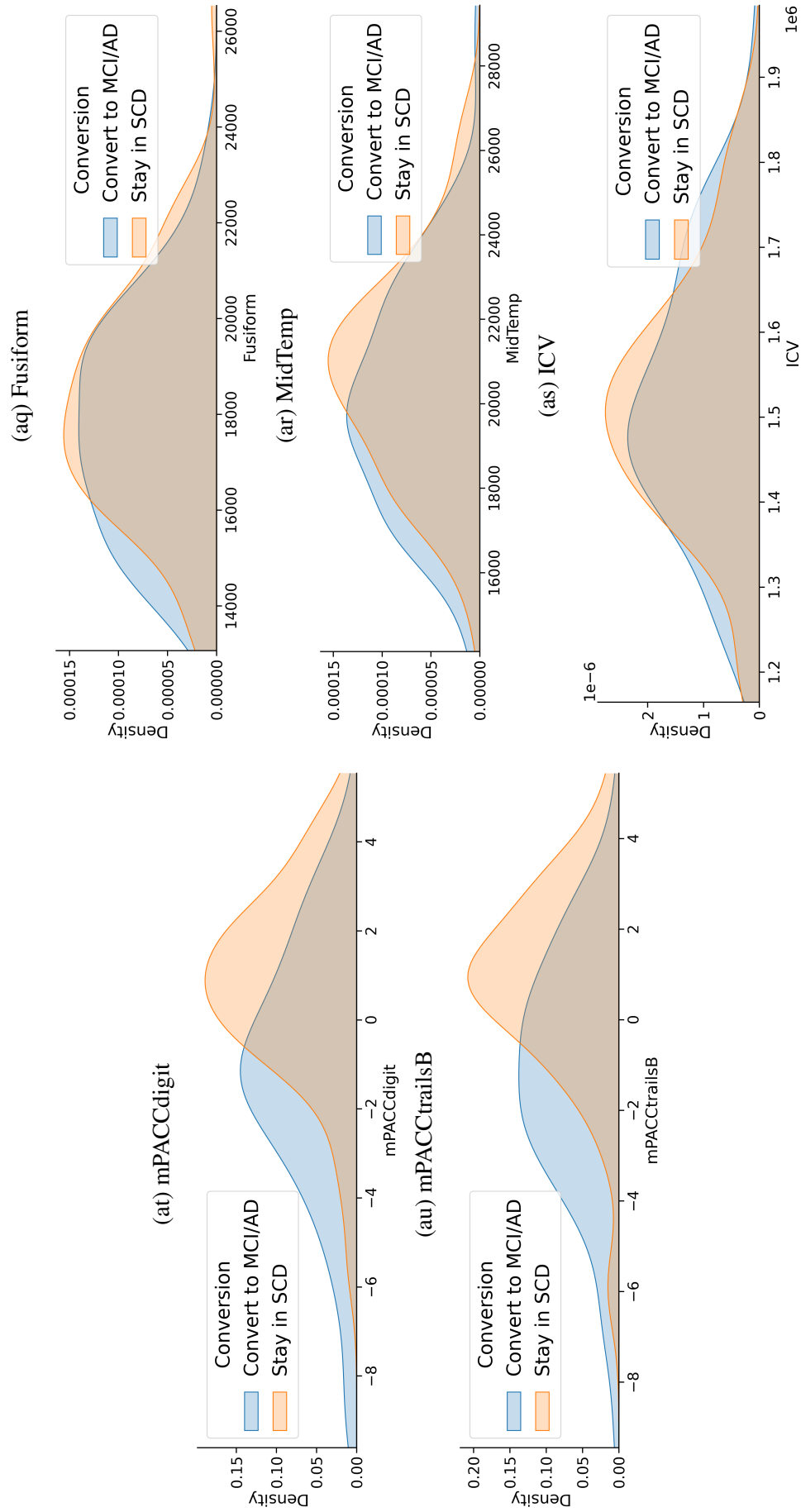


Figure 4.1: Density plots of numerical features in 10-year conversion dataset (continued)



4.3.2 Selecting Patients

Due to the subjective nature of diagnosing SCD, there is a potential misclassification of SCD patients as Cognitively Normal (CN) patients (Rabin et al. 2020; Karr et al. 2024). This misclassification can lead to individuals with SCD progressing to MCI/AD without appropriate intervention. Therefore, this project includes patients diagnosed as CN in the SCD group, while acknowledging the diagnostic challenges associated with SCD. Appendix A summarises the ADNI criteria for identifying individuals in different stages of AD continuum, including CN, SCD, MCI and AD.

Furthermore, only data from the baseline visit (i.e., the patient's first visit) was used for model development. This decision was based on the observation that subsequent visits exhibited significantly higher levels of missing data, rendering baseline data more reliable for consistent for model development. Table 4.4 compares the percentage of missing data between the baseline and all-visit datasets for the 10-year conversion task. For the final dataset used to predict conversion from SCD to MCI/AD within 10 years, a total of 225 individuals were included, of whom 138 converted to MCI/AD within the 10-year period, while 87 remained stable in the SCD stage. The code to construct the 10-year conversion dataset with individuals staying in SCD over 10 years and individuals converting to MCI/AD within 10 years, is presented in Appendix C

Table 4.4: Percentage of Missing Data at Baseline Visit Vs All visits in 10-year Conversion Data

Feature	At Baseline visit (Missing %)	At All visits (Missing %)
PLASMA_PTAU	79.56	86.59
AV45	55.56	84.57
DIGITSCOR	51.11	64.42
MoCA	50.67	70.29
EcogSPOrgan	50.22	71.09
EcogSPVisspat	49.78	70.65
EcogSPPlan	49.78	70.43
EcogPtVisspat	49.33	70.00
EcogSPDivatt	49.33	70.72
EcogSPLang	49.33	70.29
EcogPtMem	49.33	69.93
EcogSPMem	49.33	70.29
EcogPtPlan	49.33	69.93
EcogSPTotal	49.33	70.29
EcogPtLang	49.33	69.93
EcogPtTotal	49.33	69.93
EcogPtDivatt	49.33	70.07
EcogPtOrgan	49.33	70.00
PLASMA_NFL	44.00	80.65
TAU	37.78	80.07
PTAU	37.78	80.07
ABETA	37.78	80.07
PLASMA_ABETA42/40	34.22	68.99
FDG	33.33	73.48

Variable	At Baseline visit (Missing %)	At All visits (Missing %)
Hippocampus	9.78	44.06
Entorhinal	8.44	45.00
Fusiform	8.44	45.00
MidTemp	8.44	45.00
Ventricles	3.56	38.55
WholeBrain	1.78	37.61
ICV	1.33	36.23
APOE4	0.44	0.07
ADAS11	0.00	34.35
CDRSB	0.00	34.57
ADAS13	0.00	34.42
ADASQ4	0.00	34.35
AGE	0.00	0.00
LDELTOTAL	0.00	46.81
FAQ	0.00	34.57
MMSE	0.00	34.28
PTGENDER	0.00	0.00
PTEDUCAT	0.00	0.00
RAVLT_immediate	0.00	34.35
RAVLT_forgetting	0.00	34.35
PTMARRY	0.00	0.00
RAVLT_learning	0.00	34.35
TRABSCOR	0.00	34.71
RAVLT_perc_forgetting	0.00	34.35
mPACCdigit	0.00	34.35
mPACCtrailsB	0.00	34.28

4.4 Training and Test Datasets

For the 5-fold CV evaluation presented in Section 6, the dataset is partitioned into five equally sized folds. In each of the five iterations, a distinct fold is designated as the test set while the remaining four folds are used for training. This approach ensures that each data point is used for testing exactly once, and for training four times. Consequently, for every version of the dataset—whether imputed, class-balanced, or subjected to feature selection—five separate training and testing pairs are constructed, enabling the calculation of average performance metrics in a consistent and unbiased manner.

4.5 Missing Data

Despite using only baseline visit data, a significant amount of missing data remains in the 10-year conversion dataset (Table 4.4). This issue may stem from integrating data from multiple sources and variations in data collection practices across different centers. A simple approach would be to remove patients with any missing data. However, given the limited dataset of only 225 samples, this would drastically reduce the available data, making model development impractical. Instead, different imputation methods discussed in Section 2.2.6 are applied to generate multiple datasets. Each dataset is then used to develop separate individual and

ensemble models (in Chapter 5), which are evaluated using 5-fold CV. The dataset imputed with the method that yields the best overall model performance, as assessed in Section 6.1.1, is selected as the optimal imputation strategy.

To ensure consistency in handling categorical and continuous features, the following steps along with imputation were implemented:

1. **Label Encoding for Categorical Features:** Categorical features were label encoded to a unique integer value before the imputation process.
2. **Handling Range Values in Continuous Data:** Continuous features that contained range values (e.g., "> 1700") were converted into actual integers by selecting the boundary value of the range (e.g., "> 1700" was converted to 1701) before the imputation process. This standardisation allows the data to be used uniformly for model training.
3. **Standardisation of Continuous Data:** After the imputation process, the continuous data were standardised to have a mean of 0 and a standard deviation of 1. This ensures that all continuous features are on the same scale, which is crucial for many machine learning algorithms to perform optimally. Standardisation was done by subtracting the mean and dividing by the standard deviation of each feature.

4.5.1 Mean + Mode Imputation

In the Median + Mode imputation approach, missing values in continuous features are imputed using the mean imputation technique, while missing values in categorical features are imputed using the mode imputation technique. Both the median and mode imputations are implemented using the `sklearn.impute.SimpleImputer` interface, specifying the strategy as "mean" for continuous and "most_frequent" for categorical features, respectively.

4.5.2 Median + Mode Imputation

In the Median + Mode imputation approach, missing values in continuous features are imputed using the median imputation technique, while missing values in categorical features are imputed using the mode imputation technique. Both the median and mode imputations are implemented using the `sklearn.impute.SimpleImputer` interface, specifying the strategy as "median" for continuous and "most_frequent" for categorical features, respectively.

4.5.3 Random Forest Imputation

Imputation using Random Forest model (discussed in 2.2.5) has proven effective for handling missing data in complex epidemiological datasets (Shah et al. 2014). This implementation uses the `sklearn.ensemble.RandomForestRegressor` for continuous features and `sklearn.ensemble.RandomForestClassifier` for categorical features.

For each feature with missing values, the imputation process treats it as a target variable, using the remaining features as predictors. Models are trained exclusively on rows where both the target and predictor features are fully observed, ensuring data completeness during training. Once trained, the models predict and impute the missing values.

4.5.4 MICE Imputation

The MICE imputation technique (discussed in Section 2.2.6) is implemented using the `sklearn.impute.IterativeImputer` class, with the Bayesian Ridge Regression model serving as the base model. In each iteration, the base model predicts the missing values of the target feature, using the other features as input predictors. This process is repeated for 10 iterations to iteratively refine the imputed values and improve the accuracy of the final imputation.

4.6 Class Imbalance

The class imbalance in the 10-year conversion dataset poses a significant challenge for model development. With 87 samples in the Stay in SCD class and 138 samples in the Convert to MCI/AD class, predictive models may become biased toward the majority class, thereby reducing their ability to accurately classify instances from the minority class (Ishwaran and O'Brien 2021; Salmi et al. 2024). To address this issue, the resampling techniques discussed in Section 2.2.7 are applied to the best-imputed dataset identified in Section 4.5, resulting in multiple class-balanced datasets. Each dataset is then used to develop separate individual and ensemble models (in Chapter 5), which are evaluated using 5-fold cross-validation. The class balancing method that achieves the best overall model performance, as evaluated in Section 6.1.1, is selected as the optimal strategy to be effective learning from both classes.

4.6.1 Implementation of Class-balancing Techniques

Table 4.5 lists the library instantiations used to implement the resampling techniques discussed in Section 2.2.7. To ensure reproducibility of results, all instantiations were configured with a fixed **random_state** parameter.

Table 4.5: Resampling techniques and their respective library implementations

Resampling Technique	Library Instantiation
Random Oversampling	<code>imblearn.over_sampling.RandomOverSampler()</code>
SMOTE	<code>imblearn.over_sampling.SMOTE()</code>
Borderline-SMOTE	<code>imblearn.over_sampling.BorderlineSMOTE()</code>
ADASYN	<code>imblearn.over_sampling.ASYN()</code>
Random Undersampling	<code>imblearn.under_sampling.RandomUnderSampler()</code>
Cluster Centroids	<code>imblearn.under_sampling.ClusterCentroids()</code>

4.6.2 Data Leakage

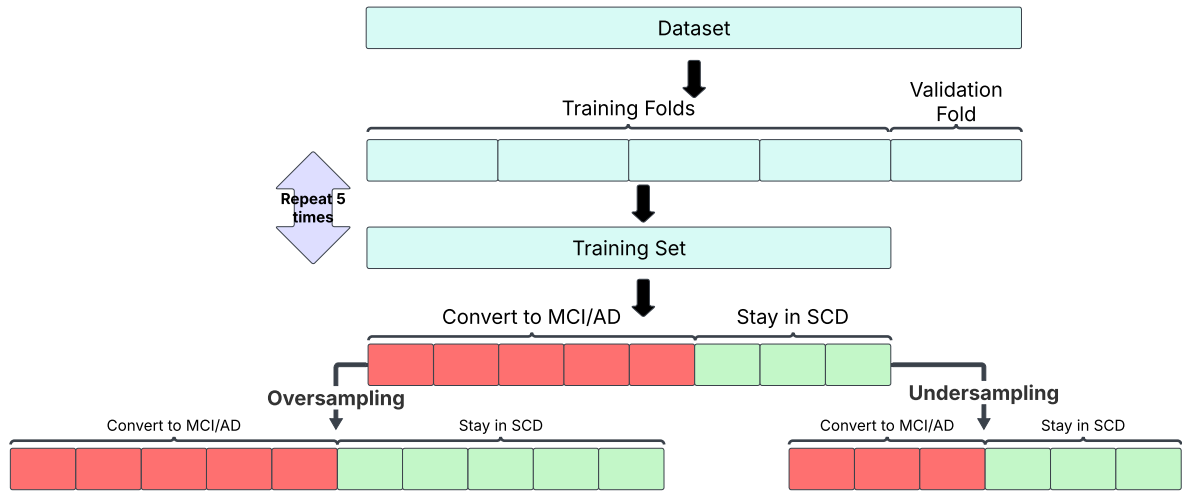


Figure 4.2: 10-year Conversion Dataset: Only resampling the training set to prevent data leakage.

The timing of data resampling is critical for reliable model evaluation. Applying oversampling before the train-test split (discussed in Section 4.4) can result in synthetic samples appearing in both sets, leading to data leakage and inflated performance metrics (Demircioğlu 2024). To prevent this, all resampling techniques in this study are applied exclusively to the training data after the train-test split in Section 4.4. This ensures that the evaluation of class balancing methods in Section 6 remains fair and unbiased. Figure 4.2 presents this process being applied on the 10-year conversion dataset, where the minority class is the "Stay in SCD" class clearly.

4.7 Feature Selection

Feature selection is employed to identify both the most informative and redundant features for predicting conversion from SCD to MCI/AD. By eliminating irrelevant or redundant features, this process enhances model accuracy, improves computational efficiency, and increases interpretability by highlighting the predictors most associated with AD progression. In this project, feature selection is performed using two complementary strategies:

1. **Manually defined feature groups** based on domain knowledge and criteria such as the absence of multicollinearity.
2. **Automated feature selection** techniques.

4.7.1 Manually Selected Feature Groups

Group A includes all 50 features derived from the ADNI dataset, as detailed in Section 4.3.1. Group B is a base subset consisting solely of demographic information, neuropsychological assessments, and clinical test scores. These features are selected for their non-invasive nature and ease of collection, making them suitable for routine clinical use. Feature groups C through G are constructed by incrementally building on Group B with additional features that are more invasive and costly, but more closely associated with Alzheimer's disease progression. This

hierarchical selection allows evaluation of the trade-off between predictive performance and the cost or invasiveness of data acquisition. Group H includes all features (Group A) except MRI and PET data, in alignment with one of the key objectives of this study—to develop predictive models that do not rely on expensive and resource-intensive imaging techniques. In addition, to address redundancy, features exhibiting multicollinearity (Section 3.3.1) were removed from Group A to form Group I (non-collinear features). The code used to identify features exhibiting multicollinearity is provided in Appendix D. Table 4.6 lists the features in each group.

Table 4.6: Feature Groups and Corresponding Features

Feature Group	Features
A - All	AGE, PTGENDER, PTEDUCAT, PTMARRY, APOE4, FDG, AV45, ABETA, TAU, PTAU, PLASMA_NFL, PLASMA_PTAU, PLASMA_ABETA42/40, CDRSB, ADAS11, ADAS13, ADASQ4, MMSE, RAVLT_immediate, RAVLT_learning, RAVLT_forgetting, RAVLT_perc_forgetting, LDELTOTAL, DIGITSCOR, TRABSCOR, FAQ, MoCA, EcogPtMem, EcogPtLang, EcogPtVisspat, EcogPtPlan, EcogPtOrgan, EcogPtDivatt, EcogPtTotal, EcogSPMem, EcogSPLang, EcogSPVisspat, EcogSPPlan, EcogSPOrgan, EcogSPDivatt, EcogSPTotal, Ventricles, Hippocampus, WholeBrain, Entorhinal, Fusiform, MidTemp, ICV, mPACCdigit, mPACCtrailsB
B - Base	AGE, PTGENDER, PTEDUCAT, PTMARRY, CDRSB, ADAS11, ADAS13, ADASQ4, MMSE, RAVLT_immediate, RAVLT_learning, RAVLT_forgetting, RAVLT_perc_forgetting, LDELTOTAL, DIGITSCOR, TRABSCOR, FAQ, MoCA, EcogPtMem, EcogPtLang, EcogPtVisspat, EcogPtPlan, EcogPtOrgan, EcogPtDivatt, EcogPtTotal, EcogSPMem, EcogSPLang, EcogSPVisspat, EcogSPPlan, EcogSPOrgan, EcogSPDivatt, EcogSPTotal, mPACCdigit, mPACCtrailsB
C - Base + CSF	Feature Group B + ABETA, TAU, PTAU
D - Base + PET	Feature Group B + FDG, AV45
E - Base + MRI	Feature Group B + Ventricles, Hippocampus, WholeBrain, Entorhinal, Fusiform, MidTemp, ICV
F - Base + Plasma	Feature Group B + PLASMA_NFL, PLASMA_PTAU, PLASMA_ABETA42/40
G - Base + APOE4	Feature Group B + APOE4
H - All Except MRI & PET	Feature Group A excluding FDG, AV45, Ventricles, Hippocampus, WholeBrain, Entorhinal, Fusiform, MidTemp, ICV

Continued on next page

Table 4.6 (continued)

Feature Group	Features
I - Non-collinear	AGE, PTGENDER, PTEDUCAT, PTMARRY, APOE4, FDG, AV45, ABETA, PTAU, PLASMA_NFL, PLASMA_PTAU, PLASMA_ABETA42/40, CDRSB, ADAS13, MMSE, RAVLT_immediate, RAVLT_learning, RAVLT_forgetting, LDELTOTAL, DIGITSCOR, TRABSCOR, FAQ, MoCA, EcogPtMem, EcogPtLang, EcogPtVisspat, EcogPtOrgan, EcogPtDivatt, EcogSPMem, EcogSPVisspat, EcogSPPlan, Ventricles, Hippocampus, Entorhinal, Fusiform, MidTemp, ICV

4.7.2 Automatic Feature Selection

RFE (discussed in Section 3.3.2) was implemented using the `sklearn.feature_selection.RFE` class to automatically identify the top 20 features contributing most to the model's predictive performance. This feature selection method was applied using two models: Random Forest and Logistic Regression.

Chapter 5

Model Construction

This section outlines the methodology of constructing the best base models through hyperparameter tuning and the best ensemble model through different combinations of base models and different ensembling techniques, for each dataset created in Section 4.

5.1 Hyperparameter Tuning

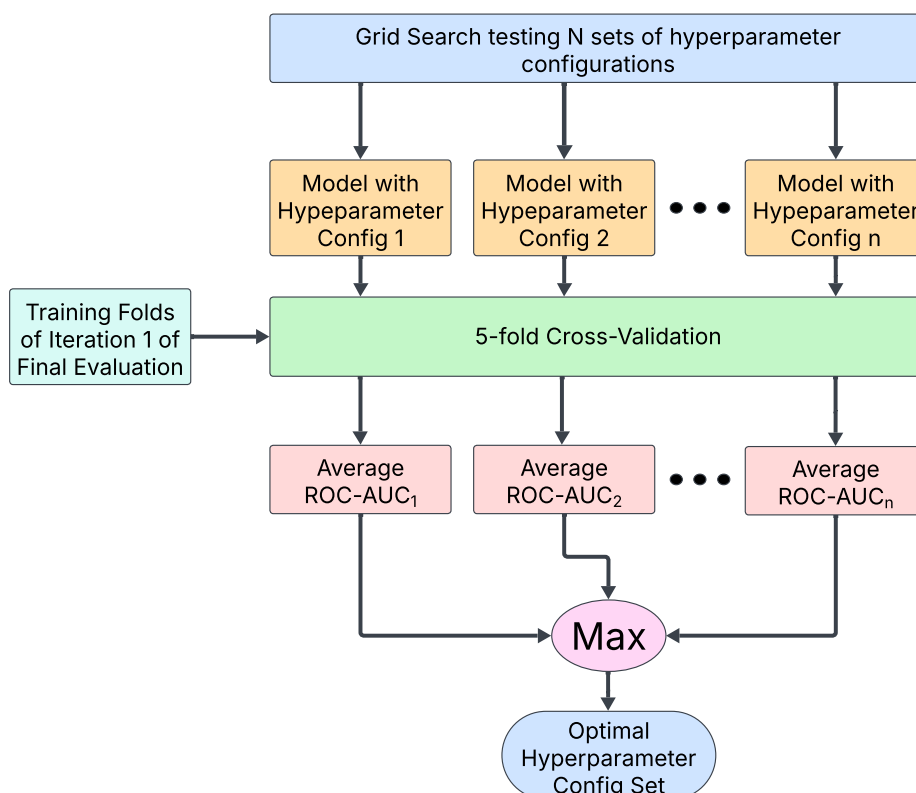


Figure 5.1: Selecting optimal hyperparameter configuration of base models.

The dataset is divided into five folds for a final 5-fold CV evaluation. In each of the five iterations in CV, four folds are used for training and one fold is used for validation.

Hyperparameter tuning is performed only using the training folds of the first iteration. This strategy minimises the risk of data leakage and overfitting, ensuring impartial model evaluation and realistic assessment of performance on unseen data.

The hyperparameter tuning process was conducted using grid search, where various configurations of hyperparameters were evaluated through 5-fold cross-validation on the training folds of the first iteration. The hyperparameter combination that produced the best average ROC-AUC score across the five folds was selected as the optimal set for each model. This process was implemented using the `sklearn.model_selection.GridSearchCV` class. Figure 5.1 provides a visual representation of the hyperparameter tuning procedure.

In addition, the **random_state** of all non-deterministic models is set to a **constant** number to allow reproducibility.

5.2 Logistic Regression (LR)

The `sklearn.linear_model.LogisticRegression` class was used to implement the LR model (discussed in Section 2.2.3). Table 5.1 lists the range of hyperparameter values explored during tuning during hyperparameter tuning.

Table 5.1: Hyperparameters and their values tested in tuning of LR Model

Hyperparameters	Values
C	0.01, 0.1, 1, 10
penalty	l1, l2
solver	liblinear, saga

5.3 Support Vector Classifier (SVC)

The SVC (discussed in Section 2.2.3) was implemented using the `sklearn.svm.SVC` class. Table 5.2 lists the hyperparameter values explored during hyperparameter tuning. Additionally, the `probability` argument was set to `True` to enable probability estimates, which are necessary for computing the ROC-AUC metric.

Table 5.2: Hyperparameters and their values tested in tuning of SVC Model

Hyperparameters	Values
C	0.01, 0.1, 1, 10, 100
kernel	linear, rbf, poly, sigmoid
gamma	scale, auto, 0.001, 0.01, 0.1

5.4 k-Nearest Neighbours (kNN)

The `sklearn.neighbors.KNeighborsClassifier` class was used to implement the kNN model (discussed in Section 2.2.3). Table 5.3 lists the hyperparameter values explored during hyperparameter tuning.

Table 5.3: Hyperparameters and their values tested in tuning of kNN Model

Hyperparameters	Values
n_neighbors	3, 5, 7, 11, 15, 19, 21, 25
weights	uniform, distance
metric	euclidean, manhattan, minkowski

5.5 Multi-Layer Perceptron (MLP)

The `sklearn.neural_network.MLPClassifier` class was used to implement the MLP model (discussed in Section 2.2.3). Table 5.4 lists the hyperparameter values explored during hyperparameter tuning. Additionally, the `early_stopping` parameter was set to `True` to prevent overfitting and improve generalisation, with 10% of the training data reserved as a validation set during training.

Table 5.4: Hyperparameters and their values tested in tuning of MLP Model

Hyperparameters	Values
hidden_layer_sizes	(50,), (100,), (50, 50), (100, 50)
activation	relu, tanh, logistic
solver	adam, sgd
alpha	0.0001, 0.001, 0.01
learning_rate	constant, adaptive
max_iter	100, 200, 500, 1000, 2000

5.6 Decision Tree (DT)

The `sklearn.tree.DecisionTreeClassifier` class was used to implement the DT model (discussed in Section 2.2.3). Table 5.5 lists the hyperparameter values explored during hyperparameter tuning.

Table 5.5: Hyperparameters and their values tested in tuning of DT Model

Hyperparameters	Values
criterion	gini, entropy, log_loss
splitter	best, random
max_depth	None, 5, 10, 15, 20, 30
max_features	None, sqrt, log2

5.7 Gaussian Naive Bayes (Gaussian NB)

The `sklearn.naive_bayes.GaussianNB` class was used to implement the Gaussian NB model (discussed in Section 2.2.3). Table 5.6 lists the values tested for the `var_smoothing` parameter.

Table 5.6: Hyperparameters and their values tested in tuning of Gaussian NB Model

Hyperparameters	Values
var_smoothing	10^{-9} , 10^{-8} , 10^{-7} , 10^{-6} , 10^{-5} , 10^{-4} , 10^{-3}

5.8 Random Forest (RF)

The `sklearn.ensemble.RandomForestClassifier` class was used to implement the RF model (discussed in Section 2.2.5). Table 5.7 lists the hyperparameter values explored during hyperparameter tuning.

Table 5.7: Hyperparameters and their values tested in tuning of RF Model

Hyperparameters	Values
n_estimators	50, 100, 200
max_depth	None, 10, 20, 30
max_features	sqrt, log2

5.9 Extreme Gradient Boosting (XGBoost)

The `xgboost.XGBClassifier` class was used to implement the XGBoost model (discussed in Section 2.2.5). Table 5.8 lists the hyperparameter values explored during hyperparameter tuning. Additionally, the evaluation metrics were set using the `eval_metric` parameter to include both `logloss` and `auc` to guide model training.

Table 5.8: Hyperparameters and their values tested in tuning of XGBoost Model

Hyperparameters	Values
n_estimators	50, 100, 200
max_depth	3, 6, 9
learning_rate	0.01, 0.1, 0.3
max_features	0, 0.1, 0.2

5.10 Ensemble Model

Once the optimal hyperparameters are determined for each base model through grid search, the construction of the ensemble model and the selection of the best ensemble begin. The ensembling methods considered include stacking, voting, and bagging. Various combinations of base models are explored, ranging from a minimum of 2 to all 8 models. Each combination is evaluated for each ensembling method to determine the most effective ensemble model.

5.10.1 Combinations of Base Models

In the context of ensemble modeling, we explored various combinations of eight individual base models to identify the optimal ensemble configuration. Specifically, we considered all possible

combinations ranging from pairs (2 models) to the full set (8 models). The total number of such combinations is calculated as follows:

$$\sum_{i=2}^8 \binom{8}{i}$$

Where $\binom{8}{i}$ represents the number of ways to choose i models from a set of 8, calculated using the binomial coefficient formula:

$$\binom{n}{k} = \frac{n!}{k!(n-k)!}$$

Therefore, there are $\sum_{i=2}^8 \binom{8}{i} = \mathbf{247}$ possible combinations of the eight base models when selecting between 2 and 8 models. This comprehensive exploration ensures that a wide range of ensemble configurations are evaluated to identify the most effective model combinations for predicting conversion.

5.10.2 Stacking

The `sklearn.ensemble.StackingClassifier` class is used to implement the Stacking ensembling technique (discussed in Section 2.2.4) with the different combinations of base estimators (247 combinations), while the hyperparameter-tuned **Logistic Regression** or **Random Forest** model is selected as the final estimator (2 combinations). Therefore, 494 (247*2) different stacking classifier configurations (listed in Appendix F.1) were developed and tested.

The base models are trained using the entire training set, while the input for the final estimator is generated through the 5-fold CV. Specifically, the training set is divided into five folds, where four folds are used to train the base models, and the remaining fold serves as a validation set to generate predictions. This process is repeated for each fold, ensuring that every sample in the training set is used for validation once. The predictions obtained from the five iterations, along with the corresponding final MCI/AD conversion labels, serve as the input and expected output to train the final estimator.

Figure 5.2 shows the visual structure of the stacking ensemble model.

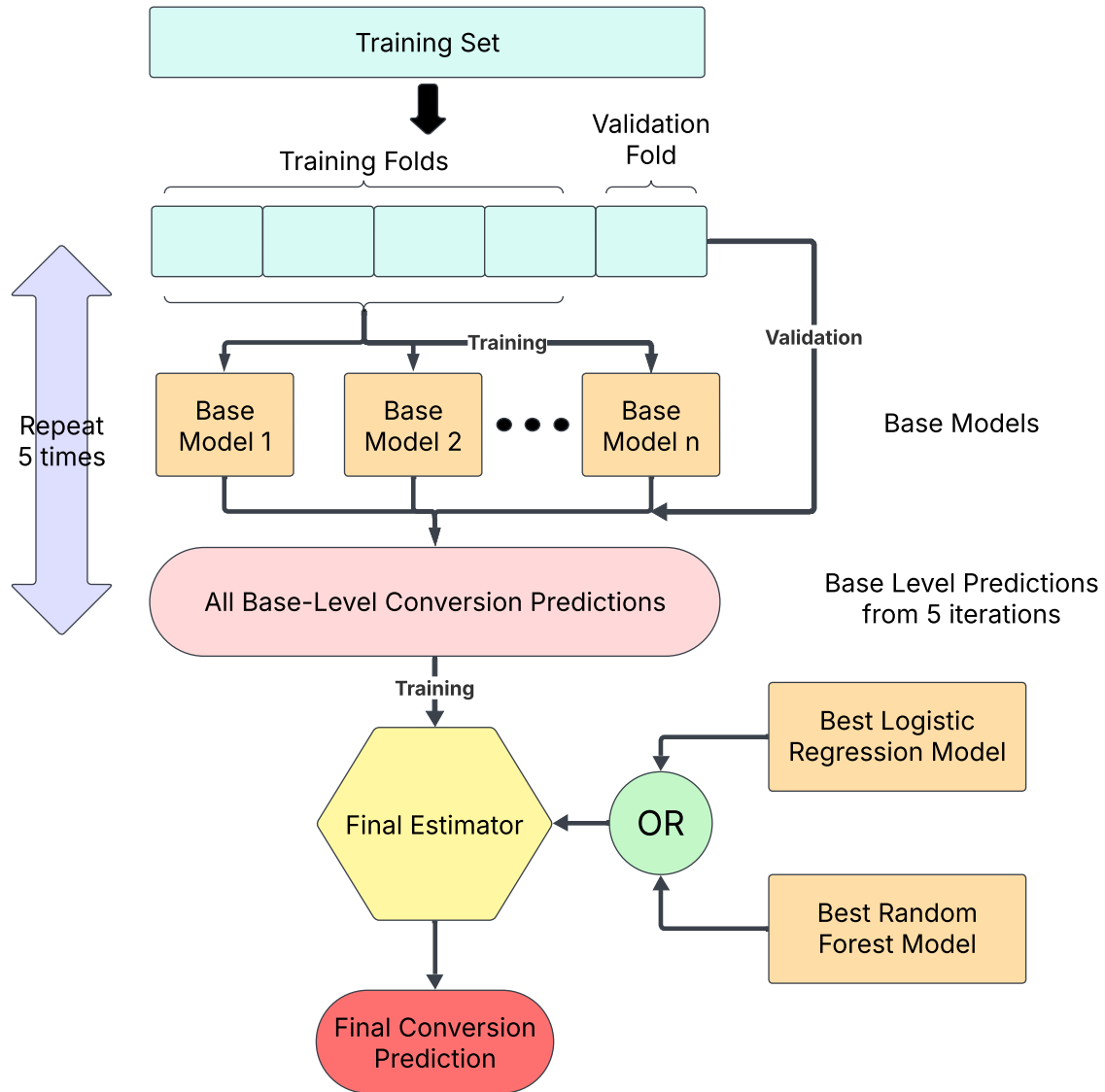
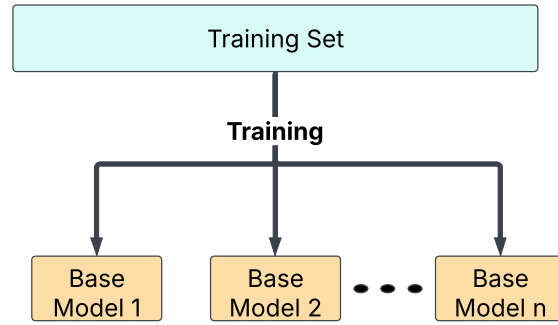


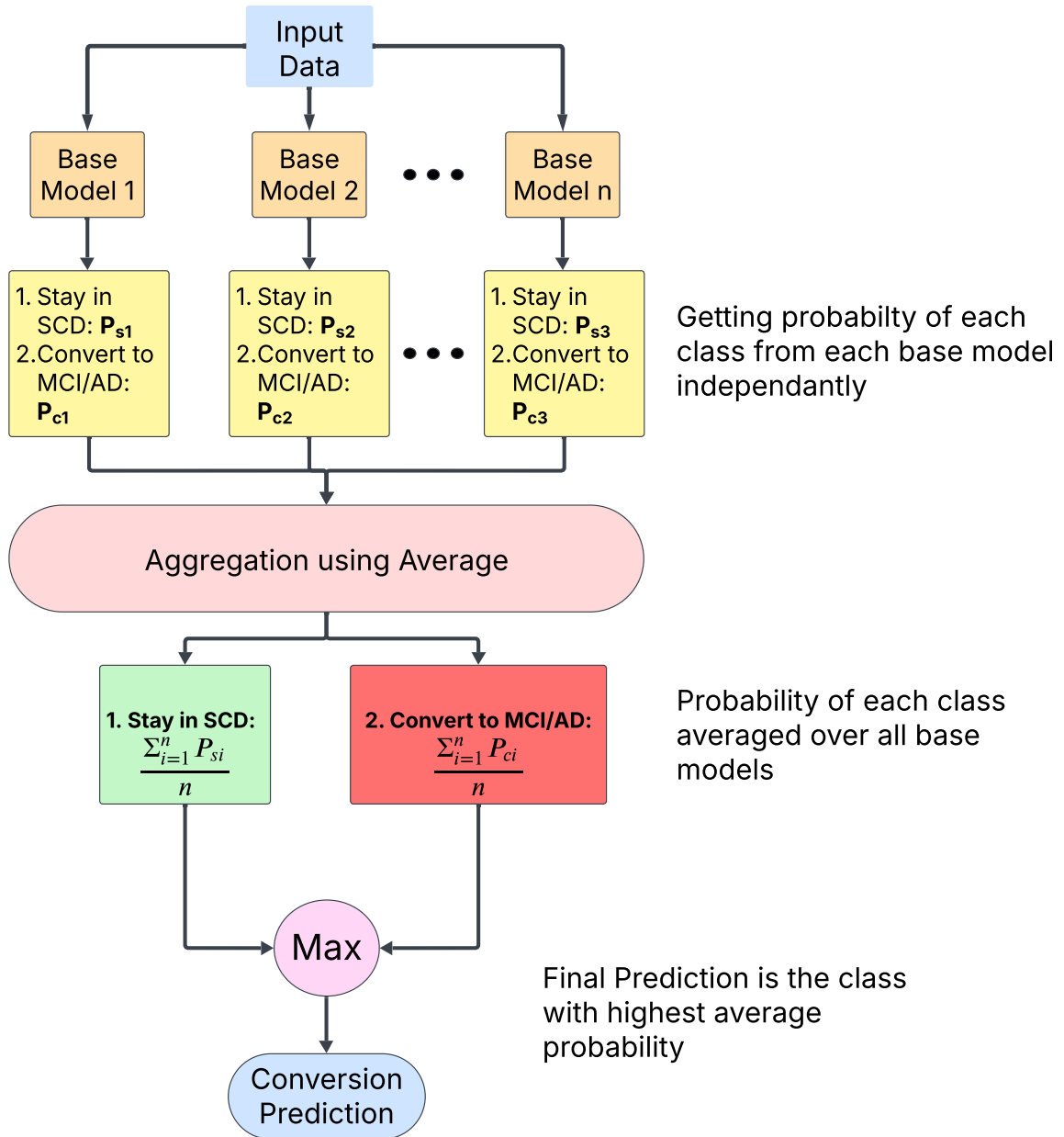
Figure 5.2: Structure of Stacking Ensemble Model

5.10.3 Voting (Soft)

The soft voting ensembling method (discussed in Section 2.2.4) is implemented using the `sklearn.ensemble.VotingClassifier` class with multiple best models from the hyperparameter tuning as base models. Therefore, 247 different soft-voting configurations (listed in Appendix F.2) were explored. The trained base model models then individually predict the probability of each output class for the input data (during inference). The predicted probabilities are then aggregated by averaging them across all base models. The final prediction is determined based on the class with the highest average probability, allowing the ensemble to use the confidence levels of individual base models for improved decision making. Figure 5.3 shows the visual structure of the soft voting ensemble model.



(a) Training of base models using the training set.



(b) Predicting conversion by aggregating the independent probability predictions of the trained base models.

Figure 5.3: Structure of Soft Voting Ensemble Model.

5.10.4 Bagging

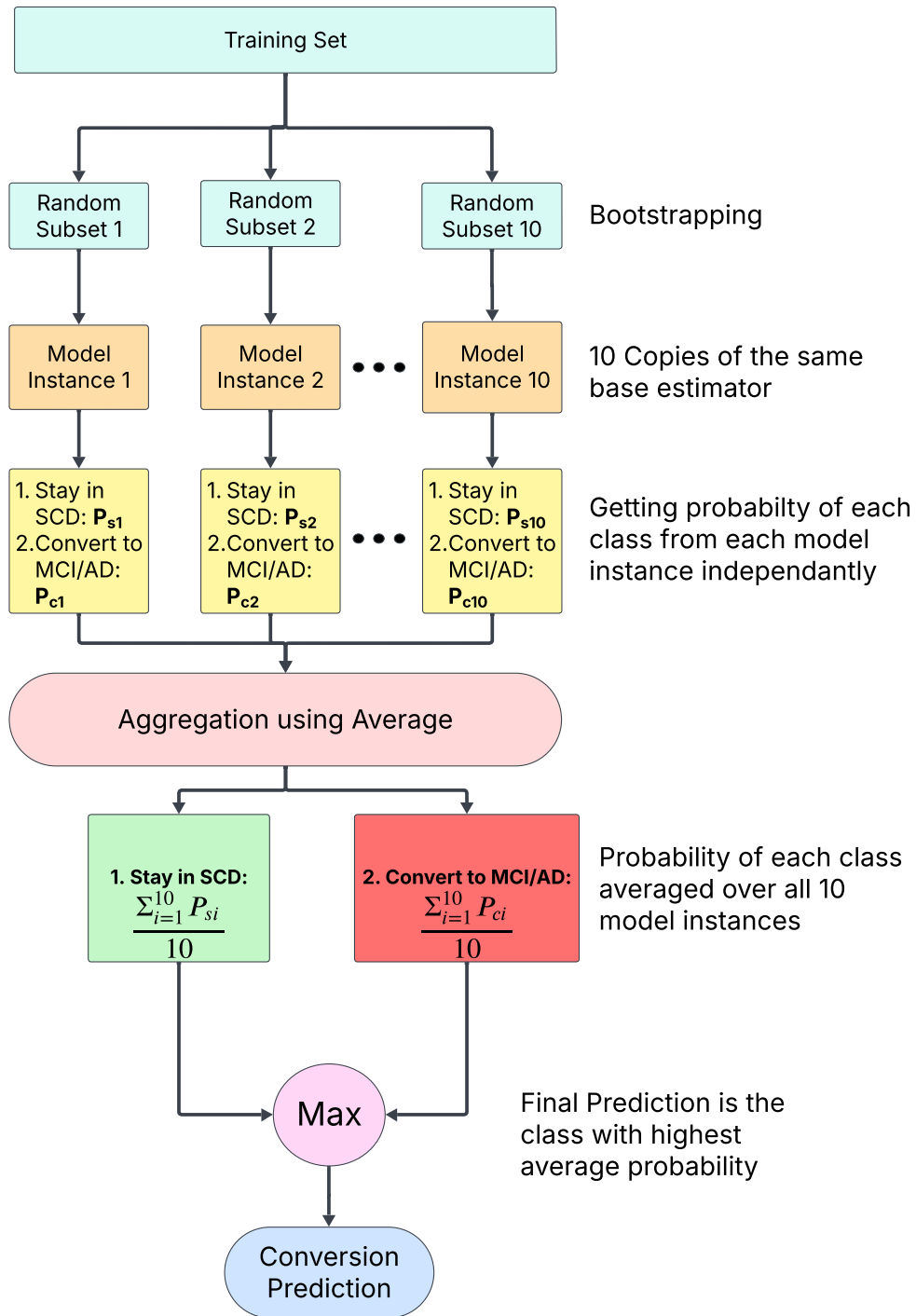


Figure 5.4: Structure of Bagging Ensemble Model

The bagging ensemble technique (discussed in Section 2.2.4) is implemented using the `sklearn.ensemble.BaggingClassifier`, with the base estimator selected from the models optimised during the hyperparameter tuning process. Ten instances of the chosen base estimator, each configured with the same optimal hyperparameters, are used to construct an ensemble, resulting in eight distinct bagging classifier configurations (detailed in Appendix

F.3). Additionally, a soft voting strategy (described in Section 2.2.4) is employed to aggregate the probabilistic output of the base models, thereby enhancing prediction accuracy. The structure of the bagging ensemble model is shown in Figure 5.4.

5.10.5 Selecting Best Ensemble Model

Each combination of ensembling techniques with different base models is evaluated using 5-fold cross-validation of the entire dataset (Figure 6.1). For each iteration, these metrics were calculated and subsequently averaged over the five iterations to obtain reliable and balanced performance estimates. To facilitate model selection, we computed an average score by taking the mean of the average ROC-AUC and the average accuracy. The ensemble model with the highest final average score was then selected as the optimal model for the dataset. The process is visually presented in Figure 5.5. This rigorous evaluation process ensures that the chosen model has robust discriminatory capabilities and generalises well to unseen data, making it suitable for real-world applications. The code for selecting the best ensemble model is presented in Appendix E.

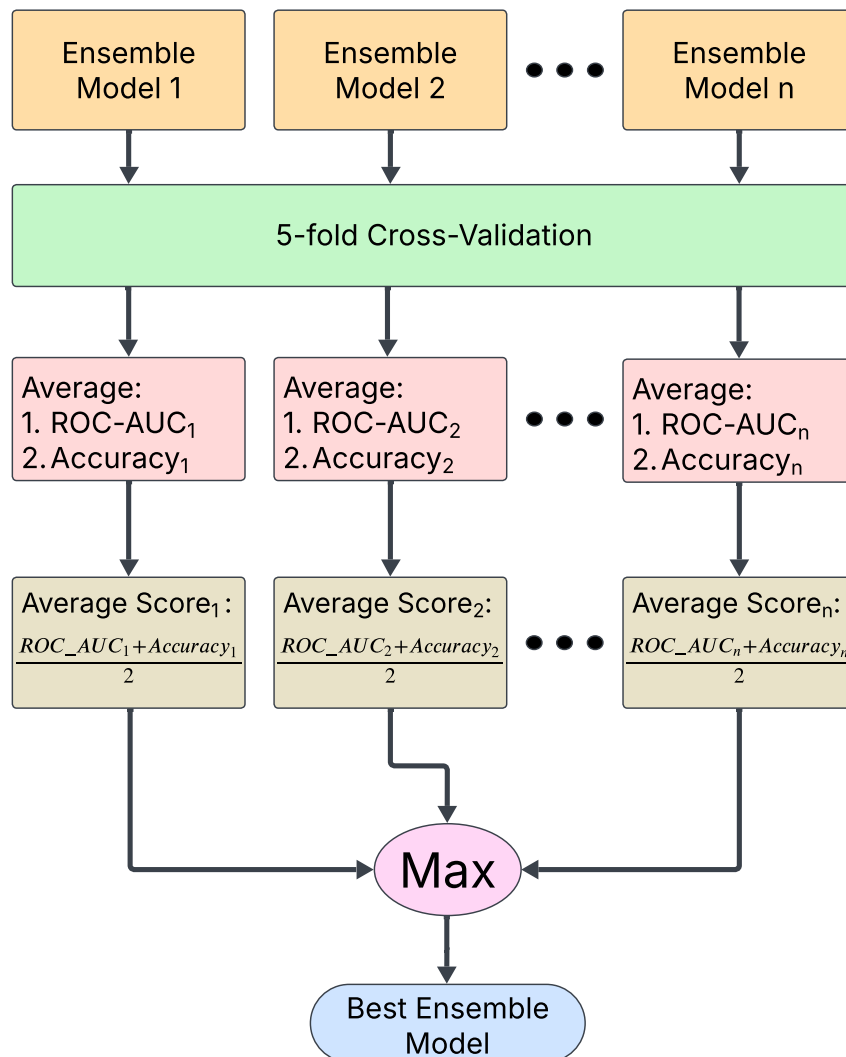


Figure 5.5: Selecting the best ensemble model from all the ensemble model configurations (listed in Appendix F).

Chapter 6

Results

For the final evaluation of models predicting conversion of SCD patients to MCI/AD within 10 years (and in the additional experiment with a 7-year conversion window), the performance of the best hyperparameter-tuned base models and the best ensemble model developed for each dataset was assessed using **5-fold CV**. The **average values** of key evaluation metrics were then computed across the 5 iterations to summarise model performance. The evaluation process is visually presented in Figure 6.1.

The 5-fold CV was selected as the evaluation method due to the limited size of the dataset (225 samples). This approach helps mitigate train-test split bias by ensuring that each data point is used for both training and testing across different folds. As a result, it provides a more robust and reliable estimate of model performance.

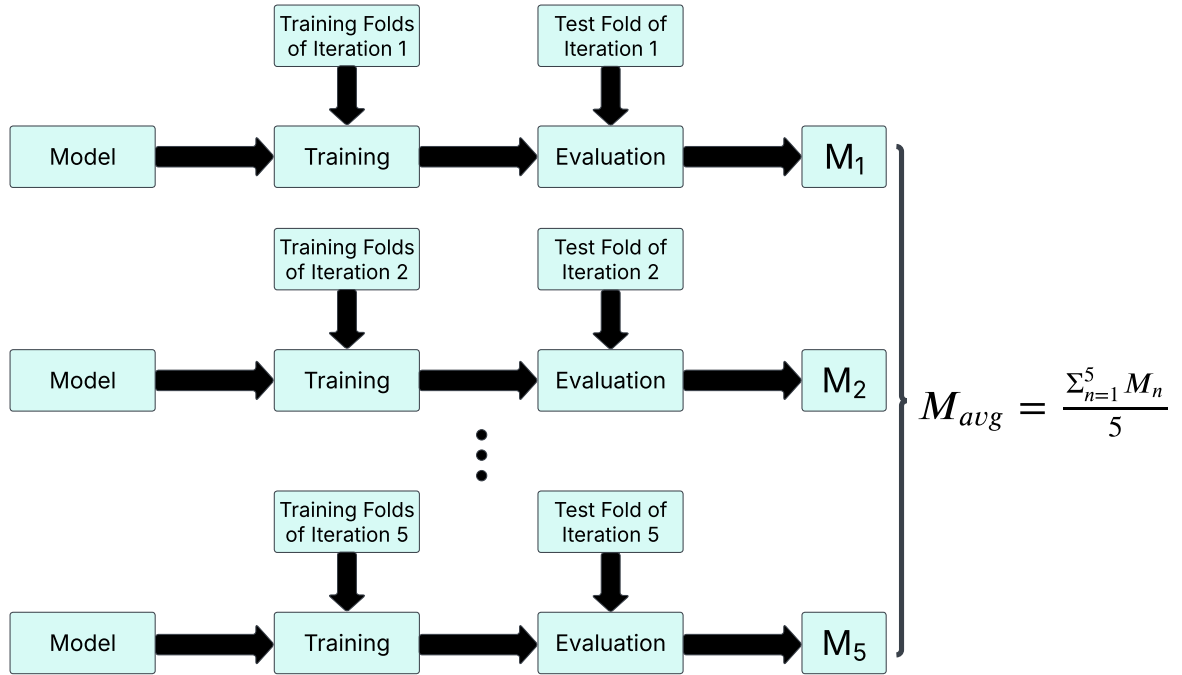


Figure 6.1: Final Evaluation Method for Each Machine Learning Model Developed. M_i denotes evaluation metric being calculated in the i^{th} iteration.

6.1 Imputation Datasets

To identify the best imputation approach, various imputation methods were applied to the 10-year conversion dataset comprising **225 samples** with all **50 initial features**. Each imputed dataset was then divided into five equal folds for evaluation using **5-fold CV**.

In each iteration of the 5 iterations in 5-fold CV, the dataset is split into:

- **Training set (80%)**: 4 folds / 180 samples (110 Convert to MCI/AD : 70 Stay in SCD)
- **Testing set (20%)**: 1 fold / 45 samples (28 Convert to MCI/AD : 17 Stay in SCD)

6.1.1 Best Imputation Method

The results from different imputation methods, averaged over five folds, for the task of predicting conversion from SCD to MCI/AD within 10 years, are presented in Table 6.1. The evaluation metrics indicate that **Random Forest Imputation** achieves the best combination of ROC-AUC (0.851) and Accuracy (0.809). This outcome is expected, as the Random Forest model effectively captures complex relationships among input features, in contrast to simpler imputation methods like Mean+Mode or Median+Mode. In Section 6.2.1, which will focus on identifying the optimal class-balancing method, we will use the dataset imputed with the Random Forest model. It is also worth noting that **Median+Mode Imputation** produced comparable results (ROC-AUC=0.857, Accuracy=0.796) while being a less complex alternative.

Table 6.1: Performance Comparison of Ensemble Models of different Imputation Methods

Imputation Method	ROC-AUC	Accuracy
Random Forest	0.851 ± 0.039	0.808 ± 0.048
MICE	0.850 ± 0.052	0.787 ± 0.093
Mean+Mode	0.845 ± 0.049	0.778 ± 0.088
Median+Mode	0.857 ± 0.055	0.796 ± 0.029

6.1.2 Detailed Performance of Best Imputation Method: Random Forest Imputation

The best ensemble model selected for the 10-year conversion dataset imputed using the RF method is a soft voting classifier that combines Logistic Regression, kNN, and Multi-Layer Perceptron as base models (Figure 6.2). Table 6.2 presents the optimal hyperparameters for each base model within the ensemble model.

The best ensemble models and the optimal hyperparameters of base models within the best ensemble models for the datasets imputed using other methods are presented in Appendix G.1.

Table 6.3 summarises the average performance metrics and corresponding optimal hyperparameters of the tuned base models for the Random Forest-imputed dataset, based on 5-fold CV. Similarly, Table 6.4 presents the comprehensive performance metrics of the selected ensemble model. In addition, Figures 6.3 and 6.4 illustrate the average confusion matrix and average ROC-AUC curve, respectively, obtained from 5-fold CV. The results demonstrate

that the ensemble model consistently outperforms all base models in terms of both ROC-AUC and accuracy, highlighting the effectiveness of ensemble learning in improving predictive performance.

Table 6.2: Optimal hyperparameters of Base Models used within Best Ensemble Model for dataset imputed using Random Forest

Model	Hyperparameters
Logistic Regression	C=0.01, solver=saga, penalty=l2
KNN	metric=euclidean, n_neighbors=25, weights=uniform
MLP	activation=tanh, alpha=0.0001, hidden_layer_sizes=(50,), learning_rate=constant, max_iter=100, solver=adam

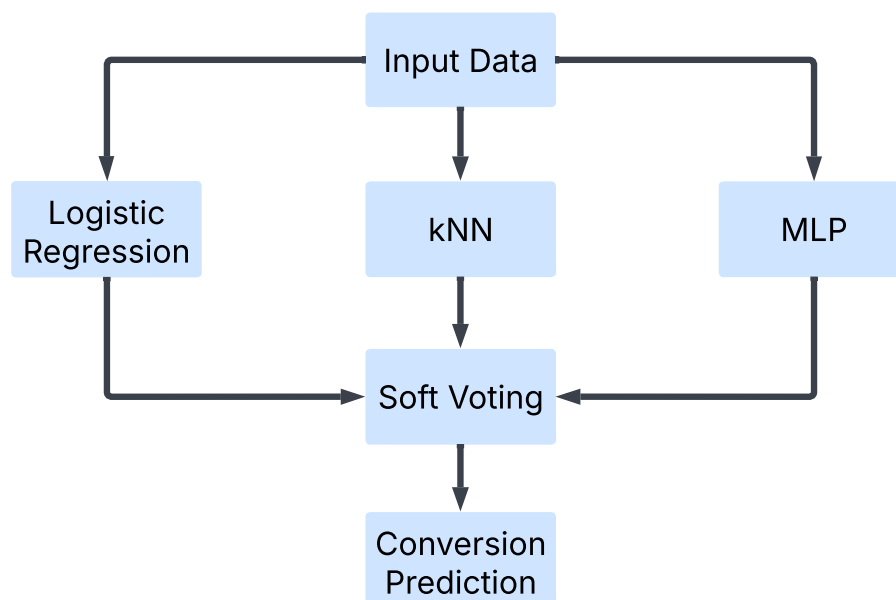


Figure 6.2: Best Ensemble Model for dataset imputed using Random Forest: Soft Voting of LR, kNN and MLP base models.

Table 6.3: Performance Comparison of Hyperparameter-tuned Base Models for dataset imputed using Random Forest

Model	ROC-AUC	Accuracy	Hyperparameters
Logistic Regression	0.848 ± 0.044	0.760 ± 0.026	C=0.01, solver=saga, penalty=l2
SVC	0.848 ± 0.041	0.613 ± 0.011	C=0.01, kernel=sigmoid, gamma=0.1

Continued on next page

Table 6.3 (continued)

Model	ROC-AUC	Accuracy	Hyperparameters
KNN	0.805 ± 0.055	0.729 ± 0.038	metric=euclidean, n_neighbors=25, weights=uniform
MLP	0.828 ± 0.045	0.729 ± 0.047	activation=tanh, alpha=0.0001, hidden_layer_sizes=(50,), learning_rate=constant, max_iter=100, solver=adam
Decision Tree	0.679 ± 0.103	0.676 ± 0.081	criterion=entropy, max_depth=10, max_features=sqrt, splitter=random
Gaussian Naive Bayes	0.798 ± 0.068	0.676 ± 0.052	var_smoothing=1e-09
Random Forest	0.814 ± 0.029	0.729 ± 0.026	max_depth=10, max_features=log2, n_estimators=200
XGBoost	0.760 ± 0.025	0.693 ± 0.038	gamma=0.1, learning_rate=0.3, max_depth=9, n_estimators=50

Metric	Average	Standard Deviation
ROC-AUC	0.851	0.039
Accuracy	0.809	0.048
Class-wise Accuracy		
Stay in SCD	0.746	0.097
Convert to MCI/AD	0.848	0.058
Precision		
Micro	0.809	0.048
Macro	0.802	0.051
Weighted	0.812	0.048
Recall		
Micro	0.809	0.048
Macro	0.797	0.052
Weighted	0.809	0.048
F1-Score		
Micro	0.809	0.048
Macro	0.797	0.052
Weighted	0.808	0.048

Table 6.4: Performance Metrics of Best Ensemble Model of dataset imputed using Random Forest

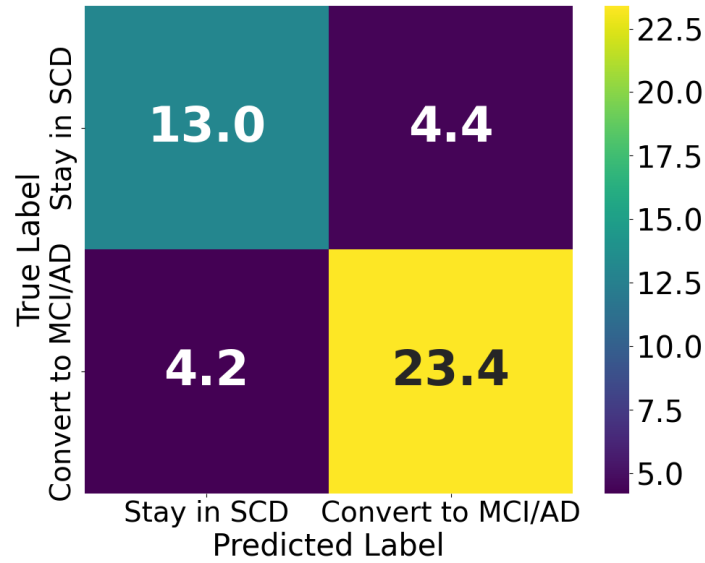


Figure 6.3: Average Confusion Matrix of Best Ensemble Model of dataset imputed using RF from 5-fold CV

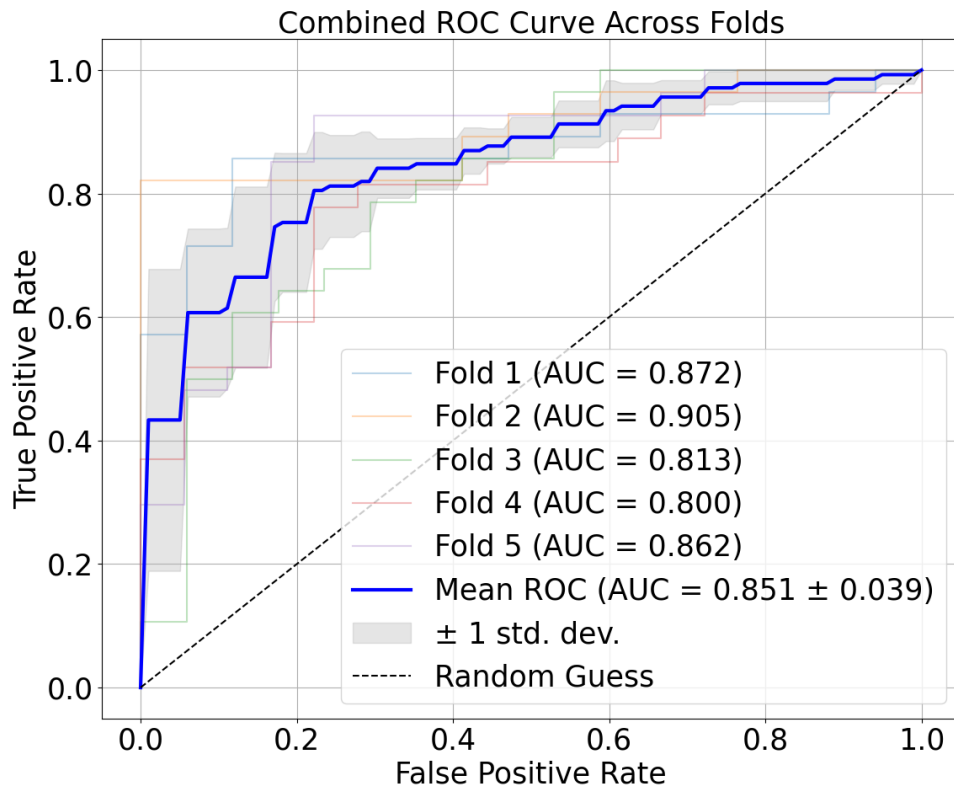


Figure 6.4: Average ROC curve of Best Ensemble Model for dataset imputed using RF from 5-fold CV. (Note: std. dev. refers to standard deviation)

6.2 Class-Balanced Datasets

To identify the best class-balancing approach, various resampling methods were applied to the 10-year conversion dataset imputed using RF imputation (the best-performing imputation

identified in Section 6.1.1). Each dataset produced using different resampling techniques was then divided into five equal folds for evaluation using **5-fold CV**.

6.2.1 Best Class-Balancing Method

The results obtained from different class-balancing methods, averaged over the five folds, for the task of predicting conversion from SCD to MCI/AD within 10 years, are presented in Table 6.5. Among the methods evaluated, SMOTE achieved the best combination of ROC-AUC (0.848) and accuracy (0.804). Although this performance is slightly lower than the base model without class-balancing (Model in Section 6.1.2), which achieved a ROC-AUC of 0.851 and accuracy of 0.808, SMOTE was selected due to its ability to maintain a balanced predictive performance across both classes. This balanced performance reduces bias toward the majority class, resulting in a more robust model for predicting conversion to MCI/AD.

For subsequent analyses in Section 6.3 and Section 6.4, which investigate the predictive utility of different feature groups and individual features, the dataset imputed using the RF technique and class-balanced using SMOTE will be used.

Table 6.5: Performance Comparison of Ensemble Models of Different Class-Balancing Methods

Class Balancing	ROC-AUC	Accuracy	Class-wise Accuracy	
			Stay in SCD	Convert to MCI/AD
None	0.851 ± 0.039	0.808 ± 0.048	0.746 ± 0.097	0.848 ± 0.058
ROS	0.835 ± 0.042	0.787 ± 0.036	0.701 ± 0.067	0.841 ± 0.043
ADASYN	0.852 ± 0.040	0.769 ± 0.075	0.827 ± 0.082	0.731 ± 0.117
SMOTE	0.848 ± 0.033	0.804 ± 0.053	0.793 ± 0.060	0.812 ± 0.072
BL-SMOTE	0.849 ± 0.044	0.773 ± 0.049	0.828 ± 0.033	0.740 ± 0.078
RUS	0.847 ± 0.050	0.778 ± 0.054	0.805 ± 0.050	0.761 ± 0.092
CCUS	0.837 ± 0.053	0.773 ± 0.057	0.690 ± 0.060	0.826 ± 0.062

6.2.2 Detailed Performance of Best Class-balancing Method: SMOTE

In each iteration of the 5 iterations in 5-fold CV, the SMOTE-resampled dataset is split into:

- **Training set:** 4 folds + synthetic samples / 220 samples (110 Convert to MCI/AD : 110 Stay in SCD (70 original + 40 synthetic))
- **Testing set:** 1 fold / 45 samples (28 Convert to MCI/AD : 17 Stay in SCD)

The best ensemble model selected for the 10-year conversion dataset imputed using RF and resampled using the SMOTE algorithm is a stacking classifier that combines LR and MLP as base models, with Random Forest as the final estimator (Figure 6.5). Table 6.6 presents the optimal hyperparameters for each base model within the ensemble model.

The best ensemble models and the optimal hyperparameters of base models within the best ensemble models for the datasets class-balanced using other methods are presented in Appendix G.2.

Table 6.7 summarises the average performance metrics and optimal hyperparameters of the base models trained on the SMOTE-balanced dataset, evaluated using 5-fold CV. Similarly, Table 6.8 presents the detailed performance metrics of the best ensemble model. In addition, Figures 6.6 and 6.7 illustrate the average confusion matrix and average ROC-AUC curve, respectively, obtained from 5-fold CV. The results demonstrate that the ensemble model consistently outperforms all individual base models in terms of both ROC-AUC and accuracy, highlighting the effectiveness of combining SMOTE with ensemble learning to enhance classification performance.

Table 6.6: Optimal hyperparameters of Base Models used in Best Ensemble Model for dataset class-balanced using SMOTE

Model	Hyperparameters
Logistic Regression	C=0.01, solver=saga, penalty=l2
MLP	activation=tanh, alpha=0.0001, hidden_layer_sizes=(50,), learning_rate=constant, max_iter=100, solver=adam
Random Forest	max_depth=10, max_features=log2, n_estimators=200

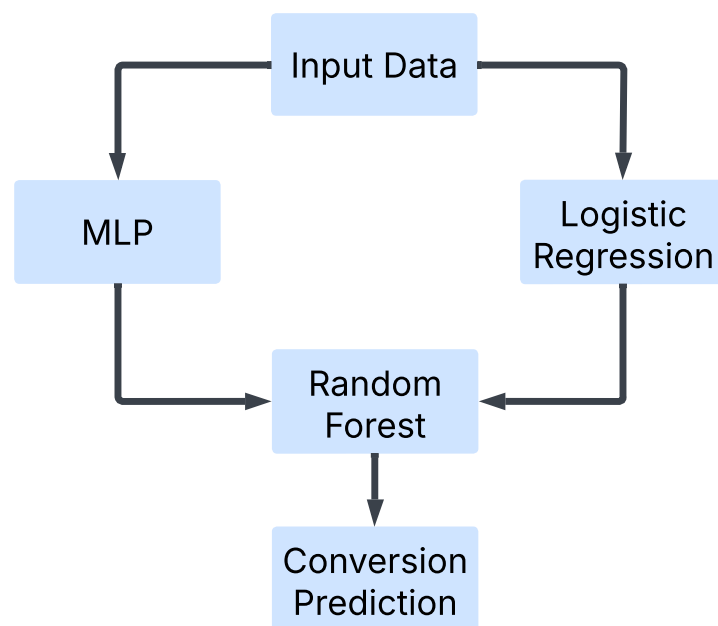


Figure 6.5: Best Ensemble Model for dataset imputed using Random Forest and class-balanced using SMOTE: Stacking of MLP and LR with RF as the final estimator.

Table 6.7: Performance Comparison of Hyperparameter-tuned Base Models for dataset class-balanced using SMOTE

Model	ROC-AUC	Accuracy	Hyperparameters
Logistic Regression	0.846 ± 0.046	0.782 ± 0.053	C=0.01, solver=saga, penalty=l2
SVC	0.758 ± 0.053	0.684 ± 0.055	C=10, kernel=rbf, gamma=0.1
KNN	0.782 ± 0.049	0.613 ± 0.023	metric=manhattan, n_neighbors=21, weights=distance
MLP	0.820 ± 0.063	0.716 ± 0.047	activation=tanh, alpha=0.0001, hidden_layer_sizes=(50,), learning_rate=constant, max_iter=100, solver=adam
Decision Tree	0.665 ± 0.028	0.640 ± 0.033	criterion=entropy, max_depth=10, max_features=sqrt, splitter=random
Gaussian Naive Bayes	0.793 ± 0.067	0.689 ± 0.058	var_smoothing=0.001
Random Forest	0.806 ± 0.021	0.738 ± 0.029	max_depth=10, max_features=log2, n_estimators=200
XGBoost	0.748 ± 0.024	0.680 ± 0.050	gamma=0.2, learning_rate=0.3, max_depth=9, n_estimators=50

Table 6.8: Average Performance Metrics of Best Ensemble Model for dataset class-balanced using SMOTE

Metric	Average	Standard Deviation
ROC-AUC	0.848	0.033
Accuracy	0.804	0.053
Class-wise Accuracy		
Stay in SCD	0.793	0.060
Convert to MCI/AD	0.812	0.072
Precision		
Micro	0.804	0.053
Macro	0.797	0.052
Weighted	0.812	0.046
Recall		
Micro	0.804	0.053
Macro	0.802	0.051
Weighted	0.804	0.053
<i>Continued on next page</i>		

Table 6.8 (continued)

Metric	Average	Standard Deviation
F1-Score		
Micro	0.804	0.053
Macro	0.797	0.054
Weighted	0.806	0.052

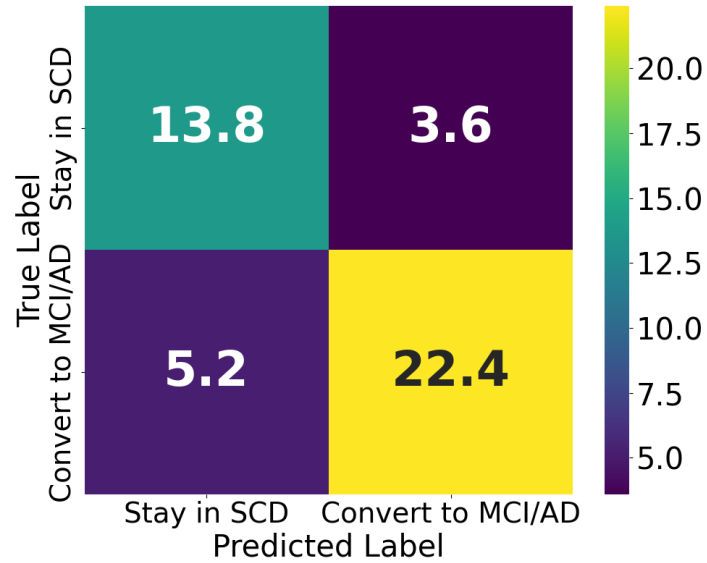


Figure 6.6: Average Confusion Matrix of Best Ensemble Model for dataset class-balanced using SMOTE from 5-fold CV

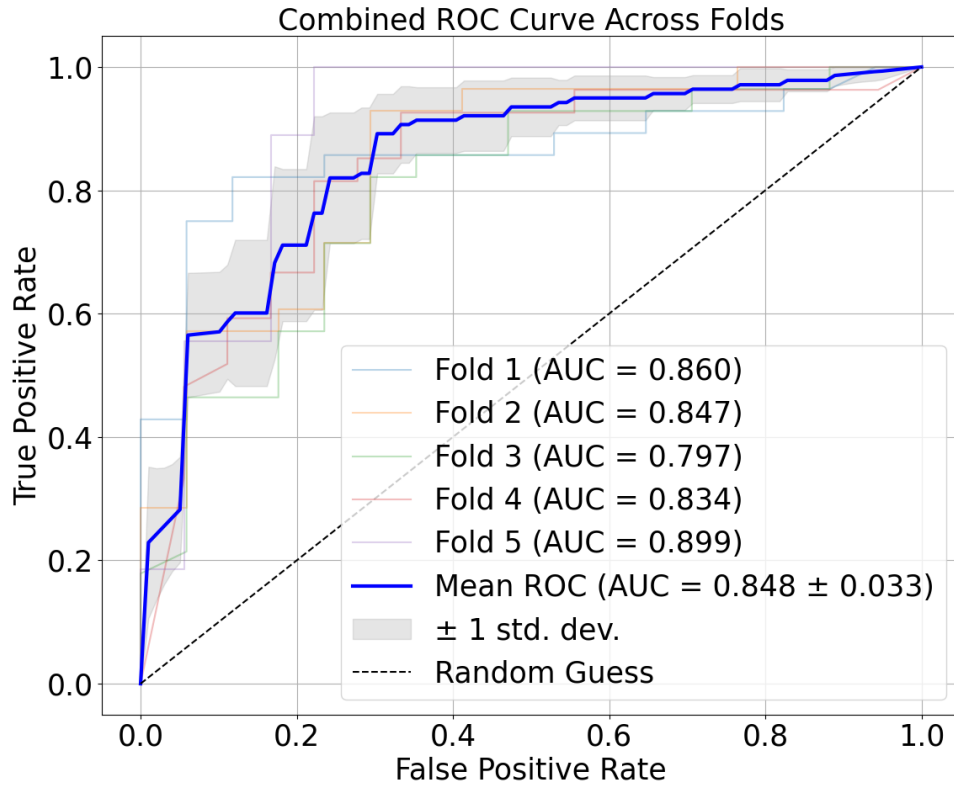


Figure 6.7: Average ROC curve of Best Ensemble Model for dataset class-balanced using SMOTE from 5-fold CV. (Note: std. dev. refers to standard deviation)

6.3 Feature Selection

To further optimise the model, manual and automatic feature selection techniques are applied. This process helps reduce model complexity, improve interpretability, and mitigate the risk of overfitting, while also enhancing computational efficiency.

6.3.1 Manually Selected Feature Groups

The 10-year conversion dataset, imputed using the RF method and class-balanced with the SMOTE algorithm, is further processed by selecting different feature groups. The model construction process (in Chapter 5) is repeated for each manually curated feature group, and the best-performing ensemble model for each group is identified for the task of predicting conversion from SCD to MCI/AD within 10 years. A comparative analysis of the performance of these ensemble models is then conducted to evaluate the impact of different feature sets on predictive capacity and generalisability.

The best ensemble models and the optimal hyperparameters of individual base models within the best ensemble models for manually selected feature groups are presented in Appendix G.3.

Performance Comparison of Manually Selected Feature Groups

In manual feature group selection, the results observed from the different ensemble models of the feature groups are shown in Table 6.9. Group A, which includes all features, is the best-performing group, with an average ROC-AUC of 0.848 and an accuracy of 0.804. This

result suggests that the full feature set provides complementary information, with minimal redundancy. Conversely, Group B with only base features is the least performing group, which was expected due to the absence of features closely related to MCI/AD conversion, such as CSF, MRI and PET data.

Among the groups C to G with features in addition to Group B, the best performing group was Group D with PET features (ROC-AUC: 0.823, Accuracy: 0.791). However, Group D was highly biased towards predicting conversion to MCI/AD rather than staying in SCD than progression to MCI/AD. Group E incorporating MRI features had a drop in performance compared to Group A. Regardless, it achieved balanced performance across both classes, unlike the class-biased performance of most groups. Group G, incorporating APOE4 data, was better at predicting stability in SCD, suggesting that APOE4 may serve as a strong indicator of stability rather than disease progression. An interesting observation is that Group F, with Plasma data, has a better accuracy and a more balanced class-wise predictive performance compared to Group C, with CSF data, which requires a more invasive collection procedure.

Group H with the removal of MRI and PET Data from all features, had good average ROC-AUC and accuracy but showed biased behaviour towards predicting conversion to MCI/AD. Group I containing only non-collinear features achieved a better average ROC-AUC (0.850) than group A but had lower accuracy and class-biased performance towards conversion, showing that the removal of collinear features lowers predictive performance.

Table 6.9: Performance Comparison of Ensemble Models of Different Feature Groups

Feature Group	ROC-AUC	Accuracy	Class-wise Accuracy	
			Stay in SCD	Convert to MCI/AD
A - All	0.848 ± 0.033	0.804 ± 0.053	0.793 ± 0.060	0.812 ± 0.072
B - Base	0.794 ± 0.040	0.729 ± 0.065	0.692 ± 0.110	0.755 ± 0.103
C - Base + CSF	0.814 ± 0.035	0.738 ± 0.071	0.839 ± 0.067	0.675 ± 0.090
D - Base + PET	0.823 ± 0.033	0.791 ± 0.048	0.727 ± 0.110	0.833 ± 0.053
E - Base + MRI	0.805 ± 0.046	0.747 ± 0.059	0.737 ± 0.075	0.754 ± 0.077
F - Base + Plasma	0.801 ± 0.059	0.747 ± 0.064	0.725 ± 0.088	0.762 ± 0.093
G - Base + APOE4	0.795 ± 0.048	0.742 ± 0.054	0.793 ± 0.075	0.711 ± 0.079
H - All Except MRI & PET	0.830 ± 0.061	0.773 ± 0.047	0.633 ± 0.039	0.862 ± 0.069
I - Non-collinear	0.850 ± 0.032	0.778 ± 0.044	0.692 ± 0.099	0.834 ± 0.077

6.3.2 Automatic Feature Selection

The top 20 features are selected using RFE for the task of predicting conversion from SCD to MCI/AD within 10 years using two models: Random Forest and Logistic Regression. The RFE process is repeated for all 5 SMOTE training datasets from Section 6.2.2 with each model (10 selection runs). The selected features across these iterations are shown in Table 6.10.

Across the 10 iterations, the following features appeared in 8 or more iterations: FDG, AV45, ABETA, PLASMA_ABETA42/40, ADAS13, DIGITSCOR, MoCA, EcogPtPlan, EcogPtTotal, mPACCtrailsB and mPACCdigit. This shows that PET, ABETA levels in plasma and CSF, and neuropsychological and cognitive test scores are the most important features. This result

also correlates with the high performance of manual feature group D with PET data (FDG and AV45).

However, 3 out of 6 MRI features that were selected were selected only once and the other selected MRI features were chosen at most in 6 times out of the 10 selection runs (Table 6.10). APOE4 feature is also not selected in any of the iterations. This shows that MRI and APOE4 features are not the most significant in predicting conversion from SCD to MCI/AD. Demographic features, except PTEDUCAT, are not selected in any of the iterations, which indicates they have less importance.

Table 6.10: Top 20 Features selected using RFE with RF and LR over the 5 SMOTE Training Folds

Feature Name	Random Forest					Logistic Regression				
Fold	1	2	3	4	5	1	2	3	4	5
AGE										
PTGENDER										
PTEDUCAT						✓	✓	✓	✓	
PTMARRY										
APOE4										
FDG	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
AV45	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
ABETA	✓		✓	✓	✓	✓	✓	✓	✓	✓
TAU										
PTAU				✓	✓	✓			✓	✓
PLASMA_NFL										
PLASMA_PTAU	✓			✓	✓	✓				✓
PLASMA_ABETA42/40	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
CDRSB						✓		✓	✓	✓
ADAS11	✓			✓			✓		✓	✓
ADAS13	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
ADASQ4	✓		✓			✓		✓		
MMSE										
RAVLT_immediate		✓		✓			✓	✓	✓	✓
RAVLT_learning			✓			✓		✓		
RAVLT_forgetting										
RAVLT_perc_forgetting										
LDELTOTAL					✓				✓	✓
DIGITSCOR	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
TRABSCOR		✓	✓	✓	✓		✓	✓	✓	
FAQ										

Continued on next page

Table 6.10 (continued)

Feature Name Fold	Random Forest					Logistic Regression				
	1	2	3	4	5	1	2	3	4	5
MoCA	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
EcogPtMem										
EcogPtLang	✓	✓				✓	✓		✓	✓
EcogPtVisspat										
EcogPtPlan	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
EcogPtOrgan										
EcogPtDivatt	✓	✓		✓		✓	✓		✓	✓
EcogPtTotal	✓	✓	✓	✓	✓	✓	✓	✓		
EcogSPMem			✓			✓		✓		✓
EcogSPLang		✓		✓	✓			✓		
EcogSPVisspat							✓			
EcogSPPlan										
EcogSPOrgan										
EcogSPDivatt			✓		✓					
EcogSPTotal	✓	✓	✓	✓	✓					
Ventricles	✓	✓	✓		✓		✓		✓	
Hippocampus			✓	✓				✓	✓	
WholeBrain										
Entorhinal	✓	✓			✓	✓	✓			✓
Fusiform		✓								
MidTemp	✓									
ICV		✓								
mPACCdigit	✓	✓	✓	✓	✓	✓	✓	✓		✓
mPACCtrailsB	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓

6.4 Importance of Individual Features

The best-performing ensemble model using all features (in Section 6.2.2) for predicting conversion from SCD to MCI/AD within 10 years was trained and evaluated using each feature individually to understand the importance of each feature. The average ROC-AUC and accuracy over 5 folds were calculated as measures for the independent performance of individual predictors. The results are shown in Table 6.11.

The Top 5 individual predictors that performed the best were ADAS13 (ROC-AUC=0.669, Accuracy=0.627), FDG (ROC-AUC=0.685, Accuracy=0.604), MoCA (ROC-AUC=0.662, Accuracy=0.596), mPACCtrailsB (ROC-AUC=0.662, Accuracy=0.596), and RAVLT_immediate (ROC-AUC=0.635, Accuracy=0.573) in their respective order. 4 out of the top 5 individual predictors are neuropsychological and cognitive test scores, highlighting that these relatively inexpensive assessments are among the strongest indicators of conversion from

SCD to MCI/AD. Additionally, the CSF PTAU (ROC-AUC=0.537, Accuracy=0.52) and ABETA (ROC-AUC=0.565, Accuracy=0.551) predictors did not show significantly better performance than the Plasma PTAU (ROC-AUC=0.524, Accuracy=0.502) and ABETA42/40 (ROC-AUC=0.531, Accuracy=0.542) predictors. This finding advocates for the need for a more careful consideration of trade-offs between invasive collection of CSF data and slightly lower performance of plasma-based biomarkers.

Most importantly, the absence of any single feature that achieves performance close to the ensemble model with all features suggests that individual features alone are insufficient to accurately predict conversion. This indicates that the cumulative contribution of all features is essential to achieve the best performance of the ensemble model. Furthermore, it confirms that all features are non-redundant, as each contributes at least a slight improvement in performance.

Table 6.11: Average Performance of SMOTE Ensemble Model with Individual Features

Feature	ROC-AUC	Accuracy
AGE	0.436	0.489
PTGENDER	0.512	0.516
PTEDUCAT	0.482	0.489
PTMARRY	0.473	0.48
APOE4	0.494	0.471
FDG	0.685	0.604
AV45	0.565	0.551
ABETA	0.569	0.551
TAU	0.513	0.502
PTAU	0.537	0.52
<i>PLASMA_NFL</i>	0.59	0.573
<i>PLASMA_PTAU</i>	0.524	0.502
<i>PLASMA_ABETA42/40</i>	0.531	0.542
CDRSB	0.549	0.511
ADAS11	0.622	0.569
ADAS13	0.669	0.627
ADASQ4	0.578	0.511
MMSE	0.538	0.573
RAVLT_immediate	0.635	0.573
<i>RAVLT_learning</i>	0.505	0.52
<i>RAVLT_forgetting</i>	0.555	0.524
<i>RAVLT_perc_forgetting</i>	0.589	0.569
LDELTOTAL	0.605	0.547
DIGITSCOR	0.575	0.569
TRABSCOR	0.595	0.569

Continued on next page

Table 6.11 (continued)

Feature	ROC-AUC	Accuracy
FAQ	0.568	0.48
MoCA	0.662	0.596
EcogPtMem	0.532	0.551
EcogPtLang	0.471	0.467
EcogPtVispat	0.494	0.462
EcogPtPlan	0.53	0.516
EcogPtOrgan	0.495	0.516
EcogPtDivatt	0.496	0.507
EcogPtTotal	0.628	0.564
EcogSPMem	0.554	0.52
EcogSPLang	0.419	0.511
EcogSPVispat	0.479	0.498
EcogSPPlan	0.567	0.529
EcogSPOrgan	0.521	0.52
EcogSPDivatt	0.551	0.542
EcogSPTotal	0.542	0.533
Ventricles	0.554	0.569
Hippocampus	0.52	0.498
WholeBrain	0.539	0.524
Entorhinal	0.494	0.467
Fusiform	0.45	0.462
MidTemp	0.491	0.547
ICV	0.501	0.507
mPACCdigit	0.607	0.564
mPACCtrailsB	0.628	0.596

6.5 Conversion within 7 years

As an extension, the preprocessing pipeline described in Chapter 4 was repeated to construct a dataset for predicting conversion from SCD to MCI/AD within 7 years. The resulting dataset comprised 315 patients, of which 202 remained stable in the SCD for more than 7 years, while 113 converted to MCI or AD within the 7-year period. Based on the findings from the 10-year conversion prediction task, the same imputation and class-balancing techniques (RF imputation and the SMOTE-balancing) were retained due to their demonstrated effectiveness. Initially, the best-performing ensemble model developed for the 10-year prediction task (in Section 6.2.2) was applied to the 7-year dataset with the full set of 50 features. However, this model achieved only a mean accuracy of 0.695 (± 0.035) and a mean ROC-AUC of 0.747 (± 0.031) in 5-fold cross-validation. Consequently, the full model development process described in Chapter 5 was repeated for the 7-year task. This resulted in a new ensemble model specifically optimised

for 7-year conversion prediction, as shown in Figure 6.8, which achieved an improved mean accuracy of 0.771 (± 0.028) and mean ROC-AUC of 0.818 (± 0.043) in 5-fold cross-validation. The hyperparameters of the individual base models included in this ensemble are summarised in Table 6.12.

Table 6.12: Optimal hyperparameters of Base Models used in SMOTE Ensemble Model

Model	Hyperparameters
Logistic Regression	C=0.01, solver=saga, penalty=l2
MLP	activation=tanh, hidden_layer_sizes=(100,50), max_iter=100, solver=adam, alpha=0.0001, learning_rate=constant,
Random Forest	max_depth=None, max_features=sqrt, n_estimators=200

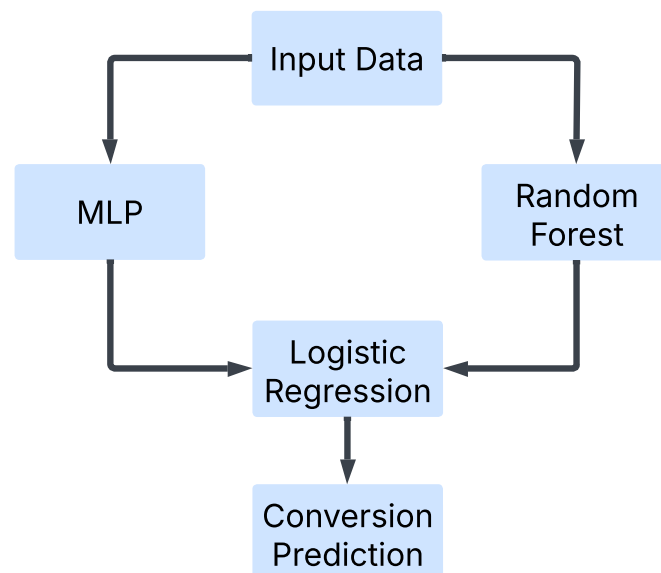


Figure 6.8: Best Ensemble Model for dataset for predicting conversion within 7 years: Stacking of MLP and RF with LR as the final estimator.

Chapter 7

Discussion

7.1 Discussion of Results

The best-performing models for predicting the conversion of SCD patients to MCI/AD within 10 years, as well as for the additional experiment with a 7-year window, were developed using the complete set of 50 features (in Section 6.2.2 and 6.5 respectively), with missing values imputed using the Random Forest technique and class imbalance addressed through the SMOTE technique. This result highlights the complexity of the relationships among features, warranting using a sophisticated, non-linear imputation method such as Random Forest. Furthermore, the superior performance of SMOTE over alternative resampling techniques suggests that undersampling discards valuable information, which is particularly detrimental in small datasets. Compared to Random Oversampling and ADASYN, SMOTE provided more effective class balancing, likely due to its structured approach to generating synthetic samples. Additionally, SMOTE outperformed Borderline-SMOTE, indicating that restricting synthetic sample generation to borderline samples may lead to overgeneralisation and reduced model performance.

Comparison with Past Literature Studies

The best-performing model developed in this project for predicting the conversion of SCD patients to MCI/AD within 10 years (Section 6.2.2) achieved an average accuracy of 0.804 (± 0.053), sensitivity of 0.793 (± 0.060), specificity of 0.812 (± 0.072), and an average ROC-AUC of 0.848 (± 0.033) based on 5-fold cross-validation. In comparison, the a-MCI-based model presented in Dolcet-Negre et al. 2023, which predicted conversion to MCI/AD within 8 years, reported a ROC-AUC of only 0.72, indicating that the model developed in this project exhibits superior discriminatory performance.

Additionally, the model developed in Lerch et al. 2024 for predicting conversion within 16 years achieved a median sensitivity of 0.83, median specificity of 0.77, and a median ROC-AUC of 0.75 across 50 random train-test splits. To provide a comparable evaluation, the best-performing 10-year model from this project was also assessed using 50 random train-test splits. It achieved a median sensitivity of 0.785, specificity of 0.718, and ROC-AUC of 0.83. These results show improved overall discriminatory performance compared to the model in Lerch et al. 2024. However, the latter exhibited slightly better class-wise accuracies, which could be due to the broader 16-year prediction window allowing more conversions to occur.

Similarly, Yue et al. 2020 presents a model for predicting conversion of SCD patients to MCI (only) within 7 years, which achieved an average accuracy of 0.6974, sensitivity of 0.6250, and specificity of 0.7308. In contrast, the model developed in this project for the same 7-year prediction task for conversion from SCD to MCI/AD (Section 6.5) achieved an average accuracy of 0.771 (± 0.028), sensitivity of 0.672 (± 0.024), specificity of 0.827 (± 0.043), and an average ROC-AUC of 0.818 (± 0.043). These results demonstrate a clear improvement in classification accuracy.

In conclusion, the ensemble models developed in this project for predicting conversion from SCD to MCI/AD within 7 and 10 years outperform existing models in research across most evaluation metrics, supporting the robustness and reliability of the proposed modelling approach.

7.2 Discussion of Methods

Using Python libraries such as Scikit-learn, Imbalanced-learn, and Matplotlib significantly enhanced the efficiency and robustness of the development process. Creating a separate Python file to serve as a custom library for frequently used functions also facilitated efficient code reuse. It streamlined the application of preprocessing steps and model construction across different experimental setups. Initially, development and experimentation were conducted using Jupyter notebooks on Google Colab. However, due to time restrictions on GPU usage in the free version of Colab, the work was migrated to a local machine equipped with an AMD Ryzen 7 4800H processor and an NVIDIA GeForce RTX 3050 GPU. The availability of local GPU resources significantly reduced the machine learning models' training and inference time.

7.3 Evaluation of Objectives

(a) Handling Missing Data

The project successfully addressed the issue of missing data by employing Random Forest imputation to estimate and replace missing values. In addition to Random Forest imputation, multiple imputation techniques were explored and evaluated (in Section 4.5 and 6.1). Random Forest imputation was selected as the preferred method due to the superior performance of models trained on datasets imputed using this technique.

(b) Handling Class Imbalance

The project successfully addressed the class imbalance in the dataset by applying the SMOTE method to generate synthetic samples for the minority class, "Stay in SCD", to equalise the number of samples of both classes in training data (110 samples each). In addition to SMOTE, several other resampling techniques, both oversampling and undersampling, were explored and evaluated (in Section 4.6 and 6.2). SMOTE was selected as the preferred class-balancing method due to its ability to increase the representation of the minority class without discarding valuable data, as well as the superior performance of models trained on SMOTE-resampled datasets.

(c) Developing Ensemble Model

The project successfully developed a stacking ensemble model to predict the conversion

of SCD patients to MCI/AD within 10 years. This model outperformed those proposed in Lerch et al. 2024, Dolcet-Negre et al. 2023, and Yue et al. 2020 across most evaluation metrics, as discussed in Section 7.1. In addition, a wide range of ensemble configurations was explored, including 494 distinct stacking ensembles, 247 voting ensemble configurations, and eight bagging ensemble setups. The stacking classifier (presented in Section 6.2.2) was ultimately selected due to its superior classification performance in predicting conversion.

(d) Feature Selection

The project successfully implemented manual and automated feature selection methods to assess the importance of individual features and feature groups in predicting the conversion of SCD patients to MCI/AD (Section 4.7 and 6.3). A particular focus was placed on evaluating the impact of removing high-cost data sources, such as MRI and PET scans, to determine whether acceptable classification performance could be achieved without them. However, the feature selection process revealed that all 50 features contributed meaningfully to model performance, as the model incorporating all features achieved the highest average classification performance.

(e) Extend to conversion within 7 Years

The task of predicting the conversion of SCD patients to MCI/AD within 10 years was further extended to a 7-year prediction period (Section 6.5). For this extended task, the project successfully employed Random Forest imputation to address missing data and applied SMOTE resampling to mitigate class imbalance. Additionally, various ensemble configurations were explored to optimise model performance. As discussed in Section 7.1, the final model successfully demonstrated superior performance compared to the model presented in Yue et al. 2020, which was also developed to predict conversion within a 7-year time frame.

7.4 Personal Reflection

Regarding time management, the decision to create a Gantt chart and stick to the project plan as closely as possible proved highly beneficial in keeping the project on track. Additionally, implementing reusable code in the form of a custom Python library enabled efficient and consistent repetition of preprocessing and model construction across various imputation methods, class-balancing techniques, and feature groups. However, an area of improvement in this project would be version control. Throughout the project, version control was managed manually by regularly uploading to cloud storage as a backup mechanism. Employing a more structured and efficient system, such as Git, would have provided better version tracking and project organisation.

Chapter 8

Conclusion and Future Work

8.1 Conclusions

This project aimed to develop ensemble models for predicting the conversion of SCD patients to MCI/AD over 10-year and 7-year periods. The primary challenges encountered were missing data and class imbalance within the dataset. Multiple imputation techniques were explored to address the issue of missing data, including mean, median, mode, Multiple Imputation by Chained Equations (MICE), and RF imputation. Among these, RF Imputation was selected as the most effective method based on model performance.

The class imbalance was tackled by evaluating various resampling techniques, including Random Oversampling, Random Undersampling, ADASYN, Clustered Centroids Undersampling, SMOTE, and Borderline SMOTE. SMOTE emerged as the best-performing technique due to its ability to balance the dataset without discarding valuable information and its consistent contribution to improved model performance.

An extensive set of ensemble configurations was evaluated to identify the optimal predictive model, comprising 494 distinct stacking ensemble configurations, 247 voting ensemble configurations, and eight bagging ensemble configurations. These ensembles were constructed using combinations of eight base models: Logistic Regression, Support Vector Classifier (SVC), k-Nearest Neighbours (kNN), Multi-layer Perceptron (MLP), Decision Tree, Gaussian Naive Bayes, Random Forest, and Extreme Gradient Boosting (XGBoost).

The best-performing model for the 10-year conversion prediction was a stacking classifier comprising Logistic Regression and MLP as base learners, with a Random Forest model as the final estimator. This model achieved an average accuracy of 0.804 (± 0.053) and an average ROC-AUC of 0.848 (± 0.033) based on 5-fold cross-validation.

For the 7-year conversion period, the best model was a stacking ensemble with MLP and Random Forest as base models and Logistic Regression as the final estimator. It achieved an average accuracy of 0.771 (± 0.028) and an average ROC-AUC of 0.818 (± 0.043) under 5-fold cross-validation.

Additionally, the feature selection process confirmed that all 50 features contributed to optimal predictive performance, as models trained on the complete feature set consistently outperformed those trained on reduced subsets.

8.2 Future Work

Future work should focus on validating the performance of the developed ensemble models using larger, real-world datasets with minimal missing data and, ideally, balanced class distributions, such as BioFINDER. Collecting structured data from a substantial number of SCD patients would enable independent validation of the proposed methodology and enhance its generalisability. Successfully replicating the model's performance on such datasets would increase confidence in its clinical applicability and support further research into the use of machine learning for Alzheimer's disease prediction.

Additionally, greater emphasis should be placed on accurately identifying SCD patients. In this project, cognitively normal patients were considered SCD patients due to the subjective nature of the diagnosis, which may have affected the precision of the model outcomes. Future efforts should also prioritise data collection across diverse demographic categories, such as geographical regions and ethnic backgrounds, to improve the robustness and inclusivity of predictive models.

From a technical standpoint, future studies could explore more advanced ensemble strategies, such as implementing weighted soft voting based on the performance of individual base models or incorporating multiple models as final estimators in stacking ensemble configurations. Moreover, broader hyperparameter tuning, particularly for complex models like Random Forest and XGBoost, may yield improved predictive performance when combined with advanced optimisation techniques such as Bayesian Optimisation.

Another important direction is enhancing model interpretability through tools like SHapley Additive exPlanations (SHAP) (Lundberg and Lee 2017) or Local Interpretable Model-agnostic Explanations (LIME) (Ribeiro, Singh, and Guestrin 2016). These can provide clinicians with valuable insights into the most influential features contributing to a patient's risk of progression, thereby improving the model's trustworthiness and clinical relevance. Further exploration of different combinations of feature groups may also uncover clinically meaningful patterns that aid in understanding disease progression.

Finally, incorporating longitudinal data from follow-up visits could enable the identification of temporal patterns associated with progression from SCD to MCI or AD. This temporal perspective may improve classification performance and deepen the understanding of how cognitive decline progresses.

References

- Ahmed Arafa, Ahmed et al. (2022). “Logistic Regression Hyperparameter Optimization for Cancer Classification”. In: *Menoufia Journal of Electronic Engineering Research* 31.1, pp. 1–8. ISSN: 1687-1189. DOI: 10.21608/mjeer.2021.70512.1034 (cit. on pp. 11, 12).
- Albert, Marilyn S. et al. (2011). “The diagnosis of mild cognitive impairment due to Alzheimer’s disease: Recommendations from the National Institute on Aging-Alzheimer’s Association workgroups on diagnostic guidelines for Alzheimer’s disease”. In: *Alzheimer’s & Dementia* 7.3, pp. 270–279. DOI: <https://doi.org/10.1016/j.jalz.2011.03.008> (cit. on p. 7).
- Ali, H. et al. (2019). “A review on data preprocessing methods for class imbalance problem”. In: *International Journal of Engineering and Technology* 8.3, pp. 390–397. URL: <https://www.sciencepubco.com/index.php/ijet/article/view/29508> (cit. on pp. 21–23).
- Anand, M. Vijay et al. (2022). “Gaussian Naïve Bayes Algorithm: A Reliable Technique Involved in the Assortment of the Segregation in Cancer”. In: *Mobile Information Systems* 2022.1, p. 2436946. DOI: <https://doi.org/10.1155/2022/2436946> (cit. on p. 15).
- As’ad, Ihwana (2023). “Advancing Healthcare Diagnostics: A Study on Gaussian Naive Bayes Classification of Blood Samples”. In: *International Journal of Artificial Intelligence in Medical Issues* 1.2, pp. 115–123. DOI: 10.56705/ijaimi.v1i2.120 (cit. on p. 16).
- Ashendorf, Lee et al. (Mar. 2008). “Trail Making Test errors in normal aging, mild cognitive impairment, and dementia”. In: *Archives of Clinical Neuropsychology* 23.2, pp. 129–137. ISSN: 0887-6177. DOI: 10.1016/j.acn.2007.11.005 (cit. on p. 31).
- Ashraf, Abdallah (2023). *Oversampling—Handling Imbalanced Data*. URL: <https://medium.com/@abdallahashraf90x/oversampling-for-better-machine-learning-with-imbalanced-data-68f9b5ac2696> (cit. on p. 21).
- Awan-Ur-Rahman (2023). *Understanding Soft Voting and Hard Voting: A Comparative Analysis of Ensemble Learning Methods*. URL: <https://medium.com/@awanurrahman.cse/understanding-soft-voting-and-hard-voting-a-comparative-analysis-of-ensemble-learning-methods-db0663d2c008> (cit. on p. 16).
- Azur, Melissa J. et al. (Mar. 2011). “Multiple imputation by chained equations: what is it and how does it work?” In: *International Journal of Methods in Psychiatric Research* 20.1, pp. 40–49. DOI: 10.1002/mpr.329 (cit. on p. 20).

- Bisong, Ekaba (2019). “The Multilayer Perceptron (MLP)”. In: *Building Machine Learning and Deep Learning Models on Google Cloud Platform: A Comprehensive Guide for Beginners*. Berkeley, CA: Apress, pp. 401–405. ISBN: 978-1-4842-4470-8. DOI: 10.1007/978-1-4842-4470-8_31 (cit. on p. 14).
- Blennow, K. et al. (2015). “Clinical utility of cerebrospinal fluid biomarkers in the diagnosis of early Alzheimer’s disease”. In: *Alzheimer’s & Dementia* 11.1, pp. 58–69. DOI: 10.1016/j.jalz.2014.02.004 (cit. on pp. 8, 31).
- Bomasang-Layno, Eva and Rachel Bronsther (Sept. 2021). “Diagnosis and Treatment of Alzheimer’s Disease: An Update”. In: *Delaware Journal of Public Health* 7.4, pp. 74–85. DOI: 10.32481/djph.2021.09.009 (cit. on p. 8).
- Bottino, Cássio M.C. et al. (Mar. 2002). “Volumetric MRI Measurements Can Differentiate Alzheimer’s Disease, Mild Cognitive Impairment, and Normal Aging”. In: *International Psychogeriatrics* 14.1, pp. 59–72. ISSN: 1041-6102. DOI: 10.1017/S1041610202008281 (cit. on p. 8).
- Breijyeh, Zeinab and Rafik Karaman (Dec. 2020). “Comprehensive Review on Alzheimer’s Disease: Causes and Treatment”. In: *Molecules* 25.24, p. 5789. DOI: 10.3390/molecules25245789 (cit. on p. 5).
- Breiman, Leo (Aug. 1996). “Bagging predictors”. In: *Machine Learning* 24.2, pp. 123–140. ISSN: 1573-0565. DOI: 10.1007/BF00058655 (cit. on p. 17).
- (2001). “Random Forests”. In: *Machine Learning* 45.1, pp. 5–32. ISSN: 1573-0565. DOI: 10.1023/A:1010933404324 (cit. on p. 18).
- Busche, Marc Aurel and Arthur Konnerth (2016). “Impairments of Neural Circuit Function in Alzheimer’s Disease”. In: *Philosophical Transactions of the Royal Society B: Biological Sciences* 371.1700, p. 20150429. DOI: 10.1098/rstb.2015.0429 (cit. on p. 5).
- Canevelli, Marco et al. (Sept. 2013). “Cognitive interventions targeting subjective cognitive complaints”. In: *American Journal of Alzheimer’s Disease & Other Dementias* 28.6, pp. 560–567. DOI: 10.1177/1533317513494441 (cit. on p. 6).
- Cervantes, Jair et al. (2020). “A comprehensive survey on support vector machine classification: Applications, challenges and trends”. In: *Neurocomputing* 408, pp. 189–215. ISSN: 0925-2312. DOI: <https://doi.org/10.1016/j.neucom.2019.10.118> (cit. on p. 12).
- Charfaoui, Younes (2023). *Resampling to Properly Handle Imbalanced Datasets in Machine Learning*. Accessed April 18, 2025. URL: <https://www.comet.com/site/blog/resampling-to-properly-handle-imbalanced-datasets-in-machine-learning/> (cit. on p. 23).
- Chawla, N. V. et al. (June 2002). “SMOTE: Synthetic Minority Over-sampling Technique”. In: *Journal of Artificial Intelligence Research* 16, pp. 321–357. ISSN: 1076-9757. DOI: 10.1613/jair.953 (cit. on p. 22).
- Chen, Tianqi and Carlos Guestrin (Aug. 2016). “XGBoost: A Scalable Tree Boosting System”. In: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge*

- Discovery and Data Mining. KDD '16. ACM*, pp. 785–794. DOI: 10.1145/2939672.2939785. URL: <http://dx.doi.org/10.1145/2939672.2939785> (cit. on p. 19).
- Corder, E. H. et al. (Aug. 1993). “Gene dose of apolipoprotein E type 4 allele and the risk of Alzheimer’s disease in late onset families”. In: *Science* 261.5123, pp. 921–923. ISSN: 0036-8075. DOI: 10.1126/science.8346443 (cit. on p. 30).
- Cunningham, Pádraig and Sarah Jane Delany (July 2021). “k-Nearest Neighbour Classifiers - A Tutorial”. In: *ACM Computing Surveys* 54.6, pp. 1–25. ISSN: 1557-7341. DOI: 10.1145/3459665 (cit. on p. 13).
- Demircioğlu, A. (May 21, 2024). “Applying oversampling before cross-validation will lead to high bias in radiomics”. In: *Scientific Reports* 14.1, p. 11563. DOI: 10.1038/s41598-024-62585-z (cit. on p. 46).
- Dolcet-Negre, Marta M. et al. (2023). “Predicting Conversion from Subjective Cognitive Decline to Mild Cognitive Impairment and Alzheimer’s Disease Dementia Using Ensemble Machine Learning”. In: *Journal of Alzheimer’s Disease* 93.1. PMID: 36938735, pp. 125–140. DOI: 10.3233/JAD-221002 (cit. on pp. 1, 2, 25, 26, 28, 74, 76).
- Donohue, Michael C. et al. (Aug. 2014). “The Preclinical Alzheimer Cognitive Composite: Measuring Amyloid-Related Decline”. In: *JAMA Neurology* 71.8, pp. 961–970. DOI: 10.1001/jamaneurol.2014.803 (cit. on p. 31).
- Estévez-González, Armando et al. (2003). “Rey verbal learning test is a useful tool for differential diagnosis in the preclinical phase of Alzheimer’s disease: comparison with mild cognitive impairment and normal aging”. In: *International Journal of Geriatric Psychiatry* 18.11, pp. 1021–1028. DOI: <https://doi.org/10.1002/gps.1010> (cit. on p. 31).
- Fan, Yuqiang et al. (2020). “Chiller fault detection and diagnosis by knowledge transfer based on adaptive imbalanced processing”. In: *Science and Technology for the Built Environment* 26.8, pp. 1082–1099. DOI: 10.1080/23744731.2020.1757327 (cit. on p. 21).
- Farias, Sarah Tomaszewski et al. (July 2008). “The measurement of everyday cognition (ECog): scale development and psychometric properties”. In: *Neuropsychology* 22.4, pp. 531–544. DOI: 10.1037/0894-4105.22.4.531 (cit. on p. 31).
- Frisoni, G. B. et al. (2010). “The clinical use of structural MRI in Alzheimer disease”. In: *Nature Reviews Neurology* 6.2, pp. 67–77. DOI: 10.1038/nrneurol.2009.215 (cit. on p. 31).
- GeeksforGeeks (2024). *Gamma Parameter in SVM*. Accessed April 19, 2025. URL: <https://www.geeksforgeeks.org/gamma-parameter-in-svm/> (cit. on p. 13).
- Grassi, M. et al. (2019). “A Novel Ensemble-Based Machine Learning Algorithm to Predict the Conversion From Mild Cognitive Impairment to Alzheimer’s Disease Using Socio-Demographic Characteristics, Clinical Information, and Neuropsychological Measures”. In: *Frontiers in Neurology* 10, p. 756. DOI: 10.3389/fneur.2019.00756 (cit. on pp. 25–28).
- Hae, Kim Jong (2019). “Multicollinearity and misleading statistical results”. In: *Korean J Anesthesiol* 72.6, pp. 558–569. DOI: 10.4097/kja.19087 (cit. on p. 26).

- Han, Hui, Wen-Yuan Wang, and Bing-Huan Mao (2005). “Borderline-SMOTE: A New Over-Sampling Method in Imbalanced Data Sets Learning”. In: *Advances in Intelligent Computing*. Ed. by De-Shuang Huang, Xiao-Ping Zhang, and Guang-Bin Huang. Berlin, Heidelberg: Springer Berlin Heidelberg, pp. 878–887. URL: https://doi.org/10.1007/11538059_91 (cit. on p. 22).
- He, Haibo et al. (2008). “ADASYN: Adaptive synthetic sampling approach for imbalanced learning”. In: *2008 IEEE International Joint Conference on Neural Networks (IEEE World Congress on Computational Intelligence)*, pp. 1322–1328. DOI: 10.1109/IJCNN.2008.4633969 (cit. on p. 21).
- He, Zhiyuan et al. (2019). *Gradient Boosting Machine: A Survey*. arXiv: 1908.06951 [stat.ML]. URL: <https://arxiv.org/abs/1908.06951> (cit. on p. 19).
- Ishwaran, Hemant and Robert O’Brien (2021). “Commentary: The problem of class imbalance in biomedical data”. In: *The Journal of Thoracic and Cardiovascular Surgery* 161.6, pp. 1940–1941. ISSN: 0022-5223. DOI: 10.1016/j.jtcvs.2020.06.052 (cit. on p. 45).
- Jack, C. R. Jr et al. (2010). “Hypothetical model of dynamic biomarkers of the Alzheimer’s pathological cascade”. In: *Lancet Neurology* 9.1, pp. 119–128. DOI: 10.1016/S1474-4422(09)70299-6 (cit. on p. 31).
- Jack, C. R. Jr. et al. (Aug. 2016). “A/T/N: An unbiased descriptive classification scheme for Alzheimer disease biomarkers”. In: *Neurology* 87.5, pp. 539–547. DOI: 10.1212/WNL.0000000000002923 (cit. on pp. 8, 31).
- Jack Jr., Clifford R. et al. (2018). “NIA-AA Research Framework: Toward a biological definition of Alzheimer’s disease”. In: *Alzheimer’s & Dementia* 14.4, pp. 535–562. DOI: <https://doi.org/10.1016/j.jalz.2018.02.018> (cit. on pp. 6, 7).
- Jagust, William J. et al. (May 2010). “The Alzheimer’s Disease Neuroimaging Initiative positron emission tomography core”. In: *Alzheimer’s & Dementia* 6.3, pp. 221–229. ISSN: 1552-5260. DOI: 10.1016/j.jalz.2010.03.003 (cit. on p. 30).
- Jain, Kopal (2021). *How to Improve Naive Bayes?* Accessed April 18, 2025. URL: <https://medium.com/analytics-vidhya/how-to-improve-naive-bayes-9fa698e14cba> (cit. on p. 16).
- Jansen, Willemijn J. et al. (May 2015). “Prevalence of Cerebral Amyloid Pathology in Persons Without Dementia: A Meta-analysis”. In: *JAMA* 313.19, pp. 1924–1938. ISSN: 0098-7484. DOI: 10.1001/jama.2015.4668 (cit. on p. 5).
- Jessen, Frank, Rebecca E. Amariglio, et al. (Nov. 2014). “A conceptual framework for research on subjective cognitive decline in preclinical Alzheimer’s disease”. In: *Alzheimer’s & Dementia* 10.6. Epub 2014 May 3, pp. 844–852. DOI: 10.1016/j.jalz.2014.01.001 (cit. on p. 1).
- Jessen, Frank, Steffen Wolfgruber, et al. (2023). “Subjective cognitive decline and stage 2 of Alzheimer disease in patients from memory centers”. In: *Alzheimer’s & Dementia* 19.2, pp. 487–497. DOI: <https://doi.org/10.1002/alz.12674> (cit. on p. 6).

- Kalirane, Mbali (2025). *XGBoost Classification In Depth*. Accessed April 18, 2025. URL: <https://www.analyticsvidhya.com/blog/2023/01/ensemble-learning-methods-bagging-boosting-and-stacking/> (cit. on pp. 16, 17).
- Karr, Julia E. et al. (Mar. 2024). “Detecting cognitive decline in high-functioning older adults: The relationship between subjective cognitive concerns, frequency of high neuropsychological test scores, and the frontoparietal control network”. In: *Journal of the International Neuropsychological Society* 30.3, pp. 220–231. DOI: 10.1017/S1355617723000607 (cit. on p. 42).
- Koteluk, Olga et al. (Jan. 2021). “How Do Machines Learn? Artificial Intelligence as a New Era in Medicine”. In: *Journal of Personalized Medicine* 11.1, p. 32. DOI: 10.3390/jpm11010032 (cit. on pp. 9, 10).
- Kueper, Jacqueline K., Mark Speechley, and Manuel Montero-Odasso (2018). “The Alzheimer’s Disease Assessment Scale-Cognitive Subscale (ADAS-Cog): Modifications and Responsiveness in Pre-Dementia Populations. A Narrative Review”. In: *Journal of Alzheimer’s Disease* 63.2, pp. 423–444. DOI: 10.3233/JAD-170991 (cit. on p. 31).
- Kummer, Alex et al. (2022). “Machine learning-based software sensors for machine state monitoring - The role of SMOTE-based data augmentation”. In: *Results in Engineering* 16, p. 100778. ISSN: 2590-1230. DOI: <https://doi.org/10.1016/j.rineng.2022.100778> (cit. on p. 23).
- Landau, Susan M. et al. (July 2011). “Associations between cognitive, functional, and FDG-PET measures of decline in AD and MCI”. In: *Neurobiology of Aging* 32.7, pp. 1207–1218. ISSN: 0197-4580. DOI: 10.1016/j.neurobiolaging.2009.07.002 (cit. on p. 30).
- Lerch, Ondrej et al. (2024). “Predicting progression from subjective cognitive decline to mild cognitive impairment or dementia based on brain atrophy patterns”. In: *Alzheimer’s Research & Therapy* 16.1, p. 153. ISSN: 1758-9193. DOI: 10.1186/s13195-024-01517-5 (cit. on pp. 2, 25, 26, 28, 74, 76).
- Liu, Chia-Chen et al. (Feb. 2013). “Apolipoprotein E and Alzheimer disease: risk, mechanisms and therapy”. In: *Nature Reviews Neurology* 9.2. Erratum in: *Nat Rev Neurol*. 2013. doi: 10.1038/nrneurol.2013.32. Liu, Chia-Chan [corrected to Liu, Chia-Chen], pp. 106–118. ISSN: 1759-4766. DOI: 10.1038/nrneurol.2012.263 (cit. on p. 30).
- Lundberg, Scott and Su-In Lee (2017). *A Unified Approach to Interpreting Model Predictions*. arXiv: 1705.07874 [cs.AI]. URL: <https://arxiv.org/abs/1705.07874> (cit. on p. 78).
- Marshall, G.A. et al. (2014). “Everyday cognition scale items that best discriminate between and predict progression from clinically normal to mild cognitive impairment”. In: *Current Alzheimer Research* 11.9, pp. 853–861. ISSN: 1875-5828. DOI: 10.2174/1567205011666141001120903 (cit. on p. 32).
- Mattsson, Niklas et al. (July 2019). “Association Between Longitudinal Plasma Neurofilament Light and Neurodegeneration in Patients With Alzheimer Disease”. In: *JAMA Neurology* 76.7. Erratum in: *JAMA Neurol*. 2019 Jul 1;76(7):872. doi: 10.1001/jamaneurol.2019.1649, pp. 791–799. ISSN: 2168-6149. DOI: 10.1001/jamaneurol.2019.0765 (cit. on pp. 8, 31).

- Mienye, Ibomoiye Domor and Nobert Jere (2024). “A Survey of Decision Trees: Concepts, Algorithms, and Applications”. In: *IEEE Access* 12, pp. 86716–86727. DOI: 10.1109/ACCESS.2024.3416838 (cit. on p. 15).
- Mishra, Arnab Kumar et al. (2022). “Feature fusion based machine learning pipeline to improve breast cancer prediction”. In: *Multimedia Tools and Applications* 81.26, pp. 37627–37655. ISSN: 1573-7721. DOI: 10.1007/s11042-022-13498-4 (cit. on p. 22).
- Mohammed, Roweida, Jumanah Rawashdeh, and Malak Abdullah (2020). “Machine Learning with Oversampling and Undersampling Techniques: Overview Study and Experimental Results”. In: *2020 11th International Conference on Information and Communication Systems (ICICS)*, pp. 243–248. DOI: 10.1109/ICICS49469.2020.239556 (cit. on p. 23).
- Morris, John C. (1997). “Clinical Dementia Rating: A Reliable and Valid Diagnostic and Staging Measure for Dementia of the Alzheimer Type”. In: *International Psychogeriatrics* 9, pp. 173–176. ISSN: 1041-6102. DOI: <https://doi.org/10.1017/S1041610297004870> (cit. on pp. 8, 31).
- Nakamura, Akihiko et al. (Feb. 2018). “High performance plasma amyloid- β biomarkers for Alzheimer’s disease”. In: *Nature* 554.7691, pp. 249–254. DOI: 10.1038/nature25456 (cit. on pp. 8, 31).
- Nasreddine, Ziad S. et al. (Apr. 2005). “The Montreal Cognitive Assessment, MoCA: A Brief Screening Tool for Mild Cognitive Impairment”. In: *Journal of the American Geriatrics Society* 53.4. Erratum in: *J Am Geriatr Soc.* 2019 Sep;67(9):1991. doi: 10.1111/jgs.15925, pp. 695–699. DOI: 10.1111/j.1532-5415.2005.53221.x (cit. on p. 31).
- Omarzai, Fraidoon (2024a). *Random Forest In-Depth*. Accessed April 18, 2025. URL: <https://medium.com/@fraidoonomarzai99/random-forest-in-depth-f0556817c40b> (cit. on p. 18).
- (2024b). *XGBoost Classification In Depth*. Accessed April 18, 2025. URL: <https://medium.com/@fraidoonomarzai99/xgboost-classification-in-depth-979f11ef4bf9> (cit. on p. 19).
- Palmqvist, Sebastian et al. (Dec. 2019). “Cerebrospinal fluid and plasma biomarker trajectories with increasing amyloid deposition in Alzheimer’s disease”. In: *EMBO Molecular Medicine* 11.12, e11170. DOI: 10.15252/emmm.201911170 (cit. on pp. 8, 31).
- Parashar, Abhay (2022). *How To Ensemble Machine Learning Models For Better Performance*. Accessed April 18, 2025. URL: <https://heartbeat.comet.ml/how-to-ensemble-machine-learning-models-for-better-performance-2d893889b4ae> (cit. on p. 17).
- Parnetti, Lucilla et al. (Jan. 2019). “Prevalence and risk of progression of preclinical Alzheimer’s disease stages: a systematic review and meta-analysis”. In: *Alzheimer’s Research & Therapy* 11.1, p. 7. ISSN: 1758-9193. DOI: 10.1186/s13195-018-0459-7 (cit. on p. 6).
- Petersen, Ronald and et al. (2005). *Alzheimer’s Disease Neuroimaging Initiative (ADNI) Protocol*. Final Protocol, March 2, 2005. Amendments through September 19, 2008. Accessed April 18, 2025. URL: https://adni.loni.usc.edu/wp-content/themes/freshnews-dev-v2/documents/clinical/ADNI-1_Protocol.pdf (cit. on p. 88).

- Petersen, Ronald and et al. (2015). *Alzheimer's Disease Neuroimaging Initiative 2 (ADNI-2) Protocol, ADC-039*. Version 5.0, Amendment #4, July 22, 2015. Accessed April 18, 2025. URL: https://adni.loni.usc.edu/wp-content/themes/freshnews-dev-v2/documents/clinical/ADNI-2_Protocol.pdf (cit. on p. 88).
- Popescu, Marius-Constantin et al. (July 2009). "Multilayer perceptron and neural networks". In: *WSEAS Transactions on Circuits and Systems* 8 (cit. on p. 14).
- Priyatno, Arif Mudi and Triyanna Widiyaningtyas (Feb. 2024). "A SYSTEMATIC LITERATURE REVIEW: RECURSIVE FEATURE ELIMINATION ALGORITHMS". In: *JITK (Jurnal Ilmu Pengetahuan dan Teknologi Komputer)* 9.2, pp. 196–207. DOI: 10.33480/jitk.v9i2.5015 (cit. on p. 27).
- Rabin, Lauren A. et al. (Sept. 2020). "An approach to classifying subjective cognitive decline in community-dwelling elders". In: *Alzheimer's & Dementia: Diagnosis, Assessment & Disease Monitoring* 12.1, e12103. DOI: 10.1002/dad2.12103 (cit. on p. 42).
- Raschka, Sebastian (2020). *Model Evaluation, Model Selection, and Algorithm Selection in Machine Learning*. arXiv: 1811.12808 [cs.LG]. URL: <https://arxiv.org/abs/1811.12808> (cit. on p. 24).
- Rattanabannakit, Chitchai et al. (2016). "The Cognitive Change Index as a Measure of Self and Informant Perception of Cognitive Decline: Relation to Neuropsychological Tests". In: *Journal of Alzheimer's Disease* 51.4, pp. 1145–1155. DOI: 10.3233/JAD-150729 (cit. on p. 8).
- Rey, André (1964). *L'examen clinique en psychologie*. Paris: Presses Universitaires de France (cit. on p. 31).
- Ribeiro, Marco Tulio, Sameer Singh, and Carlos Guestrin (2016). "Why Should I Trust You?": Explaining the Predictions of Any Classifier. arXiv: 1602.04938 [cs.LG]. URL: <https://arxiv.org/abs/1602.04938> (cit. on p. 78).
- Rosen, William G., Richard C. Mohs, and Karl L. Davis (Nov. 1984). "A new rating scale for Alzheimer's disease". In: *American Journal of Psychiatry* 141.11, pp. 1356–1364. DOI: 10.1176/ajp.141.11.1356 (cit. on p. 31).
- Salman, Hasan Ahmed, Ali Kalakech, and Amani Steiti (June 2024). "Random Forest Algorithm Overview". In: *Babylonian Journal of Machine Learning* 2024, pp. 69–79. DOI: 10.58496/BJML/2024/007 (cit. on p. 18).
- Salmi, Mabrouka et al. (2024). "Handling imbalanced medical datasets: review of a decade of research". In: *Artificial Intelligence Review* 57.10, p. 273. ISSN: 1573-7462. DOI: 10.1007/s10462-024-10884-2 (cit. on p. 45).
- Sarker, Iqbal H. (2021). "Machine Learning: Algorithms, Real-World Applications and Research Directions". In: *SN Computer Science* 2.3, p. 160. ISSN: 2661-8907. DOI: 10.1007/s42979-021-00592-x (cit. on p. 9).
- Scikit-Learn Developers (2025). *scikit-learn: Machine Learning in Python*. Accessed: 2025-04-19. URL: <https://scikit-learn.org/stable/index.html> (cit. on p. 11).

- Shah, Anoop D. et al. (Mar. 2014). “Comparison of random forest and parametric imputation models for imputing missing data using MICE: a CALIBER study”. In: *American Journal of Epidemiology* 179.6. Epub 2014 Jan 12, pp. 764–774. DOI: 10.1093/aje/kwt312 (cit. on p. 44).
- Shen, Xue-Ning et al. (2021). “Subtle cognitive impairment as a marker of Alzheimer’s pathologies and clinical progression in cognitively normal individuals”. In: *Alzheimer’s & Dementia: Diagnosis, Assessment & Disease Monitoring* 13.1, e12198. DOI: <https://doi.org/10.1002/dad2.12198> (cit. on p. 31).
- Shi, H. et al. (2021). “Nano-SAR Modeling for Predicting the Cytotoxicity of Metal Oxide Nanoparticles to PaCa2”. In: *Molecules* 26.8, p. 2188. DOI: 10.3390/molecules26082188. URL: <https://doi.org/10.3390/molecules26082188> (cit. on p. 27).
- Silverberg, Nina et al. (2018). “NIA commentary on the NIA-AA Research Framework: Towards a biological definition of Alzheimer’s disease”. In: *Alzheimer’s & Dementia* 14.4, pp. 576–578. DOI: <https://doi.org/10.1016/j.jalz.2018.03.004> (cit. on pp. 1, 5).
- Song, Minyoung et al. (Oct. 2021). “Measurement of Subjective Cognitive Decline (SCD) Using Korean-Everyday Cognition (K-ECog) as a Screening Tool: a Feasibility Study”. In: *Dementia and Neurocognitive Disorders* 20.4, pp. 80–88. DOI: 10.12779/dnd.2021.20.4.80 (cit. on p. 8).
- Sperling, Reisa A. et al. (2011). “Toward defining the preclinical stages of Alzheimer’s disease: Recommendations from the National Institute on Aging-Alzheimer’s Association workgroups on diagnostic guidelines for Alzheimer’s disease”. In: *Alzheimer’s & Dementia* 7.3, pp. 280–292. DOI: <https://doi.org/10.1016/j.jalz.2011.03.003> (cit. on pp. 6, 7).
- Tahami Monfared, Ali A. et al. (June 2022). “Alzheimer’s Disease: Epidemiology and Clinical Progression”. In: *Neurology and Therapy* 11.2, pp. 553–569. ISSN: 2193-8231. DOI: 10.1007/s40120-022-00338-8 (cit. on pp. 6, 7).
- Taunk, Kashvi et al. (2019). “A Brief Review of Nearest Neighbor Algorithm for Learning and Classification”. In: *2019 International Conference on Intelligent Computing and Control Systems (ICCS)*, pp. 1255–1260. DOI: 10.1109/ICCS45141.2019.9065747 (cit. on p. 13).
- Thabtah, Fadi, Robinson Spencer, and Yongsheng Ye (2020). “The correlation of everyday cognition test scores and the progression of Alzheimer’s disease: a data analytics study”. In: *Health Information Science and Systems* 8.1, p. 24. ISSN: 2047-2501. DOI: 10.1007/s13755-020-00114-8 (cit. on p. 32).
- Tombaugh, Tom N. and Nancy J. McIntyre (Sept. 1992). “The Mini-Mental State Examination: A Comprehensive Review”. In: *Journal of the American Geriatrics Society* 40.9, pp. 922–935. DOI: 10.1111/j.1532-5415.1992.tb01992.x (cit. on pp. 8, 31).
- Vaj, Tiya (2024). *When to use Mean/Median/Mode imputation*: URL: <https://vtiya.medium.com/when-to-use-mean-median-mode-imputation-b0fd6be247db> (cit. on p. 20).

- Wechsler, David A. (1987). *Wechsler Memory Scale–Revised*. New York: The Psychological Corporation, Harcourt Brace Jovanovich, Inc., p. 150 (cit. on p. 31).
- WHO (2025). *Dementia*. URL: <https://www.who.int/news-room/fact-sheets/detail/dementia> (cit. on p. 5).
- Williams, Melissa M. et al. (Feb. 2013). “Progression of Alzheimer’s disease as measured by Clinical Dementia Rating Sum of Boxes scores”. In: *Alzheimer’s & Dementia* 9.1 Suppl, S39–S44. DOI: 10.1016/j.jalz.2012.01.005 (cit. on p. 31).
- Williamson, M. et al. (2022). “Validation of a digit symbol substitution test for use in supervised and unsupervised assessment in mild Alzheimer’s disease”. In: *Journal of Clinical and Experimental Neuropsychology* 44.10. Epub 2023 Feb 22, pp. 768–779. DOI: 10.1080/13803395.2023.2179977 (cit. on p. 31).
- Wolpert, David H. (1992). “Stacked generalization”. In: *Neural Networks* 5.2, pp. 241–259. ISSN: 0893-6080. DOI: [https://doi.org/10.1016/S0893-6080\(05\)80023-1](https://doi.org/10.1016/S0893-6080(05)80023-1) (cit. on p. 16).
- Yang, Yuchao, Hamed A. Khorshidi, and Aickelin Uwe (July 2024). “A review on over-sampling techniques in classification of multi-class imbalanced datasets: insights for medical problems”. In: *Frontiers in Digital Health* 6, p. 1430245. DOI: 10.3389/fdgth.2024.1430245 (cit. on p. 21).
- Ying, Xue (Feb. 2019). “An Overview of Overfitting and its Solutions”. In: *Journal of Physics: Conference Series* 1168.2, p. 022022. DOI: 10.1088/1742-6596/1168/2/022022 (cit. on p. 11).
- Yue, Ling et al. (2020). “Prediction of 7-year’s Conversion from Subjective Cognitive Decline to Mild Cognitive Impairment”. In: *medRxiv*. DOI: 10.1101/2020.05.04.20091256 (cit. on pp. 1–3, 25, 26, 28, 75, 76).
- Zabor, Emily C. et al. (2022). “Logistic Regression in Clinical Studies”. In: *International Journal of Radiation Oncology, Biology, Physics* 112.2, pp. 271–277. DOI: 10.1016/j.ijrobp.2021.08.007. URL: <https://doi.org/10.1016/j.ijrobp.2021.08.007> (cit. on p. 11).
- Zhang, Yan-Ping, Li-Na Zhang, and Yong-Cheng Wang (2010). “Cluster-based majority under-sampling approaches for class imbalance learning”. In: *2010 2nd IEEE International Conference on Information and Financial Engineering*, pp. 400–404. DOI: 10.1109/ICIFE.2010.5609385 (cit. on p. 23).
- Zhang, Z. (Jan. 2016). “Missing data imputation: focusing on single imputation”. In: *Annals of Translational Medicine* 4.1, p. 9. DOI: 10.3978/j.issn.2305-5839.2015.12.38 (cit. on p. 20).

Appendix A

ADNI Criteria

Table A.1 summarises the criteria for Cognitively Normal (CN), Subjective Cognitive Decline (SCD), Mild Cognitive Impairment (MCI) and Alzheimer's Disease (AD) individuals (Petersen and al. 2005; Petersen and al. 2015).

Table A.1: Summarised ADNI criteria for CN, SCD, MCI and AD (Petersen and al. 2005; Petersen and al. 2015).

#	CN	SCD	MCI	AD
1	Subject must be free of memory complaints, verified by a study partner, beyond what one would expect for age	Subject must have a significant subjective memory concern as reported by subject, study partner, or clinician.	Subject must have a subjective memory concern as reported by subject, study partner, or clinician.	Same as EMCI
1a	n/a	Significant memory concern confirmed by Cognitive Change Index score ≥ 16 (from the first 12 questions)	n/a	n/a

2	Normal memory function documented by scoring above education-adjusted cutoffs on the Logical Memory II subscale (Delayed Paragraph Recall, Paragraph A only) from the Wechsler Memory Scale – Revised (max score = 25): a. ≥ 9 for 16+ years of education b. ≥ 5 for 8–15 years c. ≥ 3 for 0–7 years	Same as CN	Abnormal memory function documented by scoring within education-adjusted ranges on Logical Memory II subscale: a. 9–11 for 16+ years b. 5–9 for 8–15 years c. 3–6 for 0–7 years	Abnormal memory function documented by scoring within education-adjusted ranges on Logical Memory II subscale: a. ≤ 8 for 16+ years b. ≤ 4 for 8–15 years c. ≤ 2 for 0–7 years
3	Mini-Mental State Exam score between 24 and 30 (inclusive). Exceptions may be made for subjects with < 8 years of education at project director’s discretion	Same as CN	Same as CN	Mini-Mental State Exam score between 20 and 26 (inclusive). Exceptions may be made for subjects with < 8 years of education at project director’s discretion
4	Clinical Dementia Rating = 0. Memory Box score must be 0	Same as CN	Clinical Dementia Rating = 0.5. Memory Box score must be at least 0.5	Clinical Dementia Rating = 0.5 or 1.0
5	Cognitively normal, based on absence of significant impairment in cognitive functions or daily activities	Same as CN	General cognition and function preserved such that AD diagnosis cannot be made at screening	NINCDS/ADRDA criteria for probable AD

Appendix B

Combining Plasma data with ADNIMERGE dataset

```
1 # Load ADNIMERGE Dataset
2 adnimerge = pd.read_csv('./ADNIMERGE_29Sep2024.csv')
3
4
5
6 # Load Plasma NFL Datasets
7 blennowplasmanfl = pd.read_csv('./ADNI_BLENNOWPLASMANFL.csv')
8 blennowplasmanfl = blennowplasmanfl[['RID', 'EXAMDATE', 'PLASMA_NFL']]
9
10 blennowplasmanfl_long = pd.read_csv('./ADNI_BLENNOWPLASMANFLLONG.csv')
11 blennowplasmanfl_long = blennowplasmanfl_long[['RID', 'EXAMDATE',
12         'PLASMA_NFL']]
13
14 fnih_plasma = pd.read_csv('./FNIH_PLASMA_ABETA_PROJECT_ADX_VUMC.csv')
15 fnih_plasma_nfl = fnih_plasma[['RID', 'EXAMDATE', 'NF_LIGHT']].copy()
16 fnih_plasma_nfl.rename(columns={'NF_LIGHT': 'PLASMA_NFL'}, inplace=True)
17
18 # Combine the Plasma NFL data from different datasets into single dataframe
19 plasma_nfl = pd.concat([blennowplasmanfl, blennowplasmanfl_long,
20         fnih_plasma_nfl]).sort_values(by=['RID', 'EXAMDATE'])
21 plasma_nfl = plasma_nfl.drop_duplicates(subset=['RID', 'EXAMDATE'],
22         keep='first')
23
24 # Load Plasma TAU Datasets
25 uog_ptau = pd.read_csv('./UNIVERSITY_OF_GOTHENBURG_PTAU181.csv')
26 uog_ptau = uog_ptau[['RID', 'EXAMDATE', 'PLASMAPTAU181']]
27 uog_ptau.rename(columns={'PLASMAPTAU181': 'PLASMA_PTAU'}, inplace=True)
28
29 blennow_ttau = pd.read_csv('./BLENNOWPLASMATAU.csv')
30 blennow_ttau = blennow_ttau[['RID', 'EXAMDATE', 'PLASMATAU']]
31 blennow_ttau.rename(columns={'PLASMATAU': 'PLASMA_PTAU'}, inplace=True)
32
33 fnih_plasma_ptau = pd.read_csv('./FNIH_PLASMA_PTAU_PROJECT.csv')
34 fnih_plasma_ptau = fnih_plasma_ptau[fnih_plasma_ptau['IMMUNOASSAY'] ==
35         'pTau181 Simoa']
36 fnih_plasma_ptau = fnih_plasma_ptau[['RID', 'EXAMDATE', 'PTAU_181']]
37 fnih_plasma_ptau.rename(columns={'PTAU_181': 'PLASMA_PTAU'}, inplace=True)
```

```

36
37 # Combine the Plasma Ptau data from different datasets into single
    dataframe
38 plasma_ptau = pd.concat([uog_ptau,
    fnih_plasma_ptau]).sort_values(by=['RID', 'EXAMDATE'])
39 plasma_ptau = plasma_ptau.drop_duplicates(subset=['RID', 'EXAMDATE'],
    keep='first')
40
41
42
43 # Load Plasma ABETA Datasets
44 # Get ABETA42/40 ratio from ABETA 42 and ABETA 40 to remove unit
    differences in measurements
45 bateman_plasma_abeta =
    pd.read_csv('./BATEMAN_LAB_PLASMA_ABETA42_ABETA40.csv')
46 bateman_plasma_abeta = bateman_plasma_abeta[['RID', 'VISCODE2',
    'Abeta_42_conc', 'Abeta_40_conc']]
47 bateman_plasma_abeta.rename(columns={'Abeta_42_conc': 'PLASMA_ABETA42',
    'Abeta_40_conc': 'PLASMA_ABETA40', 'VISCODE2': 'VISCODE'}, inplace=True)
48 bateman_plasma_abeta['PLASMA_ABETA42/40'] =
    round(bateman_plasma_abeta['PLASMA_ABETA42'] /
    bateman_plasma_abeta['PLASMA_ABETA40'], 2)
49 bateman_plasma_abeta = bateman_plasma_abeta[['RID', 'VISCODE',
    'PLASMA_ABETA42/40']]
50
51 uw_plasma_abeta = pd.read_csv('./UW_PLASMA_EV_APOGEE.csv')
52 uw_plasma_abeta = uw_plasma_abeta[['RID', 'VISCODE', 'ABETA42', 'ABETA40']]
53 uw_plasma_abeta.rename(columns={'ABETA42': 'PLASMA_ABETA42', 'ABETA40':
    'PLASMA_ABETA40'}, inplace=True)
54 uw_plasma_abeta['PLASMA_ABETA42/40'] =
    round(uw_plasma_abeta['PLASMA_ABETA42'] /
    uw_plasma_abeta['PLASMA_ABETA40'], 2)
55 uw_plasma_abeta = uw_plasma_abeta[['RID', 'VISCODE', 'PLASMA_ABETA42/40']]
56
57 fnih_plasma_abeta = fnih_plasma[['RID', 'VISCODE2', 'ABETA42', 'ABETA40']]
58 fnih_plasma_abeta.rename(columns={'ABETA42': 'PLASMA_ABETA42', 'ABETA40':
    'PLASMA_ABETA40', 'VISCODE2': 'VISCODE'}, inplace=True)
59 fnih_plasma_abeta['PLASMA_ABETA42/40'] =
    round(fnih_plasma_abeta['PLASMA_ABETA42'] /
    fnih_plasma_abeta['PLASMA_ABETA40'], 2)
60 fnih_plasma_abeta = fnih_plasma_abeta[['RID', 'VISCODE',
    'PLASMA_ABETA42/40']]
61
62 upen_plasma_abeta = pd.read_csv('./UPENNPLASMA_20Oct2024.csv')
63 upen_plasma_abeta = upen_plasma_abeta[['RID', 'VISCODE', 'AB42', 'AB40']]
64 upen_plasma_abeta.rename(columns={'AB42': 'PLASMA_ABETA42', 'AB40':
    'PLASMA_ABETA40'}, inplace=True)
65 upen_plasma_abeta['PLASMA_ABETA42/40'] =
    round(upen_plasma_abeta['PLASMA_ABETA42'] /
    upen_plasma_abeta['PLASMA_ABETA40'], 2)
66 upen_plasma_abeta = upen_plasma_abeta[['RID', 'VISCODE',
    'PLASMA_ABETA42/40']]
67
68 # Combine the ABETA42/40 data from different datasets into single dataframe
69 plasma_abeta = pd.concat([uw_plasma_abeta, bateman_plasma_abeta,
    fnih_plasma_abeta, upen_plasma_abeta]).sort_values(by=['RID',
    'VISCODE'])

```

```
70 plasma_abeta = plasma_abeta.drop_duplicates(subset=['RID', 'VISCODE'],
71      keep='first')
72
73
74 # Merge all Plasma data to data from ADNIMERGE
75 df = adnimerge
76 df = df.merge(plasma_nfl, on=['RID', 'EXAMDATE'], how='left')
77 df = df.merge(plasma_ptau, on=['RID', 'EXAMDATE'], how='left')
78 df = df.merge(plasma_abeta, on=['RID', 'VISCODE'], how='left')
79 df.to_csv('./ADNIMERGE_with_Plasma.csv', sep=',', index=False,
      encoding='utf-8')
```

Listing B.1: Code for Combining Plasma data with ADNIMERGE dataset

Appendix C

Constructing the 10-year conversion dataset

```
1 # Consider both CN and SMC patients as SCD patients
2 SCD_bl = df[df['DX_bl'].str.contains('CN|SMC', case=False,
   na=False)].sort_values(by=['RID', 'Years_bl'])
3
4 # SCD patients that converted to MCI/AD
5 SCD_converters_all = SCD_bl[SCD_bl['DX'].str.contains('MCI|Dementia',
   case=False, na=False)]
6
7 # SCD patients not converted to MCI/AD are non-converters
8 SCD_non_converters_last_date =
   SCD_bl[~SCD_bl['RID'].isin(SCD_converters_all['RID'].values)]
9
10
11 # Keep the SCD non-converters that have data longer than 10 years from
   baseline
12 SCD_non_converters_last_date['Years_bl'] =
   SCD_non_converters_last_date['Years_bl'].astype(float)
13 SCD_non_converters_last_date =
   SCD_non_converters_last_date[(SCD_non_converters_last_date['Years_bl']
   >= 10.0)]
14 SCD_non_converters_last_date =
   SCD_non_converters_last_date.sort_values(by=['RID', 'Years_bl'])
15
16
17 # Keep the latest patient record date
18 SCD_non_converters_last_date =
   SCD_non_converters_last_date.drop_duplicates(subset=['RID'],
   keep='last')
19
20
21 # get the maximum number of years stayed in SCD for non-converters
22 max_year_stay_in_SCD = {}
23 for row in SCD_non_converters_last_date.iloc:
24     max_year_stay_in_SCD[row['RID']] = row['Years_bl']
25
26
27 # Keep all patient records that are available from 10 years before the
   latest record of non-converters (which are all eligible for our dataset)
```

```

28 SCD_stay_10_years =
    SCD_bl[SCD_bl['RID'].isin(SCD_non_converters_last_date['RID'].values)]
29 SCD_stay_10_years['Years_from_last_SCD_stable'] =
    SCD_stay_10_years.apply(lambda row: row['Years_bl'] -
        max_year_stay_in_SCD[row['RID']], axis=1)
30 SCD_stay_10_years['Years_from_last_SCD_stable'] =
    SCD_stay_10_years['Years_from_last_SCD_stable'].astype(float)
31 SCD_stay_10_years =
    SCD_stay_10_years[(SCD_stay_10_years['Years_from_last_SCD_stable'] <=
        -10.0)]
32
33
34 # Get the first record date of the SCD patients converting to MCI/AD
35 SCD_converters_first_date =
    SCD_bl[SCD_bl['DX'].str.contains('MCI|Dementia', case=False,
        na=False)].sort_values(by=['RID', 'Years_bl'])
36 SCD_converters_first_date =
    SCD_converters_first_date.drop_duplicates(subset=['RID'], keep='first')
37
38
39 # Get the year from base line in which SCD patients converted to MCI/AD
40 year_converted_to_MCI_AD = {}
41 for row in SCD_converters_first_date.iterrows():
42     year_converted_to_MCI_AD[row['RID']] = row['Years_bl']
43
44
45 # Record the difference of the year of record and the year that the
    patient converted to MCI/AD
46 SCD_converters =
    SCD_bl[SCD_bl['RID'].isin(SCD_converters_first_date['RID'].values)]
47 SCD_converters['Years_from_conversion'] = SCD_converters.apply(lambda row:
    row['Years_bl'] - year_converted_to_MCI_AD[row['RID']], axis=1)
48
49
50 # Get patient records that are within 10 years of the year of conversion.
    These records are considered as conversion data
51 SCD_converters['Years_from_conversion'] =
    SCD_converters['Years_from_conversion'].astype(float)
52 SCD_converters_within_10_years =
    SCD_converters[(SCD_converters['Years_from_conversion'] > -10.0)]
53 SCD_converters_within_10_years =
    SCD_converters_within_10_years[(SCD_converters_within_10_years[
        'Years_from_conversion'] < 0)]
54 SCD_converters_within_10_years =
    SCD_converters_within_10_years.drop_duplicates(subset=['RID',
        'Years_bl'], keep='first')
55
56
57 # Get patient records that are older than 10 years before the year of
    conversion
58 SCD_converters_after_10_years =
    SCD_converters[(SCD_converters['Years_from_conversion'].astype(float)
        <= -10.0)]
59
60
61 # patients records of patients not converted for 10 years and of
    converters 10 years before the year of conversion are considered as
    non-conversion data

```

```

62 SCD_non_converters = pd.concat([SCD_stay_10_years,
    SCD_converters_after_10_years]).sort_values(by=['RID', 'Years_bl'])
63 SCD_non_converters = SCD_non_converters.drop_duplicates(subset=['RID',
    'Years_bl'], keep='first')
64
65
66 # select necessary features
67 sel_columns = ['RID', 'Years_bl', 'AGE', 'PTGENDER', 'PTEDUCAT',
    'PTMARRY', 'APOE4', 'FDG', 'AV45', 'ABETA', 'TAU', 'PTAU',
    'PLASMA_NFL', 'PLASMA_PTAU',
68     'PLASMA_ABETA42/40', 'CDRSB', 'ADAS11', 'ADAS13', 'ADASQ4',
    'MMSE', 'RAVLT_immediate', 'RAVLT_learning',
    'RAVLT_forgetting',
69     'RAVLT_perc_forgetting', 'LDELTOTAL', 'DIGITSCOR', 'TRABSCOR',
    'FAQ', 'MOCA', 'EcogPtMem', 'EcogPtLang', 'EcogPtVisspat',
    'EcogPtPlan', 'EcogPtOrgan', 'EcogPtDivatt', 'EcogPtTotal',
    'EcogSPMem', 'EcogSPLang',
70     'EcogSPVisspat', 'EcogSPPlan', 'EcogSPOrgan', 'EcogSPDivatt',
    'EcogSPTotal', 'Ventricles', 'Hippocampus', 'WholeBrain',
    'Entorhinal', 'Fusiform', 'MidTemp', 'ICV', 'mPACCdigit',
    'mPACCtrailsB', 'MCI_AD_conversion'
71 ]
72
73
74 # Set y (conversion) column
75 SCD_converters_within_10_years['MCI_AD_conversion'] = 1
76 SCD_non_converters['MCI_AD_conversion'] = 0
77 SCD_converters_within_10_years =
    SCD_converters_within_10_years[sel_columns]
78 SCD_non_converters = SCD_non_converters[sel_columns]
79
80
81 # get only the baseline data
82 SCD_df = pd.concat([SCD_converters_within_10_years,
    SCD_non_converters]).sort_values(by=['RID', 'Years_bl'])
83 SCD_df_bl = SCD_df[(SCD_df['Years_bl'].astype(float)==0)]
84 SCD_df_bl = SCD_df_bl.drop(['RID', 'Years_bl'], axis=1)
85 SCD_df_bl.to_csv('./Third_Year_Project/SCD_bl.csv', sep=',', index=False,
    encoding='utf-8')

```

Listing C.1: Code for Constructing the 10-year conversion dataset

Appendix D

Code for Finding Multicollinear Features

```
1 # correlation matrix
2 corr_matrix = normalised_training_df.corr().abs()
3 corr_matrix.to_csv('./Third_Year_Project/corr_matrix.csv')
4
5 upper = corr_matrix.where(np.triu(np.ones(corr_matrix.shape),
6     k=1).astype(bool))
7 upper.to_csv('./Third_Year_Project/upper.csv')
8
9 high_corr_vars = [column for column in upper.columns if any(upper[column]
10     > 0.75)]
11 print(high_corr_vars)
12
13 # Checked upper.csv and found the high correlation variables and selected
14 # them manually (threshold > 0.75), rather than the first one selected as
15 # above in high_corr_vars
16 to_drop = ['TAU', 'ADAS11', 'ADASQ4', 'RAVLT_perc_forgetting',
17     'EcogPtPlan', 'EcogPtTotal', 'EcogSPLang', 'EcogSPOrgan',
18     'EcogSPDivatt',
19     'EcogSPTotal', 'mPACCtrailsB']
20
21 # drop high correlation features
22 df_reduced = normalised_training_df.drop(columns=to_drop)
23
24 # Calculate VIF for remaining features
25 vif_data = pd.DataFrame()
26 vif_data['feature'] = df_reduced.columns
27 vif_data['VIF'] = [variance_inflation_factor(df_reduced.values, i) for i
28     in range(df_reduced.shape[1])]
29
30 print(vif_data)
31
32 # remove features with VIF > 5 (ICC was not removed because when Whole
33 # Brain was removed, ICV had a VIF lower than 5)
34 df_reduced_2 = df_reduced.drop(columns=['mPACCdigit', 'WholeBrain'])
35 print(df_reduced_2.columns)
36
37 df_reduced_2.to_csv('./Third_Year_Project/reduced_df.csv', sep=',',
38     index=False, encoding='utf-8')
```

Listing D.1: Identifying Features exhibiting Multicollinearity

Appendix E

Selecting the Best Ensemble Model

```
1 def evaluate_ensemble_model(ensemble, training_data_sets, test_data_sets):
2     roc_auc_list = []
3     accuracy_list = []
4     for i in range(len(training_data_sets)):
5         X_train = training_data_sets[i].drop('MCI_AD_conversion', axis=1)
6         y_train = training_data_sets[i]['MCI_AD_conversion']
7
8         X_test = test_data_sets[i].drop('MCI_AD_conversion', axis=1)
9         y_test = test_data_sets[i]['MCI_AD_conversion']
10
11         # reproducibility
12         np.random.seed(42)
13
14         ensemble.fit(X_train, y_train)
15
16         # Evaluate the ensemble
17         y_pred_prob = ensemble.predict_proba(X_test)[:, 1]
18         y_pred = ensemble.predict(X_test)
19         roc_auc = roc_auc_score(y_test, y_pred_prob)
20         accuracy = accuracy_score(y_test, y_pred)
21
22         roc_auc_list.append(roc_auc)
23         accuracy_list.append(accuracy)
24
25     average_roc_auc = np.average(roc_auc_list)
26     average_accuracy = np.average(accuracy_list)
27
28     average_score = (average_roc_auc+average_accuracy)/2
29
30     return average_score
31
32
33
34 def get_best_ensemble_model(best_models, training_data_sets,
35                             test_data_sets):
36     best_ensemble = None
37     best_combination = None
38     best_method = None
39     best_score = 0 # combination of roc-auc and accuracy
40
41     # get best individual models from dictionary
```

```

41 best_lr_model = best_models['lr']
42 best_svc_model = best_models['svc']
43 best_knn_model = best_models['knn']
44 best_mlp_model = best_models['mlp']
45 best_dt_model = best_models['dt']
46 best_nb_model = best_models['nb']
47 best_rf_model = best_models['rf']
48 best_xgb_model = best_models['xgb']
49
50 # initialise pairs of model and its name for combinations creation
51 best_models_comb = [
52     ("lr", best_lr_model),
53     ("svc", best_svc_model),
54     ("knn", best_knn_model),
55     ("mlp", best_mlp_model),
56     ("dt", best_dt_model),
57     ("nb", best_nb_model),
58     ("rf", best_rf_model),
59     ("xgb", best_xgb_model),
60 ]
61
62 # use LR or RF model as final estimator for stacking ensemble model
63   combinations
64 final_estimators = [
65     ("lr", best_lr_model),
66     ("rf", best_rf_model)
67 ]
68
69 # Test different ensemble methods
70 for method in ["Voting", "Stacking", "Bagging"]:
71     if method != 'Bagging':
72         # Test all combinations of models
73         for size in range(2, len(best_models_comb) + 1):
74             for combination in combinations(best_models_comb, size):
75                 model_names = [name for name, _ in combination]
76                 model_instances = [(name, model) for name, model in combination]
77
78                 # print(f"Testing combination: {model_names}")
79
80                 if method == "Voting":
81                     ensemble = VotingClassifier(estimators=model_instances,
82                                                 voting="soft")
83
84                     average_score = evaluate_ensemble_model(ensemble,
85                                                             training_data_sets, test_data_sets)
86
87                     # print(f"ROC-AUC for Voting: {model_names}:
88                         {average_score:.4f}")
89
90                     # if score of the current voting ensemble model is better than
91                     # current best, update
92                     if average_score > best_score:
93                         best_score = average_score
94                         best_ensemble = ensemble
95                         best_combination = model_names
96                         best_method = method
97
98                 elif method == "Stacking":

```

```

94         for model_instance in final_estimators:
95             final_estimator = model_instance[1] # Use LR or RF as final
               estimator
96             ensemble = StackingClassifier(estimators=model_instances,
               final_estimator=final_estimator, cv=5) # Use all modeels
               as base estimators
97
98             average_score = evaluate_ensemble_model(ensemble,
               training_data_sets, test_data_sets)
99
100            # if score of the current stacking ensemble model is better
               than current best, update
101            if average_score > best_score:
102                best_score = average_score
103                best_ensemble = ensemble
104                best_combination = model_names
105                best_method = method
106
107            else:
108                for model_pair in best_models_comb:
109                    base_model = model_pair[1] # The model is the second value of the
               combination
110                    ensemble = BaggingClassifier(estimator=base_model,
               n_estimators=10, random_state=42)
111
112                    average_score = evaluate_ensemble_model(ensemble,
               training_data_sets, test_data_sets)
113
114                    # print(f"ROC-AUC for Bagging: {model_pair[0]}:
               {average_score:.4f}")
115
116                    # if score of the current bagging ensemble model is better than
               current best, update
117                    if average_score > best_score:
118                        best_score = average_score
119                        best_ensemble = ensemble
120                        best_combination = base_model
121                        best_method = method
122
123
124            return {
125                "best_ensemble": best_ensemble,
126                "best_combination": best_combination,
127                "best_method": best_method,
128                "best_score": best_score,
129            }

```

Listing E.1: Selecting the Best Ensemble Model

Appendix F

Ensemble Model Configurations

F.1 Stacking Model Configurations

Table F.1 lists the 494 distinct combinations of base models and final estimators used in the stacking classifier configurations.

Table F.1: All combinations of stacking base models with final estimators

Base Models	Final Estimator
LR, SVC	LR
LR, SVC	RF
LR, kNN	LR
LR, kNN	RF
LR, MLP	LR
LR, MLP	RF
LR, DT	LR
LR, DT	RF
LR, Gaussian NB	LR
LR, Gaussian NB	RF
LR, RF	LR
LR, RF	RF
LR, XGBoost	LR
LR, XGBoost	RF
SVC, kNN	LR
SVC, kNN	RF
SVC, MLP	LR

Continued on next page

Table F.1 (continued)

Base Models	Final Estimator
SVC, MLP	RF
SVC, DT	LR
SVC, DT	RF
SVC, Gaussian NB	LR
SVC, Gaussian NB	RF
SVC, RF	LR
SVC, RF	RF
SVC, XGBoost	LR
SVC, XGBoost	RF
kNN, MLP	LR
kNN, MLP	RF
kNN, DT	LR
kNN, DT	RF
kNN, Gaussian NB	LR
kNN, Gaussian NB	RF
kNN, RF	LR
kNN, RF	RF
kNN, XGBoost	LR
kNN, XGBoost	RF
MLP, DT	LR
MLP, DT	RF
MLP, Gaussian NB	LR
MLP, Gaussian NB	RF
MLP, RF	LR
MLP, RF	RF
MLP, XGBoost	LR
MLP, XGBoost	RF
DT, Gaussian NB	LR
DT, Gaussian NB	RF
DT, RF	LR
DT, RF	RF
DT, XGBoost	LR
DT, XGBoost	RF

Continued on next page

Table F.1 (continued)

Base Models	Final Estimator
Gaussian NB, RF	LR
Gaussian NB, RF	RF
Gaussian NB, XGBoost	LR
Gaussian NB, XGBoost	RF
RF, XGBoost	LR
RF, XGBoost	RF
LR, SVC, kNN	LR
LR, SVC, kNN	RF
LR, SVC, MLP	LR
LR, SVC, MLP	RF
LR, SVC, DT	LR
LR, SVC, DT	RF
LR, SVC, Gaussian NB	LR
LR, SVC, Gaussian NB	RF
LR, SVC, RF	LR
LR, SVC, RF	RF
LR, SVC, XGBoost	LR
LR, SVC, XGBoost	RF
LR, kNN, MLP	LR
LR, kNN, MLP	RF
LR, kNN, DT	LR
LR, kNN, DT	RF
LR, kNN, Gaussian NB	LR
LR, kNN, Gaussian NB	RF
LR, kNN, RF	LR
LR, kNN, RF	RF
LR, kNN, XGBoost	LR
LR, kNN, XGBoost	RF
LR, MLP, DT	LR
LR, MLP, DT	RF
LR, MLP, Gaussian NB	LR
LR, MLP, Gaussian NB	RF
LR, MLP, RF	LR

Continued on next page

Table F.1 (continued)

Base Models	Final Estimator
LR, MLP, RF	RF
LR, MLP, XGBoost	LR
LR, MLP, XGBoost	RF
LR, DT, Gaussian NB	LR
LR, DT, Gaussian NB	RF
LR, DT, RF	LR
LR, DT, RF	RF
LR, DT, XGBoost	LR
LR, DT, XGBoost	RF
LR, Gaussian NB, RF	LR
LR, Gaussian NB, RF	RF
LR, Gaussian NB, XGBoost	LR
LR, Gaussian NB, XGBoost	RF
LR, RF, XGBoost	LR
LR, RF, XGBoost	RF
SVC, kNN, MLP	LR
SVC, kNN, MLP	RF
SVC, kNN, DT	LR
SVC, kNN, DT	RF
SVC, kNN, Gaussian NB	LR
SVC, kNN, Gaussian NB	RF
SVC, kNN, RF	LR
SVC, kNN, RF	RF
SVC, kNN, XGBoost	LR
SVC, kNN, XGBoost	RF
SVC, MLP, DT	LR
SVC, MLP, DT	RF
SVC, MLP, Gaussian NB	LR
SVC, MLP, Gaussian NB	RF
SVC, MLP, RF	LR
SVC, MLP, RF	RF
SVC, MLP, XGBoost	LR
SVC, MLP, XGBoost	RF

Continued on next page

Table F.1 (continued)

Base Models	Final Estimator
SVC, DT, Gaussian NB	LR
SVC, DT, Gaussian NB	RF
SVC, DT, RF	LR
SVC, DT, RF	RF
SVC, DT, XGBoost	LR
SVC, DT, XGBoost	RF
SVC, Gaussian NB, RF	LR
SVC, Gaussian NB, RF	RF
SVC, Gaussian NB, XGBoost	LR
SVC, Gaussian NB, XGBoost	RF
SVC, RF, XGBoost	LR
SVC, RF, XGBoost	RF
kNN, MLP, DT	LR
kNN, MLP, DT	RF
kNN, MLP, Gaussian NB	LR
kNN, MLP, Gaussian NB	RF
kNN, MLP, RF	LR
kNN, MLP, RF	RF
kNN, MLP, XGBoost	LR
kNN, MLP, XGBoost	RF
kNN, DT, Gaussian NB	LR
kNN, DT, Gaussian NB	RF
kNN, DT, RF	LR
kNN, DT, RF	RF
kNN, DT, XGBoost	LR
kNN, DT, XGBoost	RF
kNN, Gaussian NB, RF	LR
kNN, Gaussian NB, RF	RF
kNN, Gaussian NB, XGBoost	LR
kNN, Gaussian NB, XGBoost	RF
kNN, RF, XGBoost	LR
kNN, RF, XGBoost	RF
MLP, DT, Gaussian NB	LR

Continued on next page

Table F.1 (continued)

Base Models	Final Estimator
MLP, DT, Gaussian NB	RF
MLP, DT, RF	LR
MLP, DT, RF	RF
MLP, DT, XGBoost	LR
MLP, DT, XGBoost	RF
MLP, Gaussian NB, RF	LR
MLP, Gaussian NB, RF	RF
MLP, Gaussian NB, XGBoost	LR
MLP, Gaussian NB, XGBoost	RF
MLP, RF, XGBoost	LR
MLP, RF, XGBoost	RF
DT, Gaussian NB, RF	LR
DT, Gaussian NB, RF	RF
DT, Gaussian NB, XGBoost	LR
DT, Gaussian NB, XGBoost	RF
DT, RF, XGBoost	LR
DT, RF, XGBoost	RF
Gaussian NB, RF, XGBoost	LR
Gaussian NB, RF, XGBoost	RF
LR, SVC, kNN, MLP	LR
LR, SVC, kNN, MLP	RF
LR, SVC, kNN, DT	LR
LR, SVC, kNN, DT	RF
LR, SVC, kNN, Gaussian NB	LR
LR, SVC, kNN, Gaussian NB	RF
LR, SVC, kNN, RF	LR
LR, SVC, kNN, RF	RF
LR, SVC, kNN, XGBoost	LR
LR, SVC, kNN, XGBoost	RF
LR, SVC, MLP, DT	LR
LR, SVC, MLP, DT	RF
LR, SVC, MLP, Gaussian NB	LR
LR, SVC, MLP, Gaussian NB	RF

Continued on next page

Table F.1 (continued)

Base Models	Final Estimator
LR, SVC, MLP, RF	LR
LR, SVC, MLP, RF	RF
LR, SVC, MLP, XGBoost	LR
LR, SVC, MLP, XGBoost	RF
LR, SVC, DT, Gaussian NB	LR
LR, SVC, DT, Gaussian NB	RF
LR, SVC, DT, RF	LR
LR, SVC, DT, RF	RF
LR, SVC, DT, XGBoost	LR
LR, SVC, DT, XGBoost	RF
LR, SVC, Gaussian NB, RF	LR
LR, SVC, Gaussian NB, RF	RF
LR, SVC, Gaussian NB, XGBoost	LR
LR, SVC, Gaussian NB, XGBoost	RF
LR, SVC, RF, XGBoost	LR
LR, SVC, RF, XGBoost	RF
LR, kNN, MLP, DT	LR
LR, kNN, MLP, DT	RF
LR, kNN, MLP, Gaussian NB	LR
LR, kNN, MLP, Gaussian NB	RF
LR, kNN, MLP, RF	LR
LR, kNN, MLP, RF	RF
LR, kNN, MLP, XGBoost	LR
LR, kNN, MLP, XGBoost	RF
LR, kNN, DT, Gaussian NB	LR
LR, kNN, DT, Gaussian NB	RF
LR, kNN, DT, RF	LR
LR, kNN, DT, RF	RF
LR, kNN, DT, XGBoost	LR
LR, kNN, DT, XGBoost	RF
LR, kNN, Gaussian NB, RF	LR
LR, kNN, Gaussian NB, RF	RF
LR, kNN, Gaussian NB, XGBoost	LR

Continued on next page

Table F.1 (continued)

Base Models	Final Estimator
LR, kNN, Gaussian NB, XGBoost	RF
LR, kNN, RF, XGBoost	LR
LR, kNN, RF, XGBoost	RF
LR, MLP, DT, Gaussian NB	LR
LR, MLP, DT, Gaussian NB	RF
LR, MLP, DT, RF	LR
LR, MLP, DT, RF	RF
LR, MLP, DT, XGBoost	LR
LR, MLP, DT, XGBoost	RF
LR, MLP, Gaussian NB, RF	LR
LR, MLP, Gaussian NB, RF	RF
LR, MLP, Gaussian NB, XGBoost	LR
LR, MLP, Gaussian NB, XGBoost	RF
LR, MLP, RF, XGBoost	LR
LR, MLP, RF, XGBoost	RF
LR, DT, Gaussian NB, RF	LR
LR, DT, Gaussian NB, RF	RF
LR, DT, Gaussian NB, XGBoost	LR
LR, DT, Gaussian NB, XGBoost	RF
LR, DT, RF, XGBoost	LR
LR, DT, RF, XGBoost	RF
LR, Gaussian NB, RF, XGBoost	LR
LR, Gaussian NB, RF, XGBoost	RF
SVC, kNN, MLP, DT	LR
SVC, kNN, MLP, DT	RF
SVC, kNN, MLP, Gaussian NB	LR
SVC, kNN, MLP, Gaussian NB	RF
SVC, kNN, MLP, RF	LR
SVC, kNN, MLP, RF	RF
SVC, kNN, MLP, XGBoost	LR
SVC, kNN, MLP, XGBoost	RF
SVC, kNN, DT, Gaussian NB	LR
SVC, kNN, DT, Gaussian NB	RF

Continued on next page

Table F.1 (continued)

Base Models	Final Estimator
SVC, kNN, DT, RF	LR
SVC, kNN, DT, RF	RF
SVC, kNN, DT, XGBoost	LR
SVC, kNN, DT, XGBoost	RF
SVC, kNN, Gaussian NB, RF	LR
SVC, kNN, Gaussian NB, RF	RF
SVC, kNN, Gaussian NB, XGBoost	LR
SVC, kNN, Gaussian NB, XGBoost	RF
SVC, kNN, RF, XGBoost	LR
SVC, kNN, RF, XGBoost	RF
SVC, MLP, DT, Gaussian NB	LR
SVC, MLP, DT, Gaussian NB	RF
SVC, MLP, DT, RF	LR
SVC, MLP, DT, RF	RF
SVC, MLP, DT, XGBoost	LR
SVC, MLP, DT, XGBoost	RF
SVC, MLP, Gaussian NB, RF	LR
SVC, MLP, Gaussian NB, RF	RF
SVC, MLP, Gaussian NB, XGBoost	LR
SVC, MLP, Gaussian NB, XGBoost	RF
SVC, MLP, RF, XGBoost	LR
SVC, MLP, RF, XGBoost	RF
SVC, DT, Gaussian NB, RF	LR
SVC, DT, Gaussian NB, RF	RF
SVC, DT, Gaussian NB, XGBoost	LR
SVC, DT, Gaussian NB, XGBoost	RF
SVC, DT, RF, XGBoost	LR
SVC, DT, RF, XGBoost	RF
SVC, Gaussian NB, RF, XGBoost	LR
SVC, Gaussian NB, RF, XGBoost	RF
kNN, MLP, DT, Gaussian NB	LR
kNN, MLP, DT, Gaussian NB	RF
kNN, MLP, DT, RF	LR

Continued on next page

Table F.1 (continued)

Base Models	Final Estimator
kNN, MLP, DT, RF	RF
kNN, MLP, DT, XGBoost	LR
kNN, MLP, DT, XGBoost	RF
kNN, MLP, Gaussian NB, RF	LR
kNN, MLP, Gaussian NB, RF	RF
kNN, MLP, Gaussian NB, XGBoost	LR
kNN, MLP, Gaussian NB, XGBoost	RF
kNN, MLP, RF, XGBoost	LR
kNN, MLP, RF, XGBoost	RF
kNN, DT, Gaussian NB, RF	LR
kNN, DT, Gaussian NB, RF	RF
kNN, DT, Gaussian NB, XGBoost	LR
kNN, DT, Gaussian NB, XGBoost	RF
kNN, DT, RF, XGBoost	LR
kNN, DT, RF, XGBoost	RF
kNN, Gaussian NB, RF, XGBoost	LR
kNN, Gaussian NB, RF, XGBoost	RF
MLP, DT, Gaussian NB, RF	LR
MLP, DT, Gaussian NB, RF	RF
MLP, DT, Gaussian NB, XGBoost	LR
MLP, DT, Gaussian NB, XGBoost	RF
MLP, DT, RF, XGBoost	LR
MLP, DT, RF, XGBoost	RF
MLP, Gaussian NB, RF, XGBoost	LR
MLP, Gaussian NB, RF, XGBoost	RF
DT, Gaussian NB, RF, XGBoost	LR
DT, Gaussian NB, RF, XGBoost	RF
LR, SVC, kNN, MLP, DT	LR
LR, SVC, kNN, MLP, DT	RF
LR, SVC, kNN, MLP, Gaussian NB	LR
LR, SVC, kNN, MLP, Gaussian NB	RF
LR, SVC, kNN, MLP, RF	LR
LR, SVC, kNN, MLP, RF	RF

Continued on next page

Table F.1 (continued)

Base Models	Final Estimator
LR, SVC, kNN, MLP, XGBoost	LR
LR, SVC, kNN, MLP, XGBoost	RF
LR, SVC, kNN, DT, Gaussian NB	LR
LR, SVC, kNN, DT, Gaussian NB	RF
LR, SVC, kNN, DT, RF	LR
LR, SVC, kNN, DT, RF	RF
LR, SVC, kNN, DT, XGBoost	LR
LR, SVC, kNN, DT, XGBoost	RF
LR, SVC, kNN, Gaussian NB, RF	LR
LR, SVC, kNN, Gaussian NB, RF	RF
LR, SVC, kNN, Gaussian NB, XGBoost	LR
LR, SVC, kNN, Gaussian NB, XGBoost	RF
LR, SVC, kNN, RF, XGBoost	LR
LR, SVC, kNN, RF, XGBoost	RF
LR, SVC, MLP, DT, Gaussian NB	LR
LR, SVC, MLP, DT, Gaussian NB	RF
LR, SVC, MLP, DT, RF	LR
LR, SVC, MLP, DT, RF	RF
LR, SVC, MLP, DT, XGBoost	LR
LR, SVC, MLP, DT, XGBoost	RF
LR, SVC, MLP, Gaussian NB, RF	LR
LR, SVC, MLP, Gaussian NB, RF	RF
LR, SVC, MLP, Gaussian NB, XGBoost	LR
LR, SVC, MLP, Gaussian NB, XGBoost	RF
LR, SVC, MLP, RF, XGBoost	LR
LR, SVC, MLP, RF, XGBoost	RF
LR, SVC, DT, Gaussian NB, RF	LR
LR, SVC, DT, Gaussian NB, RF	RF
LR, SVC, DT, Gaussian NB, XGBoost	LR
LR, SVC, DT, Gaussian NB, XGBoost	RF
LR, SVC, DT, RF, XGBoost	LR
LR, SVC, DT, RF, XGBoost	RF
LR, SVC, Gaussian NB, RF, XGBoost	LR

Continued on next page

Table F.1 (continued)

Base Models	Final Estimator
LR, SVC, Gaussian NB, RF, XGBoost	RF
LR, kNN, MLP, DT, Gaussian NB	LR
LR, kNN, MLP, DT, Gaussian NB	RF
LR, kNN, MLP, DT, RF	LR
LR, kNN, MLP, DT, RF	RF
LR, kNN, MLP, DT, XGBoost	LR
LR, kNN, MLP, DT, XGBoost	RF
LR, kNN, MLP, Gaussian NB, RF	LR
LR, kNN, MLP, Gaussian NB, RF	RF
LR, kNN, MLP, Gaussian NB, XGBoost	LR
LR, kNN, MLP, Gaussian NB, XGBoost	RF
LR, kNN, MLP, RF, XGBoost	LR
LR, kNN, MLP, RF, XGBoost	RF
LR, kNN, DT, Gaussian NB, RF	LR
LR, kNN, DT, Gaussian NB, RF	RF
LR, kNN, DT, Gaussian NB, XGBoost	LR
LR, kNN, DT, Gaussian NB, XGBoost	RF
LR, kNN, DT, RF, XGBoost	LR
LR, kNN, DT, RF, XGBoost	RF
LR, kNN, Gaussian NB, RF, XGBoost	LR
LR, kNN, Gaussian NB, RF, XGBoost	RF
LR, MLP, DT, Gaussian NB, RF	LR
LR, MLP, DT, Gaussian NB, RF	RF
LR, MLP, DT, Gaussian NB, XGBoost	LR
LR, MLP, DT, Gaussian NB, XGBoost	RF
LR, MLP, DT, RF, XGBoost	LR
LR, MLP, DT, RF, XGBoost	RF
LR, MLP, Gaussian NB, RF, XGBoost	LR
LR, MLP, Gaussian NB, RF, XGBoost	RF
LR, DT, Gaussian NB, RF, XGBoost	LR
LR, DT, Gaussian NB, RF, XGBoost	RF
SVC, kNN, MLP, DT, Gaussian NB	LR
SVC, kNN, MLP, DT, Gaussian NB	RF

Continued on next page

Table F.1 (continued)

Base Models	Final Estimator
SVC, kNN, MLP, DT, RF	LR
SVC, kNN, MLP, DT, RF	RF
SVC, kNN, MLP, DT, XGBoost	LR
SVC, kNN, MLP, DT, XGBoost	RF
SVC, kNN, MLP, Gaussian NB, RF	LR
SVC, kNN, MLP, Gaussian NB, RF	RF
SVC, kNN, MLP, Gaussian NB, XGBoost	LR
SVC, kNN, MLP, Gaussian NB, XGBoost	RF
SVC, kNN, MLP, RF, XGBoost	LR
SVC, kNN, MLP, RF, XGBoost	RF
SVC, kNN, DT, Gaussian NB, RF	LR
SVC, kNN, DT, Gaussian NB, RF	RF
SVC, kNN, DT, Gaussian NB, XGBoost	LR
SVC, kNN, DT, Gaussian NB, XGBoost	RF
SVC, kNN, DT, RF, XGBoost	LR
SVC, kNN, DT, RF, XGBoost	RF
SVC, kNN, Gaussian NB, RF, XGBoost	LR
SVC, kNN, Gaussian NB, RF, XGBoost	RF
SVC, MLP, DT, Gaussian NB, RF	LR
SVC, MLP, DT, Gaussian NB, RF	RF
SVC, MLP, DT, Gaussian NB, XGBoost	LR
SVC, MLP, DT, Gaussian NB, XGBoost	RF
SVC, MLP, DT, RF, XGBoost	LR
SVC, MLP, DT, RF, XGBoost	RF
SVC, MLP, Gaussian NB, RF, XGBoost	LR
SVC, MLP, Gaussian NB, RF, XGBoost	RF
SVC, DT, Gaussian NB, RF, XGBoost	LR
SVC, DT, Gaussian NB, RF, XGBoost	RF
kNN, MLP, DT, Gaussian NB, RF	LR
kNN, MLP, DT, Gaussian NB, RF	RF
kNN, MLP, DT, Gaussian NB, XGBoost	LR
kNN, MLP, DT, Gaussian NB, XGBoost	RF
kNN, MLP, DT, RF, XGBoost	LR

Continued on next page

Table F.1 (continued)

Base Models	Final Estimator
kNN, MLP, DT, RF, XGBoost	RF
kNN, MLP, Gaussian NB, RF, XGBoost	LR
kNN, MLP, Gaussian NB, RF, XGBoost	RF
kNN, DT, Gaussian NB, RF, XGBoost	LR
kNN, DT, Gaussian NB, RF, XGBoost	RF
MLP, DT, Gaussian NB, RF, XGBoost	LR
MLP, DT, Gaussian NB, RF, XGBoost	RF
LR, SVC, kNN, MLP, DT, Gaussian NB	LR
LR, SVC, kNN, MLP, DT, Gaussian NB	RF
LR, SVC, kNN, MLP, DT, RF	LR
LR, SVC, kNN, MLP, DT, RF	RF
LR, SVC, kNN, MLP, DT, XGBoost	LR
LR, SVC, kNN, MLP, DT, XGBoost	RF
LR, SVC, kNN, MLP, Gaussian NB, RF	LR
LR, SVC, kNN, MLP, Gaussian NB, RF	RF
LR, SVC, kNN, MLP, Gaussian NB, XGBoost	LR
LR, SVC, kNN, MLP, Gaussian NB, XGBoost	RF
LR, SVC, kNN, MLP, RF, XGBoost	LR
LR, SVC, kNN, MLP, RF, XGBoost	RF
LR, SVC, kNN, DT, Gaussian NB, RF	LR
LR, SVC, kNN, DT, Gaussian NB, RF	RF
LR, SVC, kNN, DT, Gaussian NB, XGBoost	LR
LR, SVC, kNN, DT, Gaussian NB, XGBoost	RF
LR, SVC, kNN, DT, RF, XGBoost	LR
LR, SVC, kNN, DT, RF, XGBoost	RF
LR, SVC, kNN, Gaussian NB, RF, XGBoost	LR
LR, SVC, kNN, Gaussian NB, RF, XGBoost	RF
LR, SVC, MLP, DT, Gaussian NB, RF	LR
LR, SVC, MLP, DT, Gaussian NB, RF	RF
LR, SVC, MLP, DT, Gaussian NB, XGBoost	LR
LR, SVC, MLP, DT, Gaussian NB, XGBoost	RF
LR, SVC, MLP, DT, RF, XGBoost	LR
LR, SVC, MLP, DT, RF, XGBoost	RF

Continued on next page

Table F.1 (continued)

Base Models	Final Estimator
LR, SVC, MLP, Gaussian NB, RF, XGBoost	LR
LR, SVC, MLP, Gaussian NB, RF, XGBoost	RF
LR, SVC, DT, Gaussian NB, RF, XGBoost	LR
LR, SVC, DT, Gaussian NB, RF, XGBoost	RF
LR, kNN, MLP, DT, Gaussian NB, RF	LR
LR, kNN, MLP, DT, Gaussian NB, RF	RF
LR, kNN, MLP, DT, Gaussian NB, XGBoost	LR
LR, kNN, MLP, DT, Gaussian NB, XGBoost	RF
LR, kNN, MLP, DT, RF, XGBoost	LR
LR, kNN, MLP, DT, RF, XGBoost	RF
LR, kNN, MLP, Gaussian NB, RF, XGBoost	LR
LR, kNN, MLP, Gaussian NB, RF, XGBoost	RF
LR, kNN, DT, Gaussian NB, RF, XGBoost	LR
LR, kNN, DT, Gaussian NB, RF, XGBoost	RF
LR, MLP, DT, Gaussian NB, RF, XGBoost	LR
LR, MLP, DT, Gaussian NB, RF, XGBoost	RF
LR, MLP, DT, Gaussian NB, RF, XGBoost	LR
LR, MLP, DT, Gaussian NB, RF, XGBoost	RF
SVC, kNN, MLP, DT, Gaussian NB, RF	LR
SVC, kNN, MLP, DT, Gaussian NB, RF	RF
SVC, kNN, MLP, DT, Gaussian NB, XGBoost	LR
SVC, kNN, MLP, DT, Gaussian NB, XGBoost	RF
SVC, kNN, MLP, DT, RF, XGBoost	LR
SVC, kNN, MLP, DT, RF, XGBoost	RF
SVC, kNN, MLP, Gaussian NB, RF, XGBoost	LR
SVC, kNN, MLP, Gaussian NB, RF, XGBoost	RF
SVC, kNN, DT, Gaussian NB, RF, XGBoost	LR
SVC, kNN, DT, Gaussian NB, RF, XGBoost	RF
kNN, MLP, DT, Gaussian NB, RF, XGBoost	LR
kNN, MLP, DT, Gaussian NB, RF, XGBoost	RF
LR, SVC, kNN, MLP, DT, Gaussian NB, RF	LR
LR, SVC, kNN, MLP, DT, Gaussian NB, RF	RF
LR, SVC, kNN, MLP, DT, Gaussian NB, XGBoost	LR

Continued on next page

Table F.1 (continued)

Base Models	Final Estimator
LR, SVC, kNN, MLP, DT, Gaussian NB, XGBoost	RF
LR, SVC, kNN, MLP, DT, RF, XGBoost	LR
LR, SVC, kNN, MLP, DT, RF, XGBoost	RF
LR, SVC, kNN, MLP, Gaussian NB, RF, XGBoost	LR
LR, SVC, kNN, MLP, Gaussian NB, RF, XGBoost	RF
LR, SVC, kNN, DT, Gaussian NB, RF, XGBoost	LR
LR, SVC, kNN, DT, Gaussian NB, RF, XGBoost	RF
LR, SVC, MLP, DT, Gaussian NB, RF, XGBoost	LR
LR, SVC, MLP, DT, Gaussian NB, RF, XGBoost	RF
LR, SVC, DT, Gaussian NB, RF, XGBoost	LR
LR, SVC, DT, Gaussian NB, RF, XGBoost	RF
LR, kNN, MLP, DT, Gaussian NB, RF, XGBoost	LR
LR, kNN, MLP, DT, Gaussian NB, RF, XGBoost	RF
SVC, kNN, MLP, DT, Gaussian NB, RF, XGBoost	LR
SVC, kNN, MLP, DT, Gaussian NB, RF, XGBoost	RF
LR, SVC, kNN, MLP, DT, Gaussian NB, RF, XGBoost	LR
LR, SVC, kNN, MLP, DT, Gaussian NB, RF, XGBoost	RF

F.2 Voting Model Configurations

Table F.2 lists the 247 distinct combinations of base models used in the voting classifier configurations.

Table F.2: All combinations of voting base models

Base Models
LR, SVC
LR, kNN
LR, MLP
LR, DT
LR, Gaussian NB
LR, RF
LR, XGBoost
<i>Continued on next page</i>

Table F.2 (continued)

Base Models
SVC, kNN
SVC, MLP
SVC, DT
SVC, Gaussian NB
SVC, RF
SVC, XGBoost
kNN, MLP
kNN, DT
kNN, Gaussian NB
kNN, RF
kNN, XGBoost
MLP, DT
MLP, Gaussian NB
MLP, RF
MLP, XGBoost
DT, Gaussian NB
DT, RF
DT, XGBoost
Gaussian NB, RF
Gaussian NB, XGBoost
RF, XGBoost
LR, SVC, kNN
LR, SVC, MLP
LR, SVC, DT
LR, SVC, Gaussian NB
LR, SVC, RF
LR, SVC, XGBoost
LR, kNN, MLP
LR, kNN, DT
LR, kNN, Gaussian NB
LR, kNN, RF
LR, kNN, XGBoost
LR, MLP, DT

Continued on next page

Table F.2 (continued)

Base Models
LR, MLP, Gaussian NB
LR, MLP, RF
LR, MLP, XGBoost
LR, DT, Gaussian NB
LR, DT, RF
LR, DT, XGBoost
LR, Gaussian NB, RF
LR, Gaussian NB, XGBoost
LR, RF, XGBoost
SVC, kNN, MLP
SVC, kNN, DT
SVC, kNN, Gaussian NB
SVC, kNN, RF
SVC, kNN, XGBoost
SVC, MLP, DT
SVC, MLP, Gaussian NB
SVC, MLP, RF
SVC, MLP, XGBoost
SVC, DT, Gaussian NB
SVC, DT, RF
SVC, DT, XGBoost
SVC, Gaussian NB, RF
SVC, Gaussian NB, XGBoost
SVC, RF, XGBoost
kNN, MLP, DT
kNN, MLP, Gaussian NB
kNN, MLP, RF
kNN, MLP, XGBoost
kNN, DT, Gaussian NB
kNN, DT, RF
kNN, DT, XGBoost
kNN, Gaussian NB, RF
kNN, Gaussian NB, XGBoost

Continued on next page

Table F.2 (continued)

Base Models
kNN, RF, XGBoost
MLP, DT, Gaussian NB
MLP, DT, RF
MLP, DT, XGBoost
MLP, Gaussian NB, RF
MLP, Gaussian NB, XGBoost
MLP, RF, XGBoost
DT, Gaussian NB, RF
DT, Gaussian NB, XGBoost
DT, RF, XGBoost
Gaussian NB, RF, XGBoost
LR, SVC, kNN, MLP
LR, SVC, kNN, DT
LR, SVC, kNN, Gaussian NB
LR, SVC, kNN, RF
LR, SVC, kNN, XGBoost
LR, SVC, MLP, DT
LR, SVC, MLP, Gaussian NB
LR, SVC, MLP, RF
LR, SVC, MLP, XGBoost
LR, SVC, DT, Gaussian NB
LR, SVC, DT, RF
LR, SVC, DT, XGBoost
LR, SVC, Gaussian NB, RF
LR, SVC, Gaussian NB, XGBoost
LR, SVC, RF, XGBoost
LR, kNN, MLP, DT
LR, kNN, MLP, Gaussian NB
LR, kNN, MLP, RF
LR, kNN, MLP, XGBoost
LR, kNN, DT, Gaussian NB
LR, kNN, DT, RF
LR, kNN, DT, XGBoost

Continued on next page

Table F.2 (continued)

Base Models
LR, kNN, Gaussian NB, RF
LR, kNN, Gaussian NB, XGBoost
LR, kNN, RF, XGBoost
LR, MLP, DT, Gaussian NB
LR, MLP, DT, RF
LR, MLP, DT, XGBoost
LR, MLP, Gaussian NB, RF
LR, MLP, Gaussian NB, XGBoost
LR, MLP, RF, XGBoost
LR, DT, Gaussian NB, RF
LR, DT, Gaussian NB, XGBoost
LR, DT, RF, XGBoost
LR, Gaussian NB, RF, XGBoost
SVC, kNN, MLP, DT
SVC, kNN, MLP, Gaussian NB
SVC, kNN, MLP, RF
SVC, kNN, MLP, XGBoost
SVC, kNN, DT, Gaussian NB
SVC, kNN, DT, RF
SVC, kNN, DT, XGBoost
SVC, kNN, Gaussian NB, RF
SVC, kNN, Gaussian NB, XGBoost
SVC, kNN, RF, XGBoost
SVC, MLP, DT, Gaussian NB
SVC, MLP, DT, RF
SVC, MLP, DT, XGBoost
SVC, MLP, Gaussian NB, RF
SVC, MLP, Gaussian NB, XGBoost
SVC, MLP, RF, XGBoost
SVC, DT, Gaussian NB, RF
SVC, DT, Gaussian NB, XGBoost
SVC, DT, RF, XGBoost
SVC, Gaussian NB, RF, XGBoost

Continued on next page

Table F.2 (continued)

Base Models
kNN, MLP, DT, Gaussian NB
kNN, MLP, DT, RF
kNN, MLP, DT, XGBoost
kNN, MLP, Gaussian NB, RF
kNN, MLP, Gaussian NB, XGBoost
kNN, MLP, RF, XGBoost
kNN, DT, Gaussian NB, RF
kNN, DT, Gaussian NB, XGBoost
kNN, DT, RF, XGBoost
kNN, Gaussian NB, RF, XGBoost
MLP, DT, Gaussian NB, RF
MLP, DT, Gaussian NB, XGBoost
MLP, DT, RF, XGBoost
MLP, Gaussian NB, RF, XGBoost
DT, Gaussian NB, RF, XGBoost
LR, SVC, kNN, MLP, DT
LR, SVC, kNN, MLP, Gaussian NB
LR, SVC, kNN, MLP, RF
LR, SVC, kNN, MLP, XGBoost
LR, SVC, kNN, DT, Gaussian NB
LR, SVC, kNN, DT, RF
LR, SVC, kNN, DT, XGBoost
LR, SVC, kNN, Gaussian NB, RF
LR, SVC, kNN, Gaussian NB, XGBoost
LR, SVC, kNN, RF, XGBoost
LR, SVC, MLP, DT, Gaussian NB
LR, SVC, MLP, DT, RF
LR, SVC, MLP, DT, XGBoost
LR, SVC, MLP, Gaussian NB, RF
LR, SVC, MLP, Gaussian NB, XGBoost
LR, SVC, MLP, RF, XGBoost
LR, SVC, DT, Gaussian NB, RF
LR, SVC, DT, Gaussian NB, XGBoost

Continued on next page

Table F.2 (continued)

Base Models
LR, SVC, DT, RF, XGBoost
LR, SVC, Gaussian NB, RF, XGBoost
LR, kNN, MLP, DT, Gaussian NB
LR, kNN, MLP, DT, RF
LR, kNN, MLP, DT, XGBoost
LR, kNN, MLP, Gaussian NB, RF
LR, kNN, MLP, Gaussian NB, XGBoost
LR, kNN, MLP, RF, XGBoost
LR, kNN, DT, Gaussian NB, RF
LR, kNN, DT, Gaussian NB, XGBoost
LR, kNN, DT, RF, XGBoost
LR, kNN, Gaussian NB, RF, XGBoost
LR, MLP, DT, Gaussian NB, RF
LR, MLP, DT, Gaussian NB, XGBoost
LR, MLP, DT, RF, XGBoost
LR, MLP, Gaussian NB, RF, XGBoost
LR, DT, Gaussian NB, RF, XGBoost
SVC, kNN, MLP, DT, Gaussian NB
SVC, kNN, MLP, DT, RF
SVC, kNN, MLP, DT, XGBoost
SVC, kNN, MLP, Gaussian NB, RF
SVC, kNN, MLP, Gaussian NB, XGBoost
SVC, kNN, MLP, RF, XGBoost
SVC, kNN, DT, Gaussian NB, RF
SVC, kNN, DT, Gaussian NB, XGBoost
SVC, kNN, DT, RF, XGBoost
SVC, kNN, Gaussian NB, RF, XGBoost
SVC, MLP, DT, Gaussian NB, RF
SVC, MLP, DT, Gaussian NB, XGBoost
SVC, MLP, DT, RF, XGBoost
SVC, MLP, Gaussian NB, RF, XGBoost
SVC, DT, Gaussian NB, RF, XGBoost
kNN, MLP, DT, Gaussian NB, RF

Continued on next page

Table F.2 (continued)

Base Models
kNN, MLP, DT, Gaussian NB, XGBoost
kNN, MLP, DT, RF, XGBoost
kNN, MLP, Gaussian NB, RF, XGBoost
kNN, DT, Gaussian NB, RF, XGBoost
MLP, DT, Gaussian NB, RF, XGBoost
LR, SVC, kNN, MLP, DT, Gaussian NB
LR, SVC, kNN, MLP, DT, RF
LR, SVC, kNN, MLP, DT, XGBoost
LR, SVC, kNN, MLP, Gaussian NB, RF
LR, SVC, kNN, MLP, Gaussian NB, XGBoost
LR, SVC, kNN, MLP, RF, XGBoost
LR, SVC, kNN, DT, Gaussian NB, RF
LR, SVC, kNN, DT, Gaussian NB, XGBoost
LR, SVC, kNN, DT, RF, XGBoost
LR, SVC, kNN, Gaussian NB, RF, XGBoost
LR, SVC, MLP, DT, Gaussian NB, RF
LR, SVC, MLP, DT, Gaussian NB, XGBoost
LR, SVC, MLP, DT, RF, XGBoost
LR, SVC, MLP, Gaussian NB, RF, XGBoost
LR, SVC, DT, Gaussian NB, RF, XGBoost
LR, kNN, MLP, DT, Gaussian NB, RF
LR, kNN, MLP, DT, Gaussian NB, XGBoost
LR, kNN, MLP, DT, RF, XGBoost
LR, kNN, MLP, Gaussian NB, RF, XGBoost
LR, kNN, DT, Gaussian NB, RF, XGBoost
LR, MLP, DT, Gaussian NB, RF, XGBoost
LR, SVC, kNN, MLP, DT, Gaussian NB
SVC, kNN, MLP, DT, Gaussian NB, RF
SVC, kNN, MLP, DT, Gaussian NB, XGBoost
SVC, kNN, MLP, DT, RF, XGBoost
SVC, kNN, MLP, Gaussian NB, RF, XGBoost
SVC, kNN, DT, Gaussian NB, RF, XGBoost
kNN, MLP, DT, Gaussian NB, RF, XGBoost

Continued on next page

Table F.2 (continued)

Base Models
LR, SVC, kNN, MLP, DT, Gaussian NB, RF
LR, SVC, kNN, MLP, DT, Gaussian NB, XGBoost
LR, SVC, kNN, MLP, DT, RF, XGBoost
LR, SVC, kNN, MLP, Gaussian NB, RF, XGBoost
LR, SVC, kNN, DT, Gaussian NB, RF, XGBoost
LR, SVC, MLP, DT, Gaussian NB, RF, XGBoost
LR, kNN, MLP, DT, Gaussian NB, RF, XGBoost
SVC, kNN, MLP, DT, Gaussian NB, RF, XGBoost
LR, SVC, kNN, MLP, DT, Gaussian NB, RF, XGBoost

F.3 Bagging Model Configurations

Table F.2 lists the 8 distinct base models used in the bagging classifier configurations.

Table F.3: All base models in bagging configurations

Base Model
LR
kNN
SVC
MLP
DT
Gaussian NB
RF
XGBoost

Appendix G

Best Ensemble Models

G.1 Best Ensemble Models of Imputed Datasets

G.1.1 Multiple Imputations by Chained Equations (MICE) with Bayesian Ridge Estimator

The best ensemble model selected for the dataset imputed using the MICE algorithm is a voting classifier that combines Logistic Regression, Support Vector Classifier, Decision Tree, Gaussian Naive Bayes, and Random Forest as base models (Figure G.1). Table G.1 presents the optimal hyperparameters for each individual model within the ensemble model.

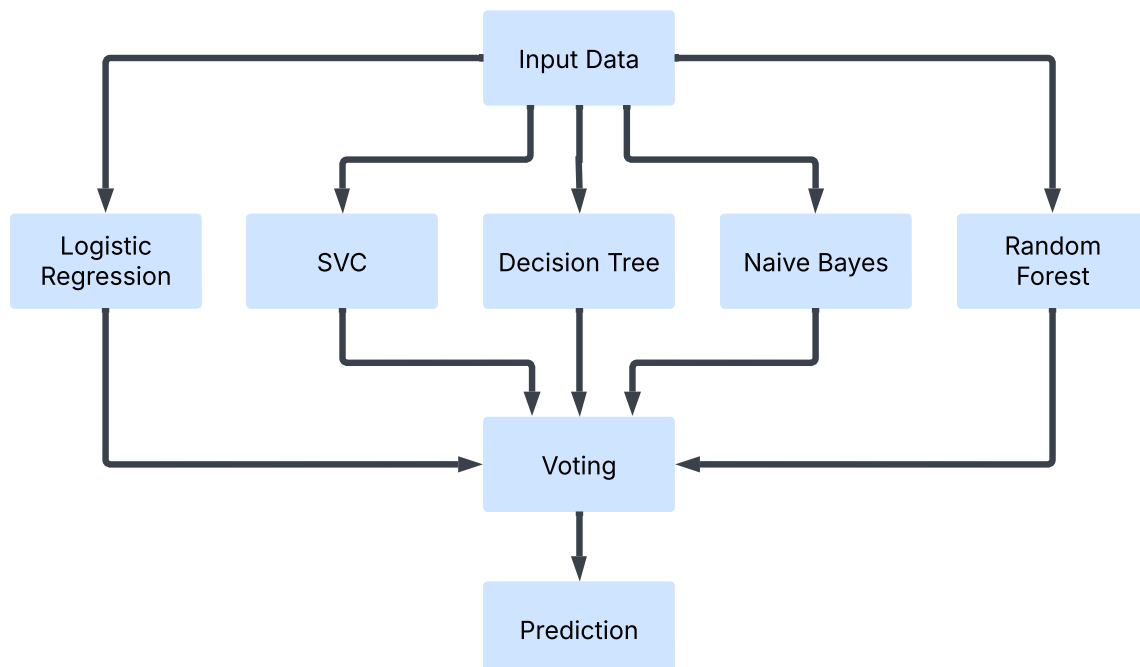


Figure G.1: Best Ensemble Model for MICE Imputation: Voting

Table G.1: Optimal hyperparameters of Individual Models Models within Ensemble Model selected for dataset imputed using MICE

Model	Hyperparameters
Logistic Regression	C=0.01, solver=liblinear, penalty=l2
SVC	C=0.1, kernel=sigmoid, gamma=0.001
Decision Tree	criterion=entropy, max_depth=5, max_features=sqrt, splitter=best
Gaussian Naive Bayes	var_smoothing= 10^{-9}
Random Forest	max_depth=None, max_features=sqrt, n_estimators=100

G.1.2 Mean + Mode Imputation

The best ensemble model selected for the dataset imputed using the Mean + Mode algorithm is a voting classifier that combines all 8 individual models as base models (Figure G.2). Table G.2 presents the optimal hyperparameters for each individual model within the ensemble model.

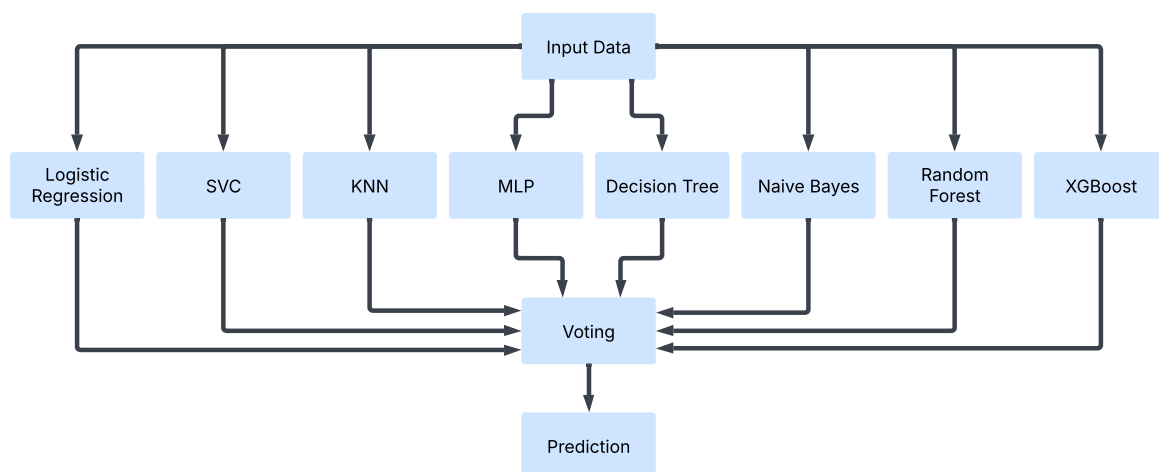


Figure G.2: Best Ensemble Model for Mean+Mode Imputation: Voting

Table G.2: Optimal hyperparameters of Individual Models Models within Ensemble Model selected for dataset imputed using Mean+Mode Method

Model	Hyperparameters
Logistic Regression	C=0.01, solver=saga, penalty=l2
SVC	C=0.01, kernel=sigmoid, gamma=scale
KNN	metric=manhattan, n_neighbors=15, weights=distance

Continued on next page

Table G.2 (continued)

Model	Hyperparameters
MLP	activation=relu, alpha=0.0001, hidden_layer_sizes=(100,50), learning_rate=constant, max_iter=100, solver=adam
Decision Tree	criterion=gini, max_depth=5, max_features=None, split- ter=random
Gaussian Naive Bayes	var_smoothing=0.001
Random Forest	max_depth=None, max_features=log2, n_estimators=200
XGBoost	gamma=0, learning_rate=0.1, max_depth=3, n_estimators=50

G.1.3 Median + Mode Imputation

The best ensemble model selected for the dataset imputed using the Median + Mode method is a voting classifier that combines Logistic Regression, Support Vector Classifier, and Multi-Layer Perceptron as base models (Figure G.2). Table G.2 presents the optimal hyperparameters for each individual model within the ensemble model.

Best Ensemble Model

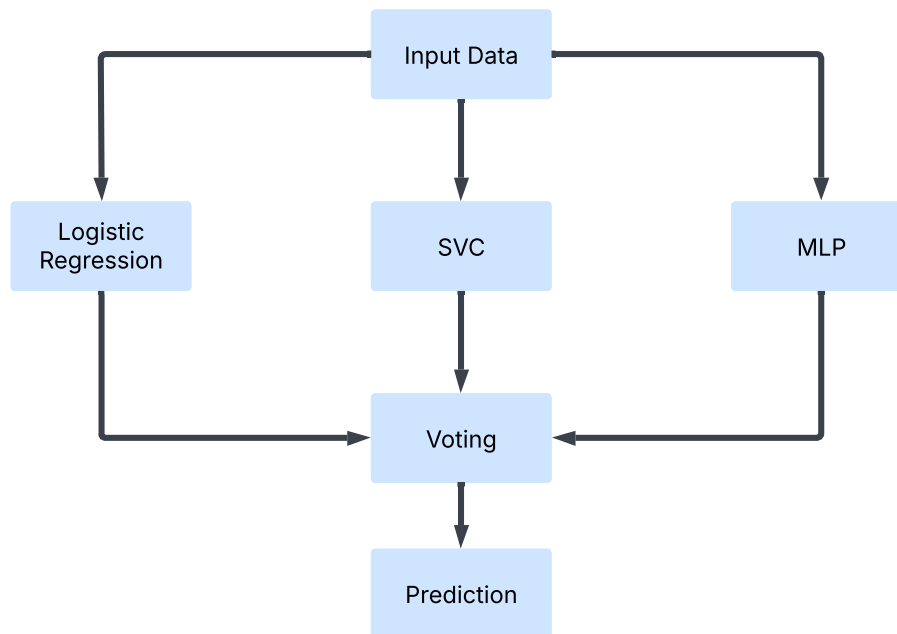


Figure G.3: Best Ensemble Model for Median+Mode Imputation: Voting

Table G.3: Optimal hyperparameters of Individual Models Models within Ensemble Model selected for dataset imputed using Median+Mode Method

Model	Hyperparameters
Logistic Regression	C=0.01, solver=saga, penalty=l2
SVC	C=0.01, kernel=sigmoid, gamma=scale
MLP	activation=tanh, alpha=0.0001, hidden_layer_sizes=(100,50), learning_rate=constant, max_iter=100, solver=adam

G.2 Best Ensemble Models of Class-balanced Datasets

G.2.1 Random Oversampling (ROS)

The best ensemble model selected for the dataset class-balanced using the Random Over-sampling algorithm is a voting classifier that combines Logistic Regression, Support Vector Classifier, k-NN, and Multi-Layer Perceptron as base models (Figure G.4). Table G.4 presents the optimal hyperparameters for each individual model within the ensemble model.

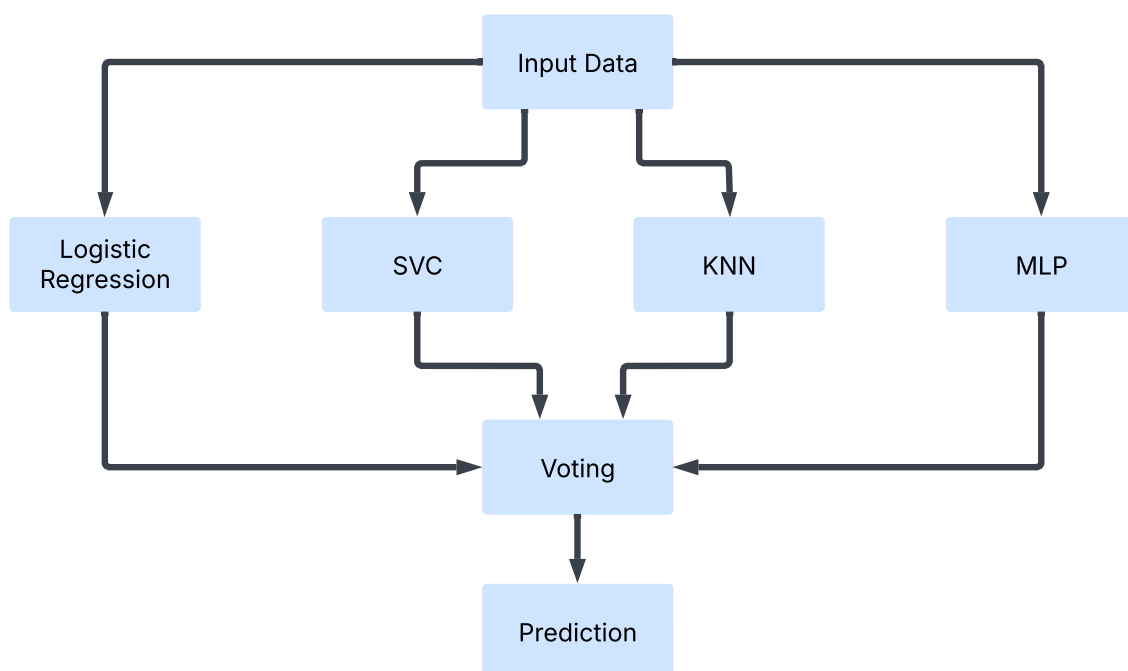


Figure G.4: Best Ensemble Model for dataset class-balanced using ROS Method: Voting

Table G.4: Optimal hyperparameters of Individual Models Models within Ensemble Model selected for dataset class-balanced using ROS Method

Model	Hyperparameters
Logistic Regression	C=0.01, solver=saga, penalty=l2
SVC	C=10, kernel=rbf, gamma=0.1
KNN	metric=euclidean, n_neighbors=21, weights=distance
MLP	activation=relu, alpha=0.0001, hidden_layer_sizes=(50,50), learning_rate=constant, max_iter=100, solver=adam

G.2.2 Adaptive Synthetic Sampling (ADASYN)

The best ensemble model selected for the dataset class-balanced using the ADASYN algorithm is a voting classifier that combines Logistic Regression and Multi-Layer Perceptron as base models (Figure G.5). Table G.5 presents the optimal hyperparameters for each individual model within the ensemble model.

Table G.5: Optimal hyperparameters of Individual Models Models within Ensemble Model selected for dataset class-balanced using ADASYN Method

Model	Hyperparameters
Logistic Regression	C=0.01, solver=saga, penalty=l2
MLP	activation=logistic, alpha=0.0001, hidden_layer_sizes=(50,), learning_rate=constant, max_iter=100, solver=adam

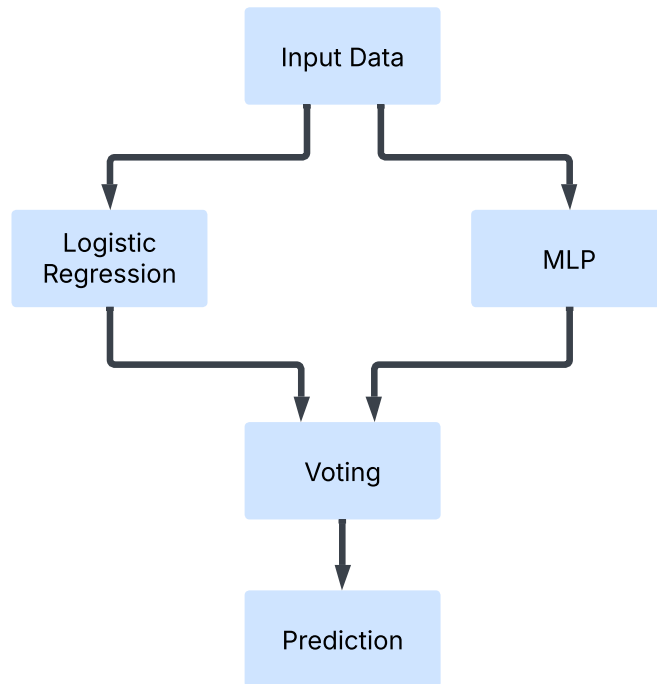


Figure G.5: Best Ensemble Model for dataset class-balanced using ADASYN Method: Voting

G.2.3 Borderline SMOTE (BL-SMOTE)

The best ensemble model selected for the dataset class-balanced using the Borderline SMOTE algorithm is a voting classifier that combines Logistic Regression and Multi-Layer Perceptron as base models (Figure G.6). Table G.6 presents the optimal hyperparameters for each individual model within the ensemble model.

Table G.6: Optimal hyperparameters of Individual Models Models used within Ensemble Model selected for dataset class-balanced using Borderline SMOTE Method

Model	Hyperparameters	
Logistic Regression	C=0.01, solver=saga, penalty=l2	
MLP	activation=logistic, hidden_layer_sizes=(50,50), max_iter=100, solver=adam	alpha=0.001, learning_rate=constant,

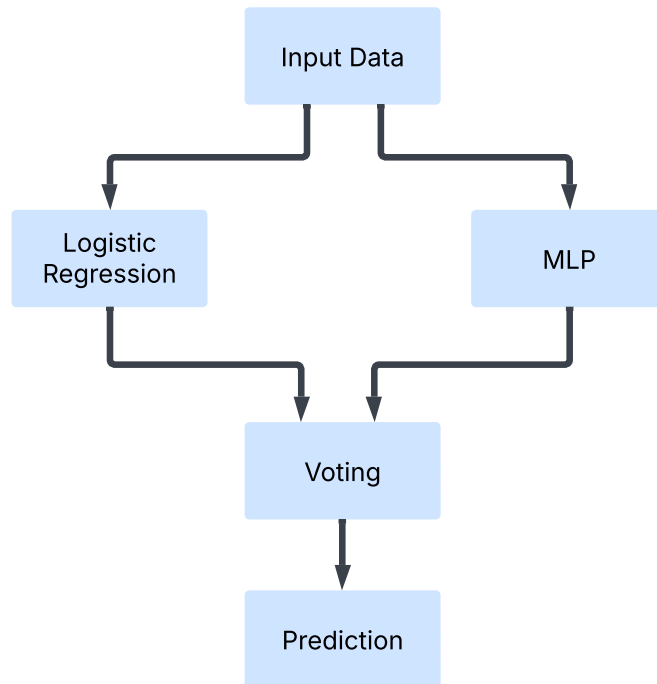


Figure G.6: Best Ensemble Model for dataset class-balanced using Borderline SMOTE Method: Voting

G.2.4 Random Undersampling (RUS)

The best ensemble model selected for the dataset class-balanced using the Random Undersampling algorithm is a stacking classifier that combines Logistic Regression, Support Vector Classifier, and Multi-Layer Perceptron as base models, with Logistic Regression as the final estimator (Figure G.7). Table G.7 presents the optimal hyperparameters for each individual model within the ensemble model.

Table G.7: Optimal hyperparameters of Individual Models Models within Ensemble Model selected for dataset class-balanced using RUS Method

Model	Hyperparameters
Logistic Regression	C=0.01, solver=saga, penalty=l2
SVC	C=0.01, kernel=linear, gamma=scale
MLP	activation=logistic, alpha=0.0001, hidden_layer_sizes=(100,50), learning_rate=constant, max_iter=100, solver=adam

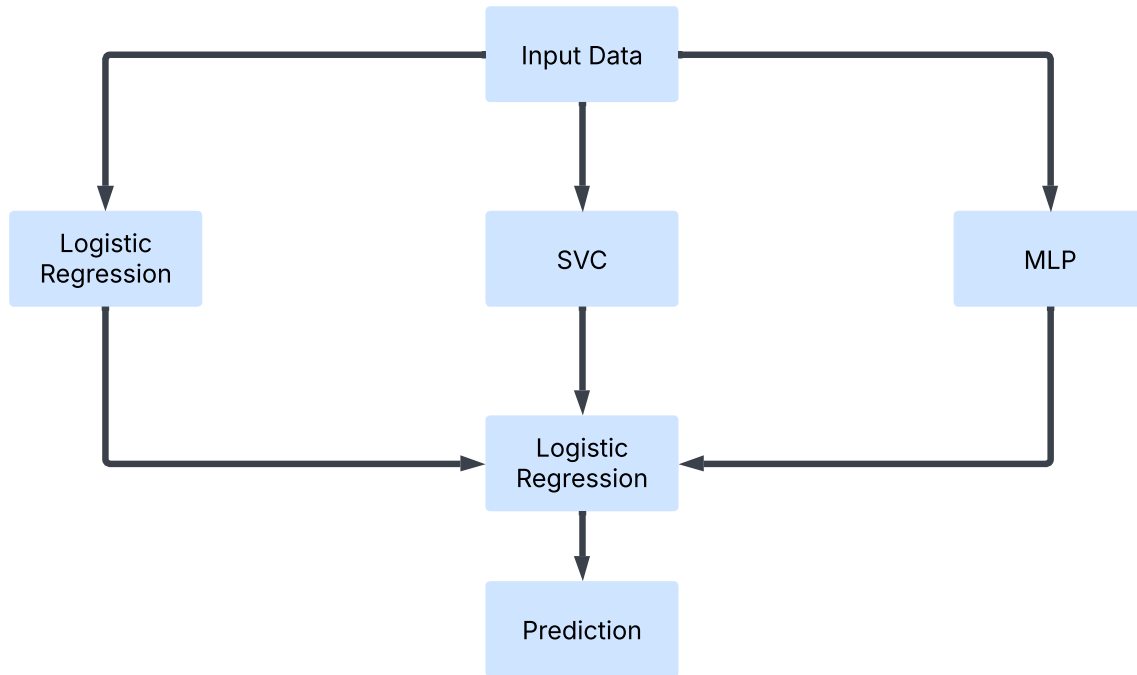


Figure G.7: Best Ensemble Model for dataset class-balanced using RUS Method: Stacking

G.2.5 Clustered Centroids Undersampling (CCUS)

The best ensemble model selected for the dataset class-balanced using the Cluster Centroids Undersampling algorithm is a stacking classifier that combines Logistic Regression, Multi-Layer Perceptron, and Gaussian Naive Bayes as base models, with Logistic Regression as the final estimator (Figure G.7). Table G.7 presents the optimal hyperparameters for each individual model within the ensemble model.

Table G.8: Optimal hyperparameters of Individual Models Models within Ensemble Model selected for dataset class-balanced using CCUS Method

Model	Hyperparameters	
Logistic Regression	C=0.01, solver=saga, penalty=l2	
MLP	activation=logistic, hidden_layer_sizes=(50,50), max_iter=100, solver=adam	alpha=0.0001, learning_rate=constant,
Gaussian Naive Bayes	var_smoothing=0.0001	

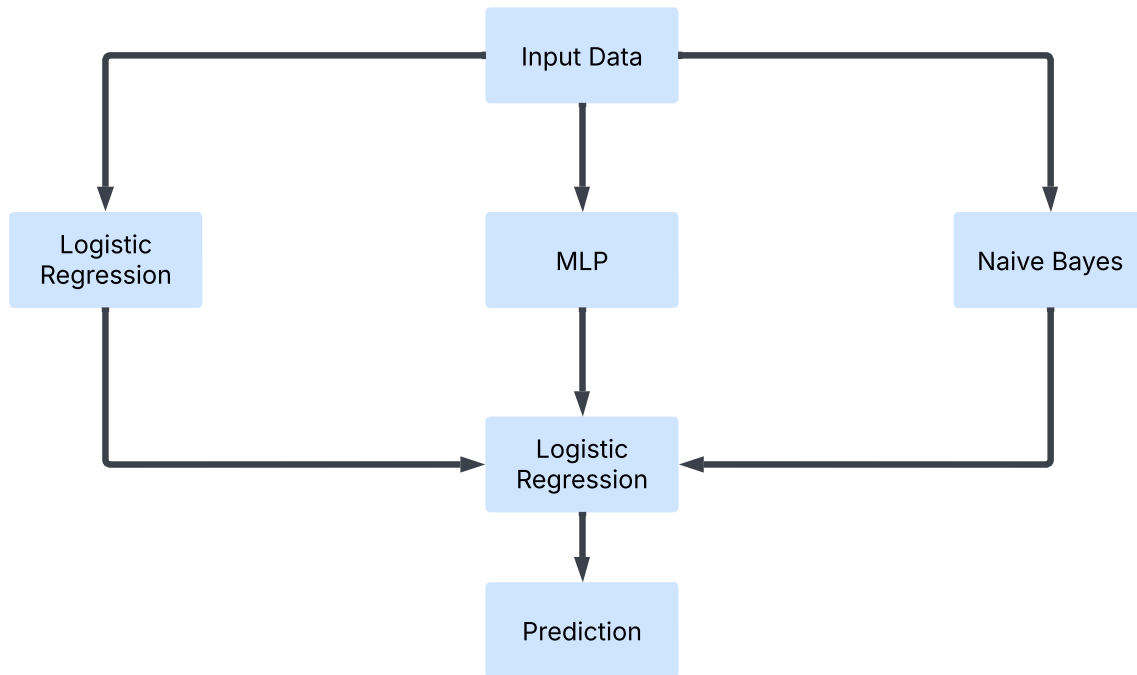


Figure G.8: Best Ensemble Model for dataset class-balanced using CCUS Method: Stacking

G.3 Best Ensemble Models of Manually Selected Feature Groups

G.3.1 Group A - All Features

The best performing model with all features is the ensemble model constructed using the dataset preprocessed with random forest imputation and SMOTE class-balancing (in Section 6.2.2).

G.3.2 Group B - Base Features

The best ensemble model selected for Group B, with Base Features, is a stacking classifier that combines Logistic Regression, k-NN, Multi-Layer Perceptron, and Random Forest as base models, with Logistic Regression as the final estimator (Figure G.9). Table G.9 presents the optimal hyperparameters for each individual model within the ensemble model.

Table G.9: Optimal hyperparameters of Individual Models Models used in Ensemble Model of Base Features

Model	Hyperparameters
Logistic Regression	C=0.01, solver=saga, penalty=l2
KNN	metric=manhattan, n_neighbors=21, weights=distance

Continued on next page

Table G.9 (continued)

Model	Hyperparameters
MLP	activation=tanh, hidden_layer_sizes=(100,50), max_iter=100, solver=adam, alpha=0.0001, learning_rate=constant,
Random Forest	max_depth=10, max_features=sqrt, n_estimators=200

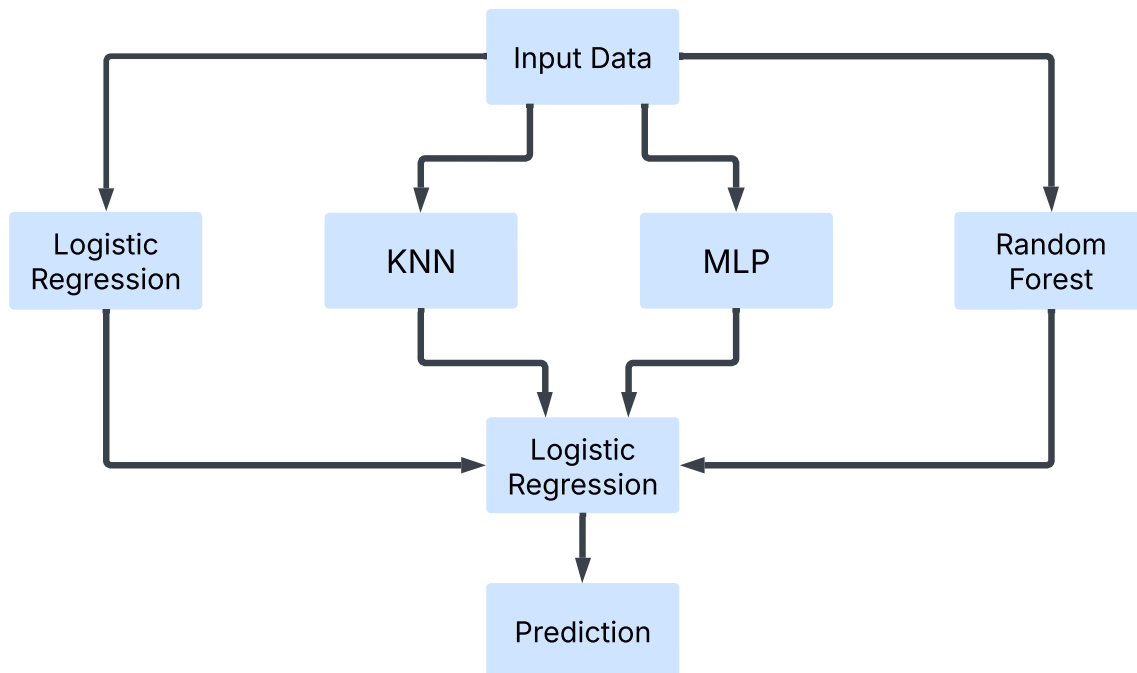


Figure G.9: Best Ensemble Model for Base Features: Stacking

G.3.3 Group C - Base + CSF Features

The best ensemble model selected for Group C, with Base + CSF features, is a voting classifier that combines Logistic Regression, Multi-Layer Perceptron, Gaussian Naive Bayes, Random Forest, and XGBoost as base models (Figure G.10). Table G.10 presents the optimal hyperparameters for each individual model within the ensemble model.

Table G.10: Optimal hyperparameters of Individual Models Models used in Ensemble Model of Base + CSF Features

Model	Hyperparameters
Logistic Regression	C=0.01, solver=liblinear, penalty=l2

Continued on next page

Table G.10 (continued)

Model	Hyperparameters
MLP	activation=tanh, alpha=0.0001, hidden_layer_sizes=(100,), learning_rate=constant, max_iter=100, solver=adam
Gaussian Naive Bayes	var_smoothing=0.001
Random Forest	max_depth=10, max_features=log2, n_estimators=50
XGBoost	gamma=0, learning_rate=0.3, max_depth=6, n_estimators=50

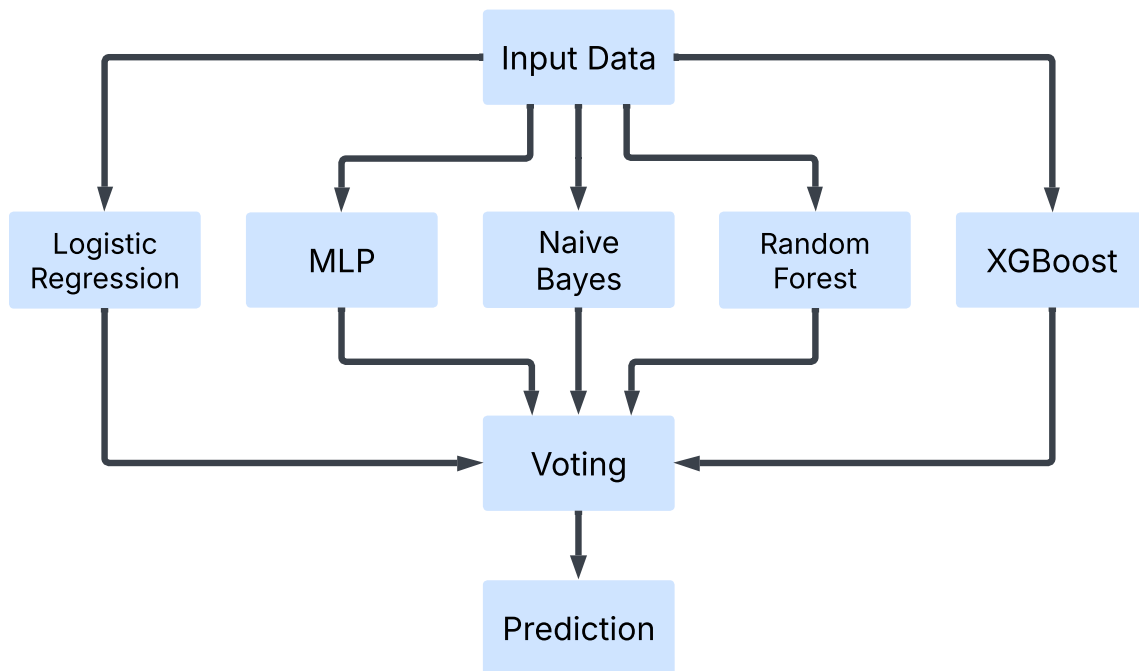


Figure G.10: Best Ensemble Model for Base + CSF Features: Voting

G.3.4 Group D - Base + PET Features

The best ensemble model selected for Group D, with Base + PET features, is a stacking classifier that combines Logistic Regression, Multi-Layer Perceptron, Gaussian Naive Bayes, Random Forest, and XGBoost as base models, with Logistic Regression as the final estimator (Figure G.11). Table G.11 presents the optimal hyperparameters for each individual model within the ensemble model.

Table G.11: Optimal hyperparameters of Individual Models Models used in Ensemble Model of Base + PET Features

Model	Hyperparameters
Logistic Regression	C=1, solver=saga, penalty=l2
Decision Tree	criterion=gini, max_depth=10, max_features=sqrt, splitter=best
Gaussian Naive Bayes	var_smoothing= 10^{-9}
Random Forest	max_depth=None, max_features=log2, n_estimators=100
XGBoost	gamma=0.1, learning_rate=0.3, max_depth=9, n_estimators=50

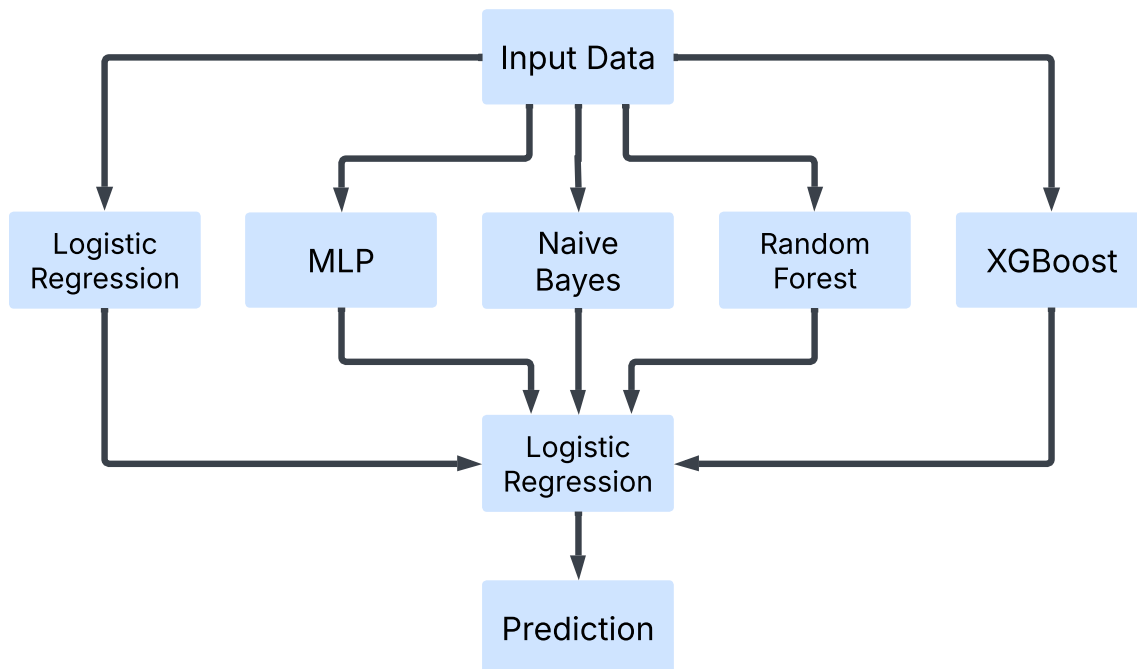


Figure G.11: Best Ensemble Model for Base + PET Features: Stacking

G.3.5 Group E - Base + MRI Features

The best ensemble model selected for Group E, with Base + MRI features, is a voting classifier that combines Logistic Regression and Random Forest as base models (Figure G.12). Table G.12 presents the optimal hyperparameters for each individual model within the ensemble model.

Table G.12: Optimal hyperparameters of Individual Models Models used in Ensemble Model of Base + MRI Features

Model	Hyperparameters
Logistic Regression	C=0.01, solver=saga, penalty=l2
Random Forest	max_depth=None, max_features=log2, n_estimators=200

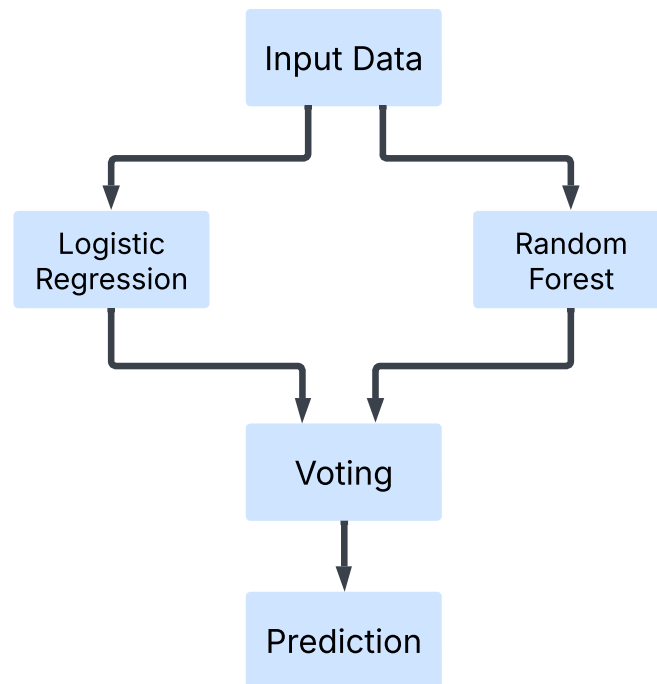


Figure G.12: Best Ensemble Model for Base + MRI Features: Voting

G.3.6 Group F - Base + Plasma Features

The best ensemble model selected for Group F, with Base + Plasma features, is a stacking classifier that combines Logistic Regression, Gaussian Naïve Bayes, Random Forest, and XGBoost as base models, with Logistic Regression as the final estimator (Figure G.13). Table G.13 presents the optimal hyperparameters for each individual model within the ensemble model.

Table G.13: Optimal hyperparameters of Individual Models Models used in Ensemble Model of Base + Plasma Features

Model	Hyperparameters
Logistic Regression	C=0.1, solver=saga, penalty=l2

Continued on next page

Table G.13 (continued)

Model	Hyperparameters
Gaussian Naive Bayes	var_smoothing=0.001
Random Forest	max_depth=10, max_features=log2, n_estimators=100
XGBoost	gamma=0.2, learning_rate=0.3, max_depth=6, n_estimators=50

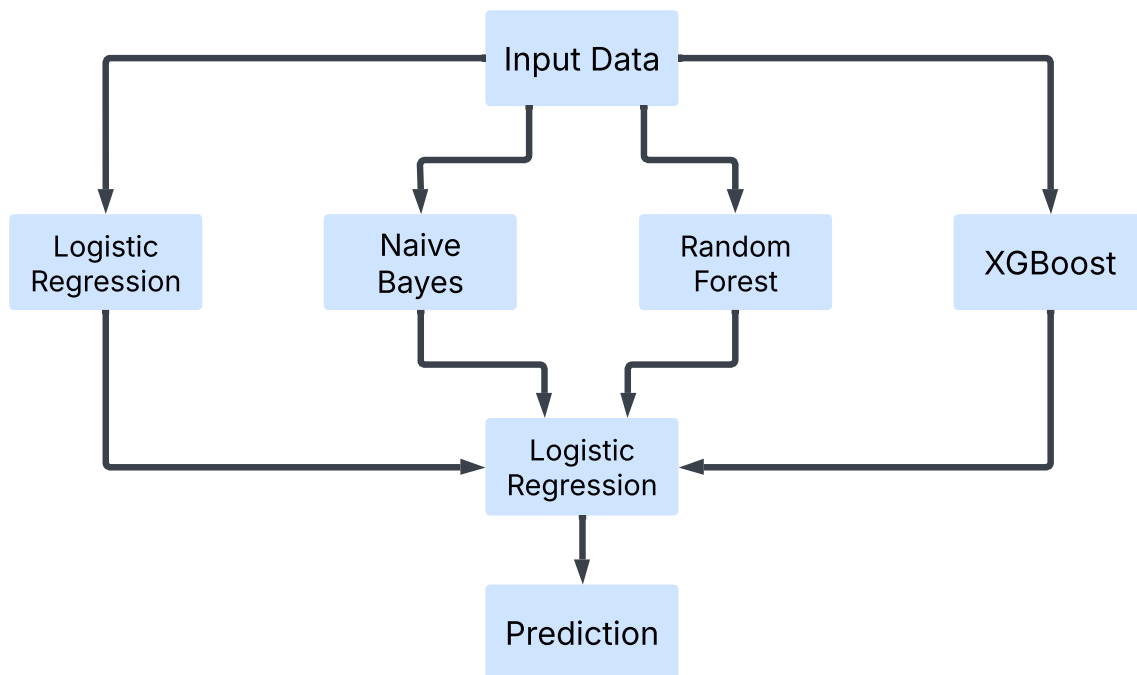


Figure G.13: Best Ensemble Model for Base + PET Features: Stacking

G.3.7 Group G - Base + APOE4 Features

The best ensemble model selected for Group G, with Base + Plasma features, is a voting classifier that combines Logistic Regression, Multi-Layer Perceptron, Gaussian Naive Bayes, Random Forest, and XGBoost as base models (Figure G.13). Table G.13 presents the optimal hyperparameters for each individual model within the ensemble model.

Table G.14: Optimal hyperparameters of Individual Models Models used in Ensemble Model of Base + APOE4 Features

Model	Hyperparameters
Logistic Regression	C=1, solver=saga, penalty=l2

Continued on next page

Table G.14 (continued)

Model	Hyperparameters
MLP	activation=tanh, alpha=0.0001, hidden_layer_sizes=(50,50), learning_rate=constant, max_iter=100, solver=adam
Gaussian Naive Bayes	var_smoothing=0.001
Random Forest	max_depth=10, max_features=sqrt, n_estimators=200
XGBoost	gamma=0.1, learning_rate=0.1, max_depth=9, n_estimators=200

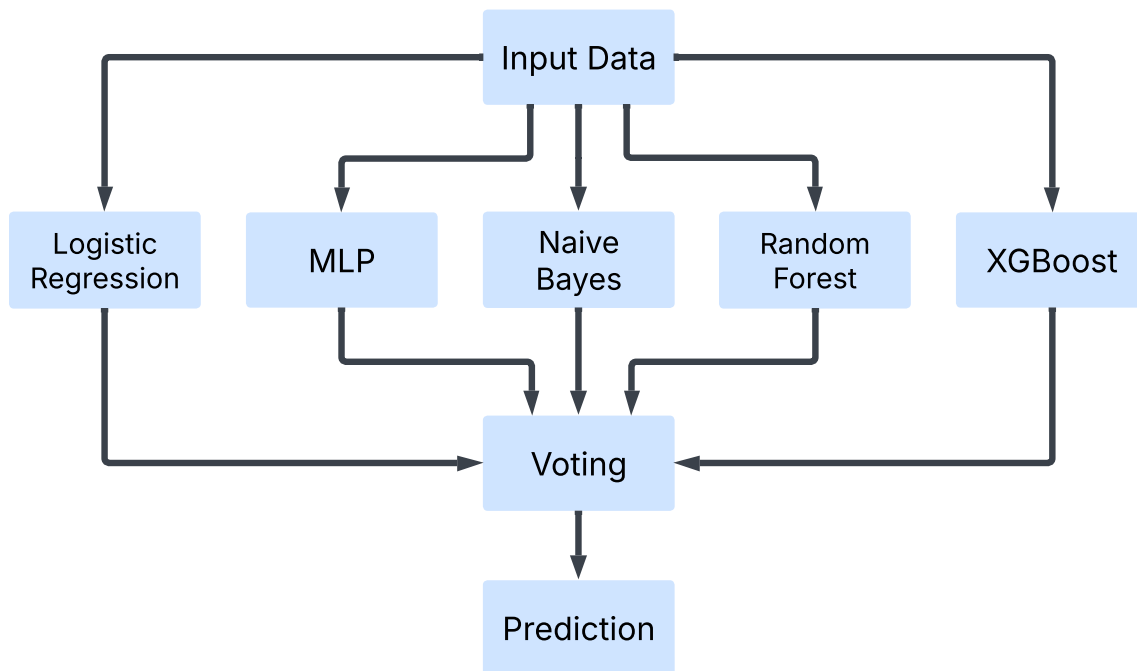


Figure G.14: Best Ensemble Model for Base + APOE4 Features: Voting

G.3.8 Group H - All Features Except MRI and PET Data

The best ensemble model selected for Group H, with all features except MRI and PET data, is a stacking classifier that combines Logistic Regression, Multi-Layer Perceptron, and Gaussian Naive Bayes as base models, with Logistic Regression as the final estimator (Figure G.13). Table G.13 presents the optimal hyperparameters for each individual model within the ensemble model.

Table G.15: Optimal hyperparameters of Individual Models Models used in Ensemble Model of All Features except MRI and PET Data

Model	Hyperparameters
Logistic Regression	C=0.1, solver=liblinear, penalty=l2
Decision Tree	criterion=gini, max_depth=5, max_features=sqrt, splitter=random
Random Forest	max_depth=10, max_features=log2, n_estimators=100

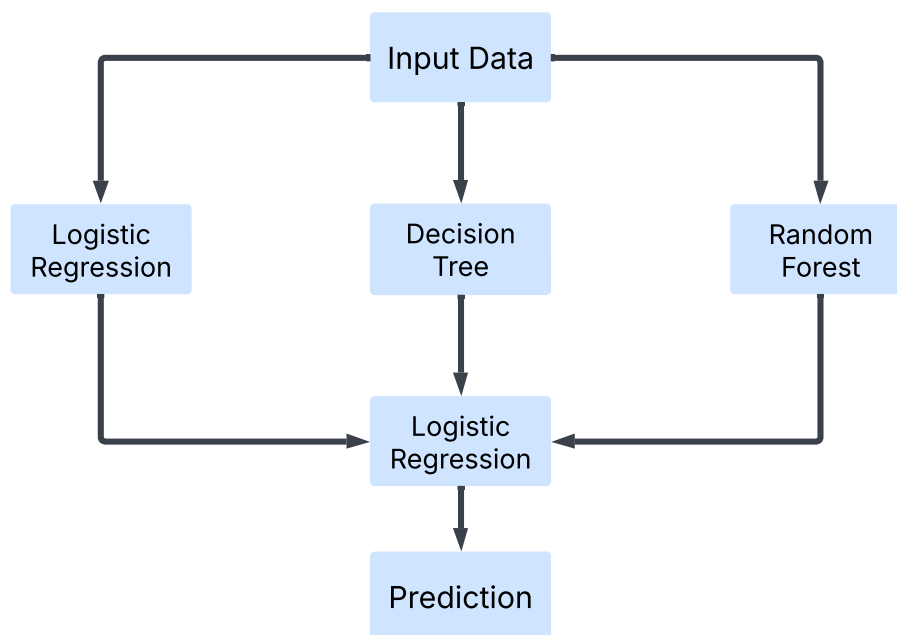


Figure G.15: Best Ensemble Model for All Features except MRI and PET Data: Stacking

G.3.9 Group I - Non-collinear Features selected with Multicollinearity check

The best ensemble model selected for Group H, with features selected with multicollinearity checks, is a stacking classifier that combines Logistic Regression, Decision Tree, and Random Forest as base models, with Logistic Regression as the final estimator (Figure G.13). Table G.13 presents the optimal hyperparameters for each individual model within the ensemble model.

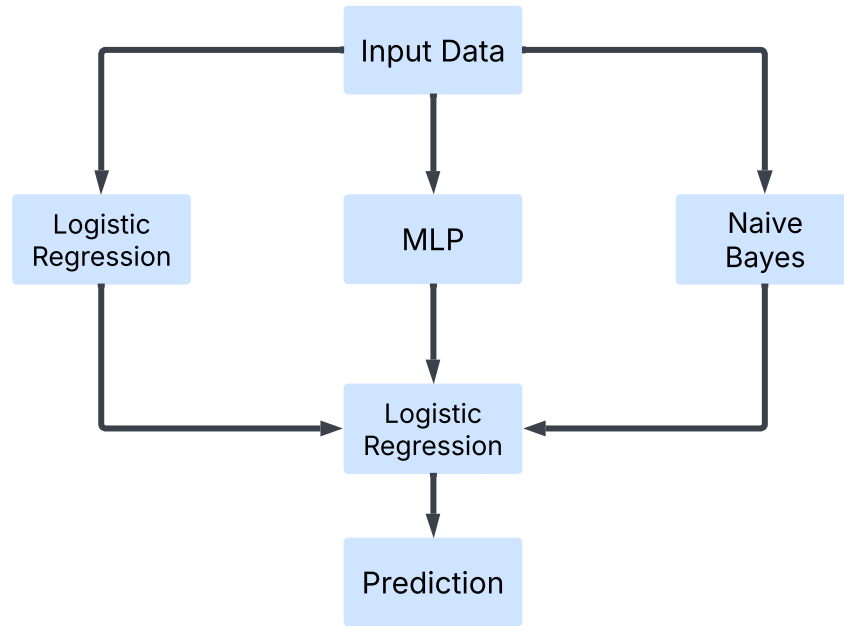


Figure G.16: Best Ensemble Model for Non-collinear Features: Stacking

Table G.16: Optimal hyperparameters of Individual Models Models used in Ensemble Model of Non-collinear Features

Model	Hyperparameters
Logistic Regression	C=0.01, solver=liblinear, penalty=l2
MLP	activation=logistic, alpha=0.01, hidden_layer_sizes=(100,), learning_rate=constant, max_iter=100, solver=adam
Gaussian Naive Bayes	var_smoothing= 10^{-9}